

Skript Digitale Signalverarbeitung

Sebastian Semper – FG Elektrische Messtechnik und Signalverarbeitung – EMS

5. Oktober 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iv
Codeschnipselverzeichnis	v
Linkverzeichnis	vi
1 Theoretische Grundlagen	1
1.1 Komplexe Zahlen	1
1.2 Signale	1
1.2.1 Definition und Typen	1
1.2.2 Signale als Vektoren	2
1.2.3 Transformation von Signalen	3
1.2.4 Zufällige Signale	4
1.2.5 Spezielle Signale	4
1.2.6 Beispiel: LTI-Systeme	5
2 Abtastung von Signalen	7
2.1 Frequenz von Signalen	7
2.1.1 Zeit-Kontinuierliche Sinussignale	7
2.1.2 Zeit-Diskrete Sinussignale	9
2.2 Zeit-Kontinuierliche Komplexe Sinussignale	10
2.3 Zeit-Diskrete Komplexe Sinussignale	11
2.4 Abtastung von Sinussignalen	13
2.5 Das Samplingtheorem	16
3 Diskrete Signale und Systeme	22
3.1 Diskrete Signale	22
3.1.1 Energie Diskreter Signale	22
3.1.2 Periodische Signale	24
3.1.3 Symmetrie von Signalen	25
3.2 Diskrete Systeme	25
3.3 Diskrete LTI-Systeme	31

3.3.1	Faltungsformel	31
3.3.2	Eigenschaften von LTI-Systemen	35
3.3.3	Rekursive IIR-Systeme	36
3.3.4	Approximation von IIR-Systemen durch FIR-Systeme	37
3.4	z-Transformation	40
3.4.1	Eigenschaften	42
3.4.2	Anwendung bei diskreten LTI-Systemen	43
4	Fourier-Transformation	47
4.1	Fourier-Transformation kontinuierlicher Signale	47
4.1.1	Fourier-Transformation kontinuierlicher periodischer Signale	47
4.1.2	Leistungsdichte-Spektrum periodischer Signale	52
4.1.3	Fourier-Transformation von kontinuierlichen aperiodischen Signalen	53
4.1.4	Leistungsdichte-Spektrum aperiodischer Signale	54
4.2	Fourier-Transformation diskreter Signale	56
4.2.1	Fourier-Transformation diskreter periodischer Signale	56
4.2.2	Leistungsdichte-Spektrum diskreter periodischer Signale	57
4.3	Fourier-Transformation diskreter aperiodischer Signale	58
4.3.1	Leistungsdichte-Spektrum diskreter aperiodischer Signale	61
4.4	Eigenschaften der Fourier-Transformationen	61
4.4.1	Beziehung der Fourier-Transformationen und der z-Transformation	61
4.4.2	Weitere Eigenschaften der DFT	64
5	Anwendung der Disreten Fourier-Transformation	69
5.1	Spektralanalyse mit der Short Time Fourier Transformation	69
5.2	Interpolation mittels DFT	75
5.2.1	Theoretische Grundlagen	76
5.2.2	Interpolation von Antennen-Richtcharakteristiken	77
5.3	Schnelle Korrelation für Radar-Signalverarbeitung	82
5.3.1	Zyklische Korrelationen	82
5.3.2	Erzeugung der Sendesequenz	86
5.3.3	MLBS als RADAR-Sendesequenz	87
6	Stochastische Signalverarbeitung	89
6.1	Grundlagen Wahrscheinlichkeitstheorie	89
6.1.1	Unabhängigkeit	90
6.1.2	Diskrete Verteilungen	90
6.1.3	Beispiele	92
6.1.4	Kenngrößen von Zufallsgrößen	93
6.2	Zufällige Signale	95
6.3	Diskrete Stationäre Signale	96
6.4	Quantisierung	98
7	Signalverarbeitung mit B-Splines	107

7.1	B-Splines als Polynome	107
7.2	Kubische B-Spline Interpolation	109
7.3	Verbindung zur Nyquist-Sampling-Theorie	112
Abkürzungsverzeichnis		114
Literaturverzeichnis		115

Abbildungsverzeichnis

1	$x_a(t) = A \cos(2\pi Ft + \theta)$, Quelle: [1]	8
2	Uniforme Abtastung einer Signals. Quelle: [1]	14
3	zeigt zwei verschiedene systemtheoretische Interpretation von linearen Systemen. Quelle: [1]	31
4	Veranschaulichung der Entstehung der ROC aus den Konvergenzkriterien. Quelle: [1]	41
5	Impulsantwort $h[n] = a^n u[n]$ für verschiedene Werte von a , Quelle: [1]	45
6	Mehr Versionen von Listing 15; Oben: $x(t) = \exp(-25(t - 0.5)^2) \cos(16\pi t)$; Unten: $x(t) = \sin(20\pi t)$;	50
7	Schematische Darstellung einer MIMO Kanalmessung. Grafik aus [5].	77
8	Links Oben: Darstellung der Messpositionen für eine Antenne, oder eine Antennenstruktur, welche sich im Ursprung des abgebildeten Koordinatensystems befindet. Rechts Oben: Darstellung der winkelabhängigen Amplitude eines einzelnen Patch-Elements. Unten: Messpunkte für die Abtastung der Funktion a . Grafiken aus [8, 4].	79
9	Messaufbau zur Kalibrierung eines Antennenarrays. Links: Drehteller für die Positionierung der AUT. Rechts: Weiterer Blickwinkel mit Referenzantenne.	80
10	Links: Periodifiziertes 2D Array $a[n_\varphi, n_\theta]$ der gemessenen Amplituden eines einzelnen Patch-Elements. Rechts: Der Betrag der zugehörigen EADF, wobei hier $\mu_1 = k_\varphi$ und $\mu_2 = k_\theta$. Grafik aus [8].	81
11	Lineares Feedback Shift Register, Quelle [1]	86
12	Visualisierung von Area Sampling, Quelle: [10]	99
13	Visualisierung des Effekts von Area Sampling im Frequenzbereich. Quelle: [10]	104
14	Kubische B-Splines für Abtastung an den Werten $n = 0, \dots, 12$	108
15	Kubische B-Spline-Interpolation für Abtastung an den Werten $n = 0, \dots, 12$	111

Codeschnipselverzeichnis

1	Berechnung und Darstellung von (2.1.1)	9
2	Berechnung und Darstellung von (2.1.2)	10
3	Berechnung und Darstellung von (2.3.1) für $f_0 = 1/16$ und $f_0 = 1/(5e)$	12
4	Visualisierung von $F_k = F + k \cdot F_s$	15
5	Berechnung und Darstellung von Theorem 2.1	21
6	Berechnung und Darstellung eines komplexen exponentiellen Signals	23
7	Zerlegung eines Signals in seinen geraden und ungeraden Anteil.	25
8	Spät divergierende Orbits der Mandelbrot-Iteration	27
9	Akkumulator für zwei verschiedene Eingangssignale.	28
10	Gleitendes Mittel mit Länge $\ell = 4$. Wir vergleichen die direkte Berechnung mit der Berechnung über die Faltung	33
11	Verkettung von mehreren Moving Averages der Länge $\ell = 3$	34
12	Die beiden möglichen Implementierungen eines kumulativen Mittelwertes	37
13	Iteratives Verfahren zur Berechnung von $\sqrt{A}$	38
14	Approximation eines exponentiellen Mittelwertes, also eines IIR-Systems, durch ein FIR-System	39
15	Berechnung und Darstellung von (4.1.1)	51

16	Modifiziertes Codeschnipsel 15 und Plot von P im Frequenzbereich.	52
17	Berechnung und Darstellung von (4.1.4)	55
18	Berechnung und Darstellung von (4.2.1)	58
19	Berechnung und Darstellung von (4.3.1)	60
20	Berechnung und Darstellung von (4.4.1)	63
21	Darstellung der DC-Sequenz und der Nyquist-Sequenz für $N = 8$	66
22	Zyklische/periodische Faltung im Zeitbereich und im Frequenzbereich.	67
23	Zero-padding im Zeitbereich einer rect-Funktion liefert eine überabtastung der sinc-Funktion im Frequenzbereich	71
24	Einfluss der Fensterbreite bei der Analyse von harmonischen Schwingungen	72
25	Geschickte Fensterung kann den Dynamikbereich erhöhen	73
26	STFT für verschiedene Fensterlängen	74
27	Analyse einer „Bassline“ mittels STFT, die eine G-Dur-Tonleiter spielt	75
28	Vergleich verschiedener Sendesignale $x[\cdot]$ bezüglich der resultierenden Korrelation $r_{x,y}[\cdot]$. .	85
29	Simulation eines LFSR mittels binären Operationen	87
30	Simulation eines RADAR-Systems basierend auf MLBS, die von LFSR erzeugt wurden. Oben verraushtes Empfangssignal $y[\cdot]$, mitte DFT $Y[\cdot]$, unten $r_{x,y}$	88
31	Visualisierung der Dichte und der Verteilungsfunktion einer diskreten Zufallsgröße	91
32	Visualisierung der Dichte der Normalverteilung für verschiedene Parameter	93
33	Drei Realisierungen von $\sin(2\pi t) + 3/10X_t$, wobei alle $X_t \sim \mathcal{N}(0, 1)$ unabhängig identisch verteilt sind (iid)	95
34	Visualisierung der beiden Sichtweisen auf stochastische Signale.	96
35	Extraktion der spektralen Komponenten eines stationären Signals	98
36	Quantisierung via Area Sampling	100
37	Quantisierung via Area Sampling	102
38	Schätzung der Dichte f_X aus $f_{q(X)}$, die aus Area Sampling gewonnen wurde	106
39	Implementierung von (7.2.1)	109
40	Berechnung der B-Spline Koeffizienten	111

Linkverzeichnis

1	https://erleuchtet.org/2010/07/ridiculously-large-buddhabrot.html . . .	26
2	https://en.wikipedia.org/wiki/Hartman-Grobman_theorem	30
3	https://encyclopediaofmath.org/index.php?title=Newton_method	37
4	https://mathworld.wolfram.com/LaurentSeries.html	40
5	https://de.wikipedia.org/wiki/Cauchyscher_Integralsatz	43
6	https://de.wikipedia.org/wiki/Fundamentalsatz_der_Algebra	44
7	https://de.wikipedia.org/wiki/Satz_von_Parseval	52
8	https://en.wikipedia.org/wiki/Plancherel_theorem	54
9	https://en.wikipedia.org/wiki/Gibbs_phenomenon	59
10	https://docs.scipy.org/doc/scipy/reference/signal.windows.html	73
11	https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.wi ndows.hann.html	73

12	https://mixbutton.com/mixing-articles/music-note-to-frequency-chart/	75
13	https://de.wikipedia.org/wiki/Irreduzibles_Polynom	87
14	https://developer.nvidia.com/gpugems/gpugems2/part-iii-high-quality-rendering/chapter-20-fast-third-order-texture-filtering	113

Allgemeine Hinweise

Die Idee hinter dem Skript zur Vorlesung ist, dass es die Zuhörer der Bürde des Mitschreibens entledigt und Zeit und Platz zum Folgen der Vorlesung frei macht. Das Skript sollte deshalb immer zur Vorlesung und Übung mitgebracht und im Idealfall mit Notizen versehen werden, bzw. zum Nachschlagen verwendet werden.

Eine stetig aktualisierte Version des Skriptes findet man unter <https://github.com/SebastianSemper/lecturenotes>.

Als Begleitmaterial ist auch eine Auswahl an Codeschnipseln bereitgestellt, die einerseits in Auszügen im Skript direkt eingebunden sind, aber auch unter dem obigen Link aufzufinden sind.

Zum Ausführen der Codeschnipsel empfehlen wir ein Python-Environment^a, in welchem folgende Pakete installiert sein sollten:

- numpy – <https://numpy.org>
- scipy – <https://scipy.org>
- matplotlib – <https://matplotlib.org>
- cupy – <https://cupy.dev> (optional für ZOOMG GPU speed)
- sympy – <https://sympy.org> (optional für symbolisches Rechnen in Python)

Diese können ganz einfach via

```
conda create -n dsv python=3.10
conda activate dsv
python -m pip install numpy scipy matplotlib
```

installiert werden.

Alternativ steht unter <https://jup.rz.tu-ilmenau.de/hub/login> ein Jupyter-Hub zur Verfügung, der eine Python IDE im Browser bereitstellt.

^a<https://github.com/conda-forge/miniforge>

1 Theoretische Grundlagen

Digitale Signalverarbeitung ist ein Feld, das sich vieler verschiedener mathematischer Grundlagen bedient, um die gefundenen Zusammenhänge rigoros, knapp und gleichzeitig elegant zu formulieren. Deshalb kommen wir nicht umhin, uns einiger dieser Grundlagen zu erinnern. Alles hier knapp aufgelistete sollte schon bekannt sein und dient nur als bequemes Nachschlagewerk für das kommende Semester.

1.1 Komplexe Zahlen

Die *komplexen Zahlen* $\mathbb{C}$ sind die Menge aller $z = x + jy$, wobei $x, y \in \mathbb{R}$ und für die imaginäre Einheit j gilt, dass $j^2 = -1$. Wir nutzen hier speziell j in Abgrenzung zu i oder j , da diese oft als Indices oder Laufvariablen auftreten. Bei $z = x + jy$ nennen wir $x = \Re(z)$ den Realteil und respektive $y = \Im(z)$ den Imaginärteil. Komplexe Zahlen lassen sich auch in der Polarform $z = r \exp(j\phi)$ darstellen, wobei $r = |z| = \sqrt{\Re(z)^2 + \Im(z)^2} \geq 0$ den Betrag und $\phi = \angle(z) = \arctan(y, x)$ das Argument von z darstellen. Die zu $z = r \exp(j\phi) = x + jy$ komplex konjugierte Zahl ist $z^* = r \exp(-j\phi) = x - jy$.

Komplexe Zahlen haben viele interessante Eigenschaften und Anwendungen, vor allem in der digitalen Signalverarbeitung. Beispielsweise für die Darstellung von einem modulierten reellen Passband Signal $s : \mathbb{R} \rightarrow \mathbb{R}$ dargestellt durch

$$s(t) = x(t) \cos(\omega t) + y(t) \sin(\omega t) \in \mathbb{R},$$

die äquivalente Darstellung im komplexen Basisband

$$s_B(t) = x(t) + jy(t) \quad \text{mit} \quad \Re(s_B(t) \exp(j\omega t)) = s(t). \quad (1.1.1)$$

existiert. Man sagt auch, dass $s_B \exp(j\omega \cdot)$ das analytische Signal zu s darstellt. Dass komplexe Zahlen viele Überraschungen bereithalten sieht man wenn man simuliert und visualisiert für welche $c \in \mathbb{C}$ die Folge

$$z_{n+1} = z_n^2 + c \quad \text{mit} \quad z_0 = c$$

konvergiert oder divergiert (Übung).

1.2 Signale

1.2.1 Definition und Typen

Wir haben gerade schon von Signalen gesprochen, ohne sie etwas genauer einzuführen. Ganz allgemein kann man sich Signale als Objekte vorstellen, die abhängig von Raum, Zeit, oder beidem, physikalische Messgrößen, wie Spannungen, Feldstärken, oder Temperaturen modellieren/abbilden. Es kann aber auch vorkommen, dass das Signal durch gewissen Transformationen inhärent einheitenlos ist.

Die theoretische Darstellung von Signalen erfolgt durch *Funktionen*, die dann noch mit einer impliziten Interpretation versehen werden. Eine Funktion $s : D \rightarrow B$ besitzt einen Namen (s), einen Definitionsbereich (D) und einen Bildbereich (B). Hierbei sind D und B zunächst irgendwelche Mengen. Die Funktion s bildet nun Paare (d, b) zwischen Mengenelementen von $d \in D$ und $b \in B$, indem man schreibt $(d, s(d))$, oder $d \mapsto s(d) = b$. Der Witz ist nun, dass man ein Signal mit physikalischer Bedeutung erhält, indem man lediglich D und B geschickt wählt, sodass diese Mengen zu der Interpretation passen, die man im Hinterkopf hat.

Ist $D = B = \mathbb{R}$ so sprechen wir von einem reellen Signal s und meist denken wir dabei bei D an die Zeitachse, weshalb wir auch $s \mapsto s(t)$, oder einfach $s(\cdot)$ schreiben. Wir werden im Folgenden fast ausschließlich von der „Zeitachse“ sprechen, aber alle Überlegungen lassen sich beispielsweise auf den Raum, oder andere Bereiche übertragen, wie wir beispielsweise in Abschnitt 5.2.2 sehen werden. Ist $D = \mathbb{R}^3$, $B = \mathbb{R}$, so denken wir meist an den dreidimensionalen Raum für den Definitionsbereich und haben also ein Signal im Raum gegeben.

Ist nun jedoch $D = \mathbb{Z}$, $B = \mathbb{R}$, so ist das Signal nur für die ganzen Zahlen $\mathbb{Z}$ definiert, weshalb wir dann mathematisch von einer Folge, oder in unserem Fall von einem *Zeitdiskreten Signal* sprechen. Meist schreiben wir hierfür kurz $s[k] \in \mathbb{R}, k \in \mathbb{Z}$, oder einfach $s[\cdot]$ im Unterschied. Man soll sich hier nicht vorstellen, dass die Werte „zwischen“ den ganzen Zahlen nur fehlen würden. So ist dies *nicht* zu verstehen. Zwischen den gegebenen Werten ist keine Information vorhanden – es gibt also nichts, was „fehlen“ könnte! In manchen Situationen werden wir die diskreten Signale explizit aufschreiben. Dann markieren wir die Stelle $k = 0$ via

$$\dots, 0, 1, 2, \underset{\uparrow}{3}, 2, 1, 0, \dots,$$

um eine bequeme Schreibweise für solche Folgen (diskreten Signale) zu erhalten.

Versuchen sie für möglichst viele verschiedene Kombinationen von D und B Beispiele zu finden (Übung).

1.2.2 Signale als Vektoren

Um mit Signalen gut umgehen zu können, ist es wichtig ihre Eigenschaften als mathematische Objekte zu kennen. Wir müssen Signale also mit „Struktur“ versehen. Wir gehen hier zunächst der Einfachheit halber von $D = B = \mathbb{R}$ aus.

Intuitiv stellt man sich beispielsweise vor, dass man Signale in ihrer Intensität verändern können sollte, und für beliebige Änderung der Intensität wieder ein Signal erhält. Definiert man für $a \in \mathbb{R}$ das Objekt $a \cdot s$ als $t \mapsto a \cdot s(t)$ oder bildet nun die Paare $(t, a \cdot s(t))$ so erhält man wieder ein Signal. Die Werte von s werden also einfach punktweise durch das Skalar a skaliert. Betrachtet man nun zwei Signale s_1, s_2 und definiert $s_1 + s_2$ als $t \mapsto s_1(t) + s_2(t)$, oder $(t, s_1(t) + s_2(t))$, so erhalten wir die Summe oder die Superposition von s_1 und s_2 . Da Signale oft physikalische Messgrößen darstellen, macht dies auch oft Sinn, da in der Physik das Prinzip der Superposition oft eine Rolle spielt. Superposition tritt meist dann auf, wenn die Messgrößen, die wir beobachten, auch als Lösung einer Differentialgleichung gefunden werden können. Bei Differentialgleichungen gilt nämlich oft das Superpositionsprinzip im Lösungsraum, was bedeutet, dass für zwei Lösungen x_1 und x_2 deren Summe $x_1 + x_2$ auch wieder eine Lösung ist.

Wenn wir die beiden Fakten nun kombinieren erhalten wir für $a_1, a_2 \in \mathbb{R}$ und zwei Signale s_1, s_2 und deren Linearkombination $a_1 s_1 + a_2 s_2$, die mit

$$(a_1 s_1 + a_2 s_2)(t) = a_1 s_1(t) + a_2 s_2(t)$$

wieder ein Signal definiert. Objekte, die diese Eigenschaft haben, nennt man *Vektoren* und diese leben in einem *Vektorraum*. Das mag erstmal nicht so schockieren, aber wir gewinnen dadurch *alle* Werkzeuge aus der linearen Algebra für unsere Zwecke.

Beispielsweise können wir nun geschickt Bausteine für eine gewisse Untermenge von Signalen finden, mit denen sich die Signale in dieser Untermenge gut und informativ beschreiben lassen. Wir könnten uns fragen, ob es für den Vektorraum der Bild-Signale eine Basis gibt, sodass für jedes Bild b eine Darstellung existiert, dass

$$b(x, y) = c_1 b_1(x, y) + c_2 b_2(x, y) + \dots, \quad (1.2.1)$$

gilt. Die Zahlen $c_1, c_2, \dots$ können also das Signal b darstellen, indem man einfach die Elemente aus der Basis hennimmt, entsprechend skaliert und summiert. Das dürfen wir, weil auch die Basisvektoren, eben *Vektoren* sind. In gewisser Weise *sind* die Koeffizienten c_i das Signal b . Vielleicht gelingt es uns, die Menge $\{b_1, b_2, \dots\}$ so zu konstruieren, dass wir immer nur *wenige* von diesen b_i brauchen, sodass wir *jedes beliebige* Bild aus einer Fotokamera durch geschickte Kombination von diesen $\{b_1, b_2, \dots\}$ darstellen können (*funnyMP3noises*)?

1.2.3 Transformation von Signalen

Noch interessanter ist aber die Manipulation von Signalen durch *Transformationen*. Der Sinn von Transformationen ist es, neue oder einfach bestimmte Einsichten in ein Signal zu gewinnen. Es kann aber auch sein, dass man Operationen, die auf Signalen ausgeführt werden sollen, „einfach“ mittransformieren kann. Vielleicht ist die gewünschte Operation nach Transformation deutlich einfacher anzuwenden? Jede Transformation liefert hierbei andere Informationen oder ist für andere Signale definiert.

Mathematisch ist eine Transformation nichts anderes als eine Abbildung zwischen Signalen. Definieren wir uns zwei Mengen von Signalen $\mathcal{S}$ und $\mathcal{T}$. Dann bildet die Transformation $T : \mathcal{S} \rightarrow \mathcal{T}$ Paare zwischen Objekten aus $\mathcal{S}$ und $\mathcal{T}$, also $s \in \mathcal{S} \mapsto Ts \in \mathcal{T}$. Nach Anwendung der Transformation T auf s erhalten wir also ein anderes Signal Ts . Gerade haben wir schon festgestellt, dass man Signale beliebig skalieren und addieren kann und es als eine Art grundlegende Eigenschaft von Signalen festgehalten. Nehmen wir nun ein Signal mit Werten

$$s(t) = a_1 s_1(t) + a_2 s_2(t)$$

und wir wenden die Transformation T auf beiden Seiten der Gleichung an

$$\{Ts\}(t) = \{T(a_1 s_1 + a_2 s_2)\}(t).$$

Ist nun die Transformation so, dass wir für beliebige $a_{1,2}$ und $s_{1,2}$ immer

$$\{Ts\}(t) = \{T(a_1 s_1 + a_2 s_2)\}(t) = a_1 \{Ts_1\}(t) + a_2 \{Ts_2\}(t) \quad (1.2.2)$$

schreiben können, so nennen wir T eine *lineare* Transformation. Zusammen mit der Superpositionseigenschaft von Signalen sieht man nun, warum Linearität so wichtig für Transformationen ist, weil es einfach zur Vektorraumstruktur von Signalen passt. Die Linearität erlaubt es uns beispielsweise auch das obige Signal b in (1.2.1) ganz einfach zu transformieren. Nehmen wir es in seiner Darstellung als

$$b(x, y) = c_1 b_1(x, y) + c_2 b_2(x, y) + \dots,$$

und wir haben eine beliebige lineare Transformation T , deren Effekt wir auf b angewendet sehen wollen. Wir suchen also $\{Tb\}(x, y)$. Aber das ist mit der Linearität ganz einfach. Wir müssen nur Tb_i kennen, also die Wirkung von T auf die Basisvektoren b_i , denn

$$\{Tb\}(x, y) = c_1 \{Tb_1\}(x, y) + c_2 \{Tb_2\}(x, y) + \dots,$$

ist eine valide Darstellung von Tb . Cool!

Beispiele für solche linearen Transformationen sind Differentiation (falls möglich), bilden der Stammfunktion (falls möglich), Verzögerung eines Zeitsignals um Zeit $a \in \mathbb{R}$, Stauchung und Streckung in eines Zeitsignals, Rotation eines Bildes, die Fourier-Transformation, die diskrete Fourier-Transformation, zyklische Faltung, oder Korrelation mit einem anderen Signal p . Gegenbeispiele sind $p(t) = \sin(s(t))$, oder $p(t) = (s(t))^\alpha$ für $\alpha \neq 1$, oder $s * s$, wobei $*$ die Faltung darstellt.

Man sieht, dass viele wichtige Operationen lineare Transformationen darstellen und wir haben mit linearer Algebra ein mächtiges Tool an unserer Seite, um mit ihnen umzugehen.

1.2.4 Zufällige Signale

Man kann auch noch eine weitere Sichtweise auf Signale haben. In manchen Fällen ist es nicht zweckmäßig, dass man ein Signal s als vollständig bekannte und fixe Funktion modelliert. Stattdessen modelliert man den Wert $s(t)$ des Signals s an der Stellen t als *Zufallsgröße*. Das heißt, dass der Wert $s(t)$ einer Verteilung $X(t)$ folgt. An jedem Zeitpunkt t „hängt“ eine solche Verteilung, die bestimmt mit welcher Wahrscheinlichkeit die Werte $s(t)$ in einem gewissen Intervall liegen. Man spricht in diesem Fall auch von *stochastischen* Signalen, im Gegensatz zu den obigen *deterministischen* Signalen.

Es kann verschiedene Gründe haben, dass man ein Signal nicht mehr deterministisch beschreiben kann/will/sollte:

- Sobald die Werte von s durch eine Messung entstanden sind, enthalten diese normalerweise Messrauschen. Dann modelliert man s meistens als Summe

$$s(t) = x(t) + n(t),$$

wobei $n(t) \sim \mathcal{N}(0, \sigma^2(t))$ meist als eine Realisierung einer mittelwertfreien Normalverteilung mit Varianz $\sigma^2(t)$ angenommen wird, und x als ein deterministisches Signal.

- Wenn man generell nicht genug Information über das Signal hat, beispielsweise, kennt man nur dessen Verteilung im Frequenzbereich, also Wahrscheinlichkeiten, dass gewisse Frequenzen vorhanden sind, oder nicht. Dennoch ist man natürlich an dem Verhalten des Signals im Zeitbereich interessiert.
- Wenn es für die Anwendung nicht notwendig ist. Dies kann der Fall sein, wenn man einen Filter entwickelt, der eine gewisse Klasse von Signalen als Eingang bekommt, kann es reichen die Verteilung der Signale zu kennen und dann den Ausgang des Filters nur stochastisch zu beschreiben.

„In 99.99 % der Fälle ist der nachgeschaltete Verstärker nicht übersteuert.“

Für solche Aussagen ist es sogar *notwendig* die Verteilung der Eingangssignale zu kennen, ansonsten ist so eine Aussage gar nicht möglich, da man eben keine Verteilung für ein deterministisches Signal angeben kann.

Um stochastische Signale korrekt handhaben zu können, ist einige Mathematik notwendig, die wir in Abschnitt 6 lediglich skizzieren werden und stattdessen versuchen ein *intuitives* Verständnis zu entwickeln.

1.2.5 Spezielle Signale

Uns werden immer wieder einige spezielle Signale begegnen, die wir hier kurz auflisten wollen.

- Die *Delta-Funktion* (*Dirac- δ*) als Funktional (eine Abbildung, die Paare von Funktionen und Skalaren bildet) δ , das angewendet auf ein Signal s , immer den Wert $s(0)$ liefert. Visualisiert wird dieses *nicht*-Signal, durch einen Impuls der Höhe 1 bei $t = 0$. Es ist nicht ohne Ironie, dass eines der wichtigsten Objekte der Signalverarbeitung selbst kein Signal ist, jedoch wie eines behandelt wird, aber dann immer mit Vorsicht.

- Die *Heavyside-Funktion* $u : \mathbb{R} \rightarrow \mathbb{R}$ mit

$$u(t) = \begin{cases} 1 & \text{für } t > 0 \\ \frac{1}{2} & \text{für } t = 0 \\ 0 & \text{für } t < 0. \end{cases}$$

Man kann δ als distributionelle Ableitung von u auffassen.

- Die *komplexe Schwingung* $s : \mathbb{R} \rightarrow \mathbb{C}$ bei Frequenz $f \in \mathbb{R}$ ist definiert als $s_f(t) = \exp(jft)$ und wir uns im Verlauf des Semesters noch einige Male begegnen. Beispielsweise gelten $s_f^*(t) = s_f(-t)$ und $s_{-f}(t)^* = s_f(t)$.

- Der *diskrete δ -Stoß* $\delta[k]$ ist definiert als

$$\dots, 0, \underset{\uparrow}{1}, 0, \dots$$

- Endliche, diskrete Signale können wir entweder durch

$$s = [0, 1, 2, \underset{\uparrow}{3}, 2, 1, 0]$$

darstellen, oder als endliche Summe von einigen diskreten δ -Stößen:

$$s[n] = \sum_{k=-2}^{k=+2} s[k] \delta[n-k]$$

1.2.6 Beispiel: LTI-Systeme

Wir werden uns zwar noch später ausführlich mit **Linear Time-Invariant (LTI)** Systemen beschäftigen, doch sie sollen hier schon als nicht-triviales Beispiel dienen. Wir sind also mit einem System $\mathcal{H}$. Ein System ist ein Objekt, das Eingangssignale auf Ausgangssignale abbildet, also Paare von Funktionen/Signalen $(x, \mathcal{H}x)$ oder (x, y) bildet.

Weiterhin hat $\mathcal{H}$ die Eigenschaft, dass für Anregungen $x : \mathbb{R} \rightarrow \mathbb{R}$ eine Verschiebungsinvarianz mit $y(t - \tau) = (\mathcal{H}x(\cdot - \tau))(t)$ gilt – es ist *zeitinvariant*. Außerdem ist $\mathcal{H}$ *linear* wie in (1.2.2) dargestellt.

Dann kann man die Wirkung von $\mathcal{H}$ auch durch Faltung mit der sog. Impulsantwort h des Systems darstellen, also

$$y(t) = (\mathcal{H}x)(t) = \int_{-\infty}^{+\infty} x(t - \tau) h(\tau) d\tau = (x * h)(t), \quad (1.2.3)$$

wobei $h = \mathcal{H}\delta$, also die Reaktion des Systems auf einen Dirac-Stoß darstellt. An dieser Darstellung sieht man sehr gut, dass das System $\mathcal{H}$ linear ist, weil die Integration linear in x ist.

Natürlich ist der Zeitbereich für diese Art von System nicht der richtige Anschauungsort. Nach Laplace-Transformation von y zu $Y = \mathcal{L}y$ sehen wir, dass wir dort stattdessen

$$Y(s) = X(s) \cdot H(s),$$

schreiben können. Hierbei sind $X = \mathcal{L}x$ und $H = \mathcal{L}h$ die Laplace-Transformationen des Eingangs und der Impulsantwort h . Nicht nur hat sich die „Berechnung“ von Y vereinfacht, sondern wir haben auch ein

besseres Gefühl für das Verhalten des Systems in Abhängigkeit von h , bzw. H , weil der Einfluss einfach multiplikativ ist.

Wir können die lineare Algebra noch ein wenig weiter treiben. Betrachten wir als Eingang die Funktion $x_s(t) = \exp(st)$ für ein beliebiges $s \in \mathbb{C}$. Dann rechnen wir einfach mit (1.2.3) nach, dass

$$(H \exp(s \cdot))(t) = \int_{-\infty}^{+\infty} \exp(s(t - \tau)) h(\tau) d\tau = \exp(st) \int_{-\infty}^{+\infty} \exp(-s\tau) h(\tau) d\tau = \exp(st) H(s),$$

gilt. Das heißt, dass die Funktionen $\exp(s \cdot)$ die *Eigenvektoren* des Operators $\mathcal{H}$ sind, denn es gilt

$$(\mathcal{H}x_s)(t) = x_s(t) \cdot H(s),$$

wobei H die Laplace-Transformation von h ist. Das heißt auch, dass $H(s)$ die zugehörigen *Eigenwerte* sind¹. Wir sehen hier also, dass Signale *wirklich* wie Vektoren funktionieren können und es sich im Fall von linearen System förmlich aufzwingt, da die Linearität des Systems zur linearen Vektorraumstruktur „passt“.

¹<https://imgflip.com/memetemplate/16005384/mind-blown>

2 Abtastung von Signalen

Als ersten Schritt der digitalen Signalverarbeitung wollen wir uns den Übergang von einem analogen Signal zu einem digitalen näher ansehen. Intuitiv können wir diesen Vorgang in vielen Anwendungen beobachten. Wir nehmen im Tonstudio mit einem Mikrofon Ton auf und eine Soundkarte wandelt das analoge Signal in einen zeitdiskreten und quantisierten WAV-Datenstrom um. In einer Fotokamera, trifft ein Feld von Lichtstrahlen ein und wird von einem **Complementary Metal Oxide Semiconductor (CMOS)**-Sensor „direkt“ abgetastet und in diskrete Helligkeitswerte pro Farbkanal umgewandelt. Eine Antenne wandelt ein anliegendes elektro-magnetisches Feld in eine Spannung um, welche nachträglich von einem **Analog-to-Digital Converter (ADC)** abgetastet und quantisiert wird.

Mathematisch modellieren wir analoge Signale $x_a : D \rightarrow B$ meist mit $D = B = \mathbb{R}$. Die Wandlung von analog zu digital transformiert dieses Signal in eine Funktion $x : \mathbb{Z} \rightarrow Q$ um, wobei nun $|Q| < \infty$ gilt. Das heißt, dass das Signal nach AD-Wandlung nur noch endlich viele Werte (aus Q) auf einer diskreten Menge an Punkten (auf $\mathbb{Z}$) annehmen kann. Meistens folgt schließlich noch ein Kodierungsschritt, der die Werte in Q in eine Folge von Bits transformiert. Wir werden uns zunächst nur mit der Diskretisierung in Zeit befassen, weil es einfach einfacher ist und die Quantisierung später in Abschnitt 6.4 behandeln. Das heißt, dass wir uns vorstellen, dass das digitalisierte Signal nur an einer diskreten Menge von Punkten Informationen über das abgetastete Signal beinhaltet, wobei hier aber explizit $Q = \mathbb{R}$ gesetzt wird, wir also keine Quantisierung betrachten. Weiterhin sind wir nicht an der physikalischen Umsetzung von **ADCs** interessiert, sondern höchstens an deren systemtheoretischer Modellierung.

Die zentralen Fragen sind nun:

- Wie muss der Vorgang der Abtastung gestaltet sein, dass keine Information verloren geht?
- Wie können wir die Eigenschaften des analogen Signals in dessen abgetasteter Version wiederfinden?
- Wie können wir eine Intuition für den Vorgang der Abtastung entwickeln?

2.1 Frequenz von Signalen

2.1.1 Zeit-Kontinuierliche Sinussignale

Meistens werden wir uns in der Vorlesung mit reell- oder komplexwertigen Zeitsignalen befassen, d.h. wir modellieren unsere Signale als $x_a : \mathbb{R} \rightarrow \mathbb{R}$ oder $x_a : \mathbb{R} \rightarrow \mathbb{C}$. Wobei physikalische Signale natürlich nur reellwertig sind, doch manchmal ist die Darstellung als komplexwertige Funktion besser handhabbar, siehe (1.1.1). Das heißt, dass die Abtastung im Zeitbereich vonstatten geht, was sofort den Begriff der *Frequenz* auf den Plan ruft, da Frequenz mit Einheit $1/[s]$ eng mit Zeit $[s]$ verknüpft ist.

Betrachten wir also erst einmal, welchen Einfluss Abtastung auf Signale mit einzelnen Frequenzen hat, am Beispiel von

$$x_a(t) = A \cos(\Omega t + \theta), \quad (2.1.1)$$

wobei wir hier $A \in \mathbb{R}$ als Amplitude, $\Omega \in \mathbb{R}_0^+$ als Kreisfrequenz 1 rad s^{-1} , $t \in \mathbb{R}$ als Zeit 1 s und die Phase $\theta \in \mathbb{R}$ mit Einheit 1 rad nutzen. Alternativ können wir auch zur Frequenz $F \in \mathbb{R} \text{ s}^{-1} = 1 \text{ Hz}$ übergehen. Dann erhalten wir

$$x_a(t) = A \cos(2\pi F t + \theta).$$

Diese Funktion ist in Abb. 1 dargestellt. Wir sehen, dass die Funktion periodisch ist mit Periode $T_p = 1/F$.

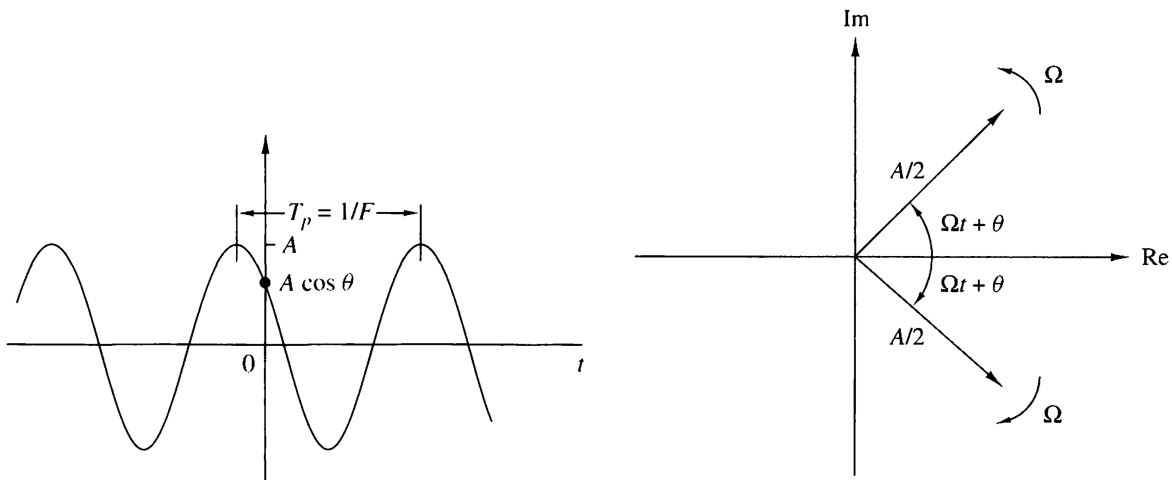


Abbildung 1: $x_a(t) = A \cos(2\pi Ft + \theta)$, Quelle: [1]

Das heißt, dass $x_a(t + k \cdot T_p)$ für $k \in \mathbb{Z}$ nicht vom Signal $x_a(t)$ zu unterscheiden ist! Also gilt

$$x_a(t) = x_a(t + k \cdot T_p) \quad \text{für alle } k \in \mathbb{Z}, t \in \mathbb{R}.$$

Es gibt noch eine alternative Darstellung von der obigen Funktion durch die Addition von zwei zueinander konjugierten *Phasoren* als

$$x_a(t) = A \cos(2\pi Ft + \theta) = \frac{A}{2} \exp(j(\Omega t + \theta)) + \frac{A}{2} \exp(-j(\Omega t + \theta)).$$

Da die beiden überlagerten Phasoren so interpretiert werden können als rotierten diese in gegensätzliche Richtungen, ist es gerechtfertigt der physikalischen Intuition entgegen auch von „negativen“ Frequenzen zu sprechen. Wir erlauben also $F \in \mathbb{R}$, womit auch der Spezialfall $T_p = \infty$, also $x_a(t) = A$ abgedeckt ist. Ein kleines Beispiel findet man in Codeschnipsel 1.

```

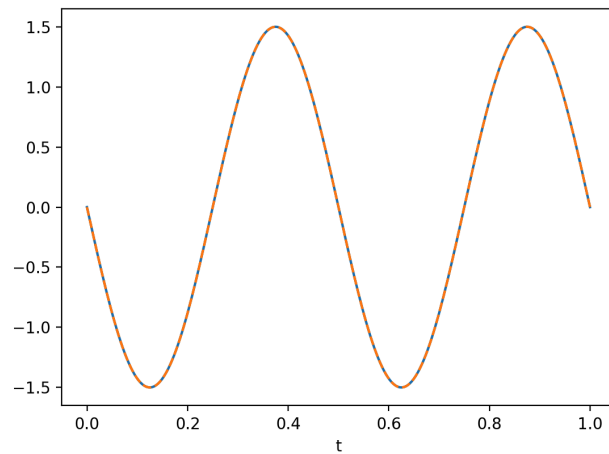
A = 1.5
F = 2
theta = np.pi / 2

def harm(t: float) -> float:
    return A * np.cos(
        2 * np.pi * (F * t + theta))

def phasor(t: float) -> complex:
    return 0.5 * A * np.exp(
        -1j * (2 * np.pi * (F * t + theta)))

T = np.linspace(0, 1, 255)
plt.plot(T, harm(T), label="harm(T)")
plt.plot(T, phasor(T) + phasor(T).conj(),
         linestyle="--", label="harm(T)")
plt.xlabel("t")
plt.show()

```



Codeschnipsel 1: Berechnung und Darstellung von (2.1.1), siehe [dsv/code/cont_harms.py](#)

2.1.2 Zeit-Diskrete Sinussignale

Als nächstes gehen wir zu dem eigentlich interessanten Fall über, bei welchem wir von zeitdiskreten harmonischen Signalen sprechen. Dabei gehen wir vorerst *nicht* davon aus, dass das Signal durch Abtastung eines analogen/stetigen Signals entstanden ist, sondern betrachten es ganz losgelöst für sich. In Analogie zu (2.1.1) definieren wir

$$x[n] = A \cos(\omega n + \theta) = A \cos(2\pi f n + \theta). \quad (2.1.2)$$

Wichtig bei diskreten Signalen ist, dass ihre physikalische Interpretierbarkeit nicht direkt gegeben ist, da $n \in \mathbb{Z}$ nur die diskreten Werte „nummeriert“, also *einheitenlos* ist. Deshalb hat $f \in \mathbb{R}$ lediglich als Einheit „Zyklen pro Sample“, was man auch daran sieht, dass für $f = 1$ gilt $x[n] = A \cos(2\pi n + \theta) = A \cos(\theta)$. Es existiert nun ein wichtiger Unterschied zwischen $x[\cdot]$ von (2.1.2) und $x_a(\cdot)$ von (2.1.1). Das Signal $x[\cdot]$ ist nur periodisch, falls f eine rationale Zahl ist, also $f = p/q$ für $p, q \in \mathbb{Z}$ und $q \neq 0$. Wieso?

Ein zeitdiskretes Signal ist periodisch, falls $x[n + N] = x[n]$ für alle $n \in \mathbb{Z}$. Für unser Signal in (2.1.2) heißt das also, dass

$$\cos(2\pi f n + \theta) = \cos(2\pi f(n + N) + \theta) = \cos(2\pi f n + 2\pi f N + \theta)$$

Da $\cos$ Periode $2\pi k$ für $k \in \mathbb{Z}$ besitzt, muss $2\pi f N = 2\pi k$ gelten, also

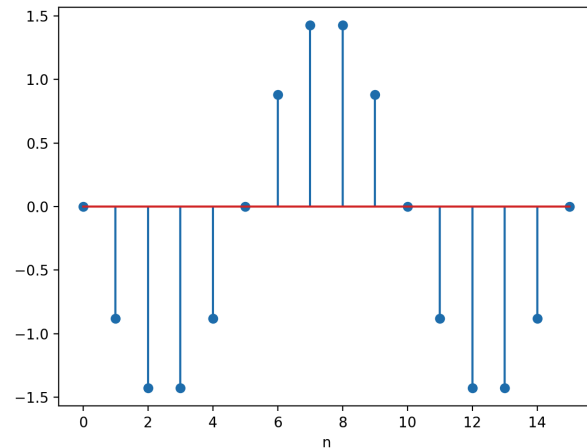
$$f = \frac{k}{N}.$$

Andersherum kann man die kleinste Periode N ermitteln, indem man $f = k/N$ vollständig kürzt, sodass also Zähler und Nenner keine gemeinsamen Teiler mehr haben, und man dann den Nenner des resultierenden Bruches als N setzt. Ein Beispiel wird in Codeschnipsel 2 gezeigt. Man kann interessante Ergebnisse erzielen, wenn man diesen Plot für $\omega = 0, \pi/8, \pi/4, \pi/2$ und $\omega = \pi$ erzeugt (Übung).

```
A = 1.5
f = 0.1
theta = 0.25
```

```
def harm(n: int) -> float:
    return A * np.cos(
        2 * np.pi * (f * n + theta))
```

```
n = np.arange(16)
plt.stem(n, harm(n), label="harm[n]")
plt.xlabel("n")
plt.show()
```



Codeschnipsel 2: Berechnung und Darstellung von (2.1.2), siehe [dsv/code/disc_harms.py](#)

Die Signale der Form (2.1.2) haben noch eine weitere wichtige Eigenschaft, die sich wieder aus der 2π -Periodizität von $\cos$ ergibt. Betrachten wir noch einmal (2.1.2) und wir finden, dass

$$\cos(\omega n + \theta) = \cos(\omega n + 2\pi n + \theta) = \cos((\omega + 2\pi)n + \theta).$$

Das heißt, dass

$$x[n] = \cos(\omega n + \theta) = \cos((\omega + 2\pi k)n + \theta) = x_k[n]$$

für *alle* $k \in \mathbb{Z}$ gilt. Das heißt, dass sich $x_k[\cdot]$ nicht von $x[\cdot]$ unterscheiden lässt. Man nennt dann jedes der $x_k[\cdot]$ einen *Alias* von $x[\cdot]$. Man kann deshalb auch umgekehrt sagen, dass für jedes ω mit $|\omega| > \pi$ ein zugehöriges ω_a mit $|\omega_a| < \pi$ existiert, sodass

$$\cos(\omega n + \theta) = \cos(\omega_a n + \theta)$$

gilt. Vergewissern sie sich von dieser Tatsache, indem sie verschiedene Aliase basierend auf Codeschnipsel 2 visualisieren (Übung).

Stellen wir uns für einen kurzen Moment vor, dass wir wissen, dass wir $x[n]$ durch Abtastung einer Funktion wie in (2.1.1) erhalten haben. Es gilt in diesem Fall $x[n] = A \cos(2\pi F n \Delta T + \theta)$ für ein uns unbekanntes $F \in \mathbb{R}$. Selbst wenn wir wissen, dass nur *eine* Frequenz in diesem Signal vor Abtastung vorhanden war, können wir *nicht* entscheiden, welche das war, da jeder Alias von $x[n]$ für die Abtastwerte von der kontinuierlichen Version in Frage kommen kann. *Es ist also nicht möglich, beliebige Signale ohne zusätzliches Vorwissen „einfach“ abzutasten.*

2.2 Zeit-Kontinuierliche Komplexe Sinussignale

Wir wollen eine bestimmte Untermenge an Funktionen betrachten: komplexe Schwingungen, welche mit einer Frequenz F_k schwingen, wobei diese Frequenz ein ganzzahliges Vielfaches einer Grundfrequenz F_0 ist. Das heißt, wir betrachten also $F_k = k \cdot F_0$ für $k \in \mathbb{Z}$ und erhalten

$$x_k(t) = \exp(j2\pi F_k t) = \exp(j2\pi k F_0 t) = \cos(j2\pi k F_0 t) + j \sin(j2\pi k F_0 t).$$

Jedes der x_k hat (wegen der Periodizität von $\cos$ und $\sin$) Periode $1/F_k = T_k = T_0/k$. Das heißt für wachsendes $|k|$ werden die Perioden immer um ein Vielfaches kürzer. Umgekehrt besitzen immernoch alle x_k gemeinsame

Periode, T_0 , da für jedes k gilt, dass $T_k \cdot k = T_0$. Wir haben auch (noch) kein Problem mit Aliasing zwischen den x_k , da bei kontinuierlichen Signalen gilt, dass $x_{k_1} \neq x_{k_2}$, falls $k_1 \neq k_2$.

Wie wir in Abschnitt 1.2.2 gesehen haben, können wir beliebige Linearkombination aus Signalen bilden und erhalten wieder ein Signal. Wir können also für eine Folge von $c_k \in \mathbb{C}$ die Linearkombination der x_k bilden und erhalten

$$x_a = \sum_{k \in \mathbb{Z}} c_k x_k : \mathbb{R} \rightarrow \mathbb{C} \quad \text{mit} \quad t \mapsto x_a(t) = \sum_{k \in \mathbb{Z}} c_k x_k(t) = \sum_{k \in \mathbb{Z}} c_k \exp(j2\pi k F_0 t). \quad (2.2.1)$$

Die erste Schreibweise ist absichtlich ohne das Argument t , um zu verdeutlichen, dass Signale *wirklich* als eigenständige Vektoren werden können und dass $x_a(t) \in \mathbb{C}$ „nur“ die Auswertung von x_a an der Stelle t ist, welche *strikt* von dem Vektor x_a zu unterscheiden ist.

Natürlich ist (2.2.1) als Fourier-Reihe von x_a bekannt, und die c_k sind die Fourierkoeffizienten von x_a . Wie in Abschnitt 1.2.2 können wir also die c_k durch die Fourier-Reihe in (2.2.1) mit x_a identifizieren, und uns die c_k somit eine alternative Darstellung von x_a liefern. Es ist hier zu beachten, dass ein „kontinuierliches“ Objekt, also x_a , durch eine Menge von diskreten Werten *vollständig* beschrieben wird!

2.3 Zeit-Diskrete Komplexe Sinussignale

Analog zu Abschnitt 2.2, wollen wir uns Eigenschaften zeit-diskreter komplexer Schwingungen herleiten, die von einer bestimmten Grundfrequenz f_0 definiert werden. Da wir, im Unterschied zum kontinuierlichen Fall, nicht für alle $f \in \mathbb{R}$ eine periodische Funktion erhalten würden, wählen wir $f_0 = 1/N$ für ein $N \in \mathbb{N}$. Die Intension ist, dass wir im Falle von Frequenz f_0 $1/N$ Perioden pro Abtastwert erhalten werden. Umgekehrt wird die Schwingung mit Frequenz f_0 genau Periodenlänge von N Samples/Abtastwerten haben. Dann definieren wir für $k \in \mathbb{Z}$ die Signale $x_k[\cdot] : \mathbb{N} \rightarrow \mathbb{C}$ als

$$x_k[n] = \exp(j2\pi k f_0 n). \quad (2.3.1)$$

Man sieht nun leicht, dass $f_k = k/N$ immer eine rationale Zahl ist und $x_k[\cdot]$ Periodenlänge N/k hat, falls $k/N \in \mathbb{Z}$. Dies ist in Codeschnipsel 3 dargestellt.

Außerdem findet man wieder, dass $x_k[\cdot]$ ein Alias von $x_{k+N}[\cdot]$ sein muss, denn mit $f_0 = 1/N$ ergibt sich

$$x_{k+N}[n] = \exp(j2\pi(k+N)f_0 n) = \exp\left(j2\pi \frac{k+N}{N} n\right) = \exp\left(j2\pi \frac{k}{N} n\right) \exp\left(j2\pi \frac{N}{N} n\right) = x_k[n].$$

Das heißt, dass nur N verschiedene $x_k[\cdot]$ existieren, wenn man diese mit Grundfrequenz $f_0 = 1/N$ definiert. Normalerweise nimmt man jene $x_k[\cdot]$ für $k = 0, 1, \dots, N-1$.

Nun können wir auch wieder, wie in (2.2.1) eine Linearkombination der $x_k[\cdot]$ bilden. Man beachte, dass in diesem Fall die Summation natürlich *endlich* sein wird, da wir nur N verschiedene $x_k[\cdot]$ zur Verfügung haben. Wir bilden also wieder eine Linearkombination von den Vektoren $x_k[\cdot]$ durch

$$x[\cdot] = \sum_{k=0}^{N-1} c_k x_k[\cdot]$$

und erhalten so ein Signal mit Werten

$$x[n] = \sum_{k=0}^{N-1} c_k x_k[n] = \sum_{k=0}^{N-1} c_k \exp\left(j2\pi \frac{k}{N} n\right), \quad (2.3.2)$$

```

A = 1
f = 1 / 16
# f = 1 / (5 * np.exp(1))
theta = np.pi / 2
# N = 32
N = 720

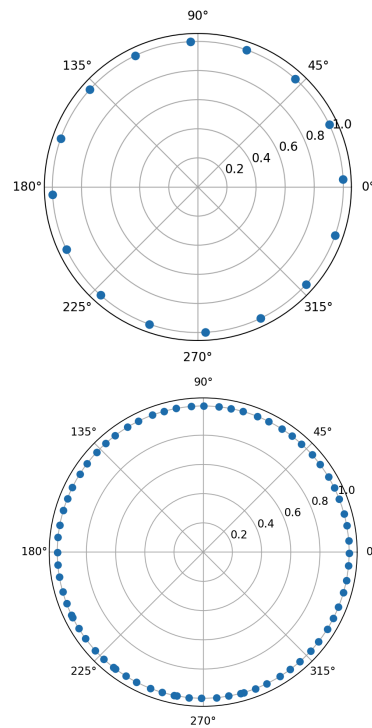
def harm(
    n: np.ndarray[int],
) -> np.ndarray[complex]:
    return A * np.exp(
        1j * 2 * np.pi * (f * n + theta)
    )

```

```

n = np.arange(N)
x_n = harm(n)
fig, ax = plt.subplots(
    subplot_kw={"projection": "polar"}
)
ax.plot(
    np.angle(x_n),
    np.abs(x_n),
    marker="o",
    linewidth=0,
)
plt.show()

```



Codeschnipsel 3: Berechnung und Darstellung von (2.3.1) für $f_0 = 1/16$ und $f_0 = 1/(5e)$, siehe [dsv/code/disc_harms_comp.py](#)

was die Fourier-Reihe eines diskreten und periodischen Signals darstellt, wobei in diesem Fall der Vektor $\mathbf{c} = [c_k]_{k=0}^{N-1} \in \mathbb{C}^N$, der die Koeffizienten in der Fourier-Reihe enthält, eine alternative Repräsentation des Signals ist.

Das Signal $x[\cdot]$ selbst ist periodisch mit Periodenlänge N , d.h. das Signal ist durch die Werte $\mathbf{x} = [x[n]]_{n=0}^{N-1} \in \mathbb{C}^N$ *vollständig* definiert. Das heißt, wir können das Signal $x[\cdot]$ mit dem *endlich-dimensionalen* Vektor $\mathbf{x} \in \mathbb{C}^N$ mit $x_n = x[n]$ identifizieren. Da genauso jedes der $x_k[\cdot]$ auch Periodenlänge N hat, erhalten wir auf die gleiche Weise Vektoren $\mathbf{x}_k \in \mathbb{C}^N$ mit $(\mathbf{x}_k)_n = x_k[n]$. Mit diesen endlich-dimensionalen Vektoren sind wir nun in der Lage die Fourier-Reihe in (2.3.2) mit „normalen“ Vektoren durch

$$\mathbf{x} = c_1 \mathbf{x}_1 + c_2 \mathbf{x}_2 + \dots + c_{N-1} \mathbf{x}_{N-1} \quad (2.3.3)$$

auszudrücken. Wir sehen also wieder *direkt*, dass Signale wirklich wie Vektoren behandelt werden können.

Gleichung (2.3.2) kann auch als Abbildung $M : \mathbb{C}^N \rightarrow \mathbb{C}^N$ interpretiert werden, da wir in die rechte Seite von (2.3.2) einfach ein $\mathbf{c} \in \mathbb{C}^N$ stecken können und wir erhalten das entsprechende $\mathbf{x} = M(\mathbf{c})$. Noch weiter ist die Abbildung M sogar linear, da

$$M(\mathbf{c}^1 + \mathbf{c}^2) = \sum_{k=0}^{N-1} (c_k^1 + c_k^2) \exp\left(j2\pi \frac{k}{N} n\right) = \sum_{k=0}^{N-1} c_k^1 \mathbf{x}_k + \sum_{k=0}^{N-1} c_k^2 \mathbf{x}_k = M(\mathbf{c}^1) + M(\mathbf{c}^2).$$

Das heißt, dass es zur linearen Abbildung M auch eine *Matrix* $\mathbf{M} \in \mathbb{C}^{N \times N}$ geben muss, welche uns einfach $\mathbf{c}$ in $\mathbf{x}$ umtransformiert, indem wir $\mathbf{x} = \mathbf{M} \cdot \mathbf{c}$ als Matrix-Vektor-Produkt berechnen. Dies liegt daran, dass wir im Grunde die Einträge in $\mathbf{x}$ beziehungsweise $\mathbf{c}$ als *Koordinaten* in der kanonischen Standardbasis $\{\mathbf{e}_1, \dots, \mathbf{e}_N\}$ betrachten können.

Wenn wir (2.3.3) genau studieren sehen wir, dass wir die Matrix $\mathbf{M}$ bilden können, indem wir deren k -te Spalte $\mathbf{M}_{\cdot,k}$ gleich $\mathbf{x}_k$ setzen. Wenn wir noch sicher sein könnten, dass $\mathbf{M}$ invertierbar ist, also eine Matrix $\mathbf{M}^{-1}$ mit $\mathbf{M} \cdot \mathbf{M}^{-1} = \mathbf{I}_N$ ², könnten wir sogar mit $\mathbf{x}$ via $\mathbf{c} = \mathbf{M}^{-1} \cdot \mathbf{x}$ die Fourierkoeffizienten $\mathbf{c}$ direkt aus einem gegebenen N -periodischen Signal $x[\cdot]$ bestimmen.

Wir haben hier eine sogenannte Isomorphie zwischen abgetasteten Summen von komplexen Schwingungen mit Grundfrequenz $1/N$ und dem Vektorraum $\mathbb{C}^N$ geschaffen. Dies hat zur Folge, dass eine Darstellung eines solchen diskreten Signals „in beiden Welten“ leben kann. In der Welt der Abtastwerte $x[n]/\mathbf{x}$, als auch in der Welt der Koeffizienten $\mathbf{c}$.

2.4 Abtastung von Sinussignalen

Es gibt viele Möglichkeiten ein analoges Signal zu digitalisieren. Im Allgemeinen ist das Ziel eine kontinuierliche Funktion/ein analoges Signal x_a so in eine Menge von diskreten Werten $x[\cdot]$ „umwandeln“, dass „keine Information verloren geht“. Wir beschränken uns auf Abtastung (denglisch: Sampling), welche ein analoges Signal auf eine regelmäßige Art und Weise *direkt* auswertet. Diese Art wird manchmal auch „Nyquist-Sampling“ genannt, weil die theoretische Grundlage für „erfolgreiches“ Sampling durch das Nyquist-Theorem gelegt ist. Wir stellen uns Abtastung so vor, dass wir das Signal $x_a : \mathbb{R} \rightarrow \mathbb{R}$ direkt an gewissen Stellen beobachten können. Wir können uns eine Art Sampling-Operator $\mathcal{S}$ vorstellen, der ein analoges Signal x_a in eine abgetastete Version $x_a \mapsto \mathcal{S}(x_a)[\cdot] = x[\cdot]$ transformiert. Durch die regelmäßige/uniforme Abtastung in Zeitabständen $T > 0$ von x_a erhalten wir also

$$x[n] = \mathcal{S}(x_a)[n] = x_a(nT) \quad \text{mit } n \in \mathbb{Z}.$$

²Hier steht $\mathbf{I}_N \in \mathbb{C}^{N \times N}$ für die Einheitsmatrix im $\mathbb{C}^N$

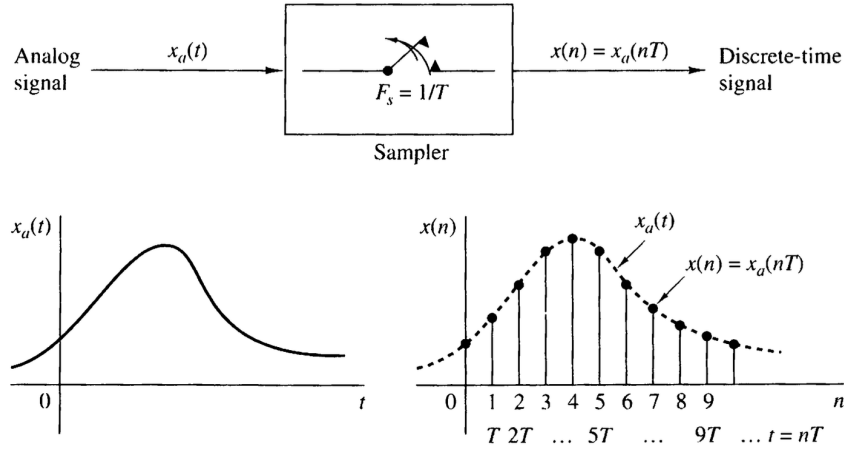


Abbildung 2: Uniforme Abtastung einer Signals. Quelle: [1]

Wir nennen $F_s = 1/T$ die Sampling-Frequenz, oder die Abtastrate. Der Vorgang ist schematisch in Abbildung 2 dargestellt.

Da wir $x[\cdot]$ so modellieren, dass es „einfach“ eine Folge von reellen/komplexen Zahlen ist, müssen wir bei der Interpretation von $x[\cdot]$ auch immer gleichzeitig im Hinterkopf behalten, dass der Wert $x[n]$ dem Wert $x_a(nT) = x_a(n/F_s)$ entspricht. Das bedeutet, dass t (als Argument von x_a) und n (als Argument von $x[\cdot]$) durch

$$t = nT = n/F_s$$

miteinander in Verbindung stehen. Diese Verbindung, dass n einem Zeitpunkt nT entspricht und dass wir F_s „viele“ Samples pro Zeiteinheit haben, muss also in unserer Vorstellung immer mitschwingen. Man sieht auch nun eindrucksvoll, dass dadurch $n = t \cdot F_s$ einheitenlos geworden ist, beziehungsweise sein muss.

Weiterhin folgt aus $t = n/F_s$, dass es auch einen Zusammenhang zwischen der Frequenz F eines kontinuierlichen Signals und der Frequenz f im Zeit-diskreten geben muss. Um diesen herzuleiten, betrachten wir eine einfache analoge Schwingung, wie in (2.1.1), also

$$x_a(t) = A \cos(2\pi Ft + \theta)$$

und wir stellen uns vor, dass wir dieses Signal mit Samplerate $F_s = 1/T$ abtasten. Dann erhalten wir zunächst

$$x[n] = x_a(nT) = A \cos(2\pi F nT + \theta) = A \cos\left(2\pi \frac{F}{F_s} n + \theta\right).$$

Da wir nun $x[n]$ in die Form von (2.1.2) gebracht haben, sehen wir, dass dort dann $f = F/F_s$ gelten muss.

Wie ist dies zu interpretieren? Wir sind mit einem speziellen analogen Signal x_a gestartet, welches als Sinussignal die Frequenz F enthält. Durch das Sampling mit Rate F_s entsteht eine *diskrete* Schwingung mit Frequenz $f = F/F_s$ in der Form von (2.1.2). Doch wir haben bereits gesehen, dass sich diskrete Schwingungen mit Frequenz f wegen *Aliasing* nicht von anderen diskreten Schwingungen mit Frequenz $f + k$ unterscheiden lassen. Das heißt im Umkehrschluss, dass sich *nach* der Abtastung die ursprüngliche Frequenz F nicht mehr eindeutig bestimmen lässt, da alle

$$F_k = F + k \cdot F_s$$

```

theta = -np.pi / 2 # rad
F = 1.5 # Hz
F_s = 1.8 # Hz
T_s = 1.0 / F_s # s

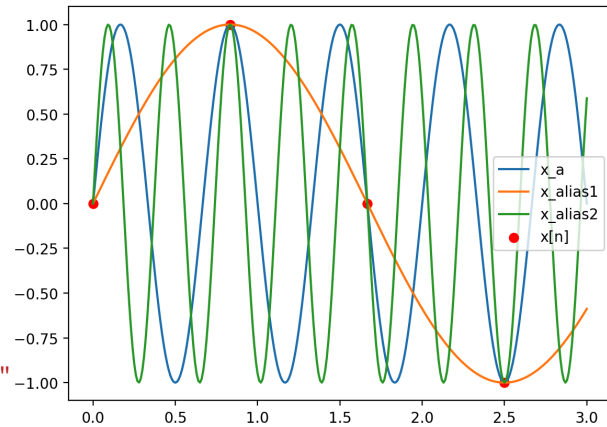
F_alias1 = F - 1 * F_s # Hz
F_alias2 = F + 1 * F_s # Hz

def x_a(t, F, theta):
    return np.cos(2 * np.pi * F * t + theta)

T = np.linspace(0, 3, 301, endpoint=True) # s
n = np.arange(4) * T_s # none

plt.plot(T, x_a(T, F, theta), label="x_a")
plt.plot(
    T, x_a(T, F_alias1, theta), label="x_alias1"
)
plt.plot(
    T, x_a(T, F_alias2, theta), label="x_alias2"
)
plt.scatter(
    n, x_a(n, F, theta), color="r", label="x[n]"
)
plt.legend()
plt.show()

```



Codeschnipsel 4: Visualisierung von $F_k = F + k \cdot F_s$, siehe [dsv/code/aliasing.py](#)

bei Abtastung von $A \cos(2\pi F_k t + \theta)$ dieselben Abtastwerte $x[n]$ ergeben würden.

Wenn wir nun also behaupten wollen, dass wir im digitalen irgendetwas Sinnvolles zu tun gedenken, dann können wir dies gerade nicht so ohne weiteres. Denn bis jetzt haben wir nicht einmal für einfachste Schwinungen eine Möglichkeit, wie man aus Abtastwerten das ursprüngliche analoge Signal rekonstruieren kann. Diese missliche Lage wird in Codeschnipsel 4 dargestellt, wo ein mögliches x_a , dessen Abtastwerte $x[n]$ und zwei mögliche Aliase visualisiert sind. Man sieht, wie die Aliase so geschaffen sind, dass auch sie Ursprung für die Abtastwerte $x[n]$ sein könnten.

Die Frage ist nun, wie wir das Sampling gestalten müssen, dass wir aus $x[n]$ eindeutig das Signal x_a rekonstruieren können. In diesem Fall ist mit „rekonstruieren“ gemeint, dass wir aus den diskreten Werten $x[n]$ die Werte $x_a(t)$ für beliebige $t \in \mathbb{R}$ korrekt bestimmen können. Wie wir oben gesehen haben, ist im Digitalen nur sinnvoll von Frequenzen $f \in [-1/2, +1/2]$ zu sprechen, da wir wegen Aliasing für alle anderen $f' \notin [-1/2, +1/2]$ ein $f \in [-1/2, +1/2]$ finden, das dieselben Werte in (2.1.2) produziert. Wegen des Zusammenhangs $f = F/F_s$ macht es also Sinn sich auch im Analogen auf den entsprechenden Bereich zu beschränken, der demnach von der Samplingrate F bestimmt wird. Das heißt, wenn wir nur analoge Signale betrachten, bei welchen

$$-\frac{F_s}{2} \leq F \leq +\frac{F_s}{2}$$

gilt, dann sind wir in der Lage aus $x[\cdot]$ das originale x_a zu rekonstruieren, weil wir *annehmen*, dass unser

abgetastetes Signal nicht im Bereich der Aliase liegt. Damit folgt die folgende wichtige Beobachtung:

Eine Möglichkeit der perfekten Rekonstruktion von analogen Signalen, die einzelne Frequenzen beinhalten, aus deren Abtastwerten ist die Einschränkung auf einen bestimmten Frequenzbereich.

Umgekehrt können wir auch bei Vorwissen über die maximale Frequenz $F_{\max}$, also $F \in [-F_{\max}, +F_{\max}]$ in einem Signal x_a die Samplingrate ermitteln, sodass im Digitalen die Aliase $f + k$ für $k \neq 0$ nicht im Bereich $[-F_{\max}, +F_{\max}]$ liegen. Damit

$$-\frac{1}{2} \leq f \leq +\frac{1}{2}$$

gilt, muss also auch

$$-\frac{1}{2} \leq \frac{F}{F_s} \leq +\frac{1}{2} \quad \text{für alle } F \in [-F_{\max}, +F_{\max}]$$

gelten. Deshalb muss schlussendlich $F_s > 2F_{\max}$ gelten, damit wir aus den Abtastwerten $x[\cdot]$ die ursprüngliche Schwingung x_a wieder exakt rekonstruieren können.

Es ist zu beachten, dass wir diese Überlegungen erst einmal „nur“ für Signale der Form (2.1.1) und (2.1.2) durchgeführt haben und wir nun einen Weg finden müssen, wie wir . Im nächsten Abschnitt werden wir das Abtasttheorem für beliebige aperiodische Signale herleiten.

2.5 Das Samplingtheorem

Wir beschäftigen uns nun also allgemein mit dem Problem der Abtastung eine beliebigen analogen aperiodischen Signals x_a zur Folge

$$x[n] = x_a(nT).$$

Für die bequeme Analyse von kontinuierlichen Signalen und ihrem Anteil an harmonischen Komponenten, benutzen wir die **Fourier Transform (FT)** $\mathcal{F}$, welche ein Signal $x : \mathbb{R} \rightarrow \mathbb{C}$ zu $x \mapsto \mathcal{F}x = X : \mathbb{R} \rightarrow \mathbb{C}$ transformiert, wobei die Fourier-Transformierte X von x durch

$$X(F) = \int_{-\infty}^{+\infty} x(t) \exp(-j2\pi Ft) dt \quad (2.5.1)$$

definiert ist. Wir werden versuchen, die Kombination von Klein- und Großbuchstaben $x \leftrightarrow X$ als **FT**-Paar beizubehalten. Die zugehörige inverse **FT**, also die Abbildung von X auf x ist gegeben durch

$$x(t) = \int_{-\infty}^{+\infty} X(F) \exp(j2\pi Ft) dF. \quad (2.5.2)$$

Wir wollen nun ausarbeiten, wie man diese Art Integral-Transformation geometrisch, also wenn man Signale als Vektoren behandelt, interpretieren kann, um eine alternative Sicht auf solche Art Transformation zu erhalten.

Betrachten wir dazu das Signal x wieder als Element aus einem Vektorraum, in dem zusätzlich zur Vektorraumstruktur ein *Skalarprodukt* definiert ist. Um zu erläutern, was das bringt, starten wir mit etwas Bekanntem. Im endlich-dimensionalen Fall, also wenn $x, y \in \mathbb{C}^N$, dann ist ein mögliches Skalarprodukt $\langle \cdot, \cdot \rangle : \mathbb{C}^N \times \mathbb{C}^N \rightarrow \mathbb{R}$ durch

$$\langle x, y \rangle = \sum_{i=1}^N x_i \cdot y_i^* = y^H \cdot x$$

gegeben, wobei der letzte Ausdruck das Matrix-Vektor-Produkt einer $\mathbb{C}^{N \times 1}$ Matrix und einem Vektor in $\mathbb{C}^N$ darstellt. Man kann nun $\langle x, y \rangle$ betrachten, um zu ermitteln, wie „ähnlich“ die Vektoren x und y sind. Beispielsweise nennen wir die beiden Vektoren orthogonal/senkrecht, falls $\langle x, y \rangle = 0$, diese sich also überhaupt nicht ähnlich sind. In diesem Fall interpretieren wir dies intuitiv so, dass x und y keine „gemeinsamen“ Anteile haben. Beispielsweise, wenn wir einen dritten Vektor $z = \alpha x + \beta y$ betrachten und es gilt, dass $\langle x, y \rangle = 0$, dann schlägt sich dies auch auf das Skalarprodukt von $\langle x, z \rangle$ nieder. Denn wir können berechnen, dass

$$\langle x, z \rangle = \langle x, \alpha x + \beta y \rangle = \alpha \langle x, x \rangle \quad \text{und} \quad \langle y, z \rangle = \langle y, \alpha x + \beta y \rangle = \beta \langle y, y \rangle$$

gelten muss. Beim Bilden von $\langle x, z \rangle$ können wir uns also vorstellen, dass wir ermitteln, wie viele „Anteile“ von x in z zu finden sind. Da nun x und y keine gemeinsamen Anteile haben, können wir also α und β unabhängig voneinander aus z extrahieren.

Man sieht, dass dies in gewisser Weise erlaubt mit hochdimensionalen Objekten eine Art Geometrie zu betreiben. Man kann noch weiter gehen und mit einem Skalarprodukt Korrelationen, Winkel und Längen definieren.

Wir wollen nun dieses Konzept einfach genauso auf kontinuierliche Signale anwenden und dann die **FT** in diesem Licht interpretieren. Man kann zeigen, dass für zwei Funktionen $x, y : \mathbb{R} \rightarrow \mathbb{C}$ die Abbildung $(x, y) \mapsto \langle x, y \rangle$ mit

$$\langle x, y \rangle = \int_{-\infty}^{+\infty} x(t) \cdot y^*(t) dt \quad (2.5.3)$$

ein Skalarprodukt definiert. Auch hier halten wir an der Interpretation fest, dass wir damit bestimmen, wie ähnlich sich (*die Funktionen!*) x und y sind. Definieren wir nun die Signale $k_F : \mathbb{R} \rightarrow \mathbb{C}$ durch $k_F(t) = \exp(j2\pi Ft)$, dann können wir (2.5.1) umschreiben zu

$$\langle x, k_F \rangle = \int_{-\infty}^{+\infty} x(t) \cdot \exp(j2\pi Ft)^* dt = \int_{-\infty}^{+\infty} x(t) \cdot \exp(-j2\pi Ft) dt = X(F).$$

Dies suggeriert uns also, dass wir die **FT** X an der Stelle F als Skalarprodukt von dem zu transformierenden Signal x und dem geeignet gewählten k_F Transformationskern interpretieren können. Die Zuordnung von $F \mapsto \langle x, k_F \rangle$ definiert dann im Grunde die Funktion X für alle F . Da die k_F komplexe Schwingungen mit Frequenz F darstellen, kann man sich die **FT** auch als Analysewerkzeug vorstellen, das „extrahiert“, zu welchem Anteil x die Frequenz F beinhaltet. Deshalb ist es auch gängig, dass man X als das *Spektrum* von x bezeichnet.

Beispiel 2.1. Wir betrachten das Signal $x : \mathbb{R} \rightarrow \mathbb{C}$ mit

$$x(t) = 4 \cdot \exp(j2\pi 3t) - 2 \cdot \exp(j2\pi 7t)$$

Dann berechnen wir die **FT** X an der Stelle F , indem wir $\langle x, k_F \rangle$ schrittweise und durch Nachschlagen in Tabellen berechnen:

$$\langle \exp(j2\pi 3t), k_F \rangle = \int_{-\infty}^{+\infty} \exp(j2\pi 3t) \cdot \exp(-j2\pi Ft) dt = \int_{-\infty}^{+\infty} \exp(j2\pi(3-F)t) dt = \delta(F-3) + \delta(-(F-3)),$$

$$\langle \exp(j2\pi 7t), k_F \rangle = \int_{-\infty}^{+\infty} \exp(j2\pi 7t) \cdot \exp(-j2\pi Ft) dt = \int_{-\infty}^{+\infty} \exp(j2\pi(7-F)t) dt = \delta(F-7) + \delta(-(F-7)),$$

Dann setzen wir einfach zusammen, indem wir die Linearität des Skalarproduktes ausnutzen:

$$X(f) = 4 \cdot \langle \exp(j2\pi 3\cdot), k_F \rangle - 2 \langle \exp(j2\pi 7\cdot), k_F \rangle = 4(\delta(F-3) + \delta(-(F-3))) - 2(\delta(F-7) + \delta(-(F-7))).$$

Auch für diskrete Signale lassen sich Skalarprodukte und die entsprechende **FT** definieren. Wir erhalten für eine Folge/ein diskretes Signal $x[\cdot]$ die **Discrete Time Fourier Transform (DTFT)** als Skalarprodukt von $x[\cdot]$ und der entsprechenden diskreten komplexen Schwingung $k_f[n] = \exp(j2\pi fn)$, also

$$X(f) = \sum_{n \in \mathbb{Z}} x[n] k_f[n]^* = \sum_{n \in \mathbb{Z}} x[n] \exp(-j2\pi fn). \quad (2.5.4)$$

Die inverse **DTFT** ergibt sich durch

$$x[n] = \int_{-1/2}^{+1/2} X(f) \exp(j2\pi fn) df.$$

Wenn wir uns an die Aliasing-Eigenschaft der diskreten Schwingungen k_f erinnern, also dass k_f nicht von k_{f+k} zu unterscheiden ist, dann muss dasselbe für die Skalarprodukte $\langle x[\cdot], k_f \rangle$ gelten. Dies bedeutet dann, dass X als **DTFT** von $x[\cdot]$ eine *periodische* Funktion sein muss, da

$$X(f) = \langle x[\cdot], k_f[\cdot] \rangle = \langle x[\cdot], k_{f+k}[\cdot] \rangle = X(f+k)$$

gilt. Das heißt, dass sich aus Aliasing von diskreten Schwingungen Periodizität der **DTFT** mit Periodenlänge 1 ergibt.

Wir haben nun alle Werkzeuge zusammen, um genauer zu Analysieren, was bei Abtastung von x_a zu $x[\cdot]$ mit Abtastrate F_s geschieht. Wir erinnern uns, dass für die Abtastpunkte $t = nT = n/F_s$ gilt. Wir erhalten dann mit $x[n] = x_a(nT)$ und der inversen **FT** angewandt auf X_a , dass

$$x[n] = x_a(nT) = \int_{-\infty}^{+\infty} X_a(F) \exp(j2\pi F/F_s n) dF$$

gelten muss. Setzen wir nun auch für $x[\cdot]$ die inverse **DTFT** ein, erhalten wir mit $f = F/F_s$ (siehe Abschnitt 2.4) als Variablentransformation, dass

$$\frac{1}{F_s} \int_{-F_s/2}^{+F_s/2} X(f) \exp(j2\pi F/F_s n) dF = \int_{-\infty}^{+\infty} X_a(F) \exp(j2\pi F/F_s n) dF$$

gilt. Wichtig ist an diesem Ausdruck, dass wir nun einen Zusammenhang zwischen der **DTFT** X der Abtastwerte $x[\cdot]$ und der **FT** X_a des analogen Signals x_a gefunden haben. Die Integration auf beiden Seiten ist auch bezüglich desselben Integrationskernes $\exp(j2\pi \cdot /F_s n)$. Das heißt, dass es möglich sein sollte, einen Ausdruck herzuleiten, der die **DTFT** X in Abhängigkeit von der **FT** X_a ausdrückt. Mit ein wenig algebraischem Grindwork findet man

$$X(f) = F_s \sum_{k \in \mathbb{Z}} X_a((f-k)F_s). \quad (2.5.5)$$

Wie ist dies nun zu interpretieren? Wir haben oben schon gesehen, dass das Spektrum von $x[\cdot]$ periodisch sein muss. In (2.5.5) berechnet sich also das periodische Spektrum X durch Wiederholung von um $1/F_s$ gestauchte Kopien von X_a mit Abstand $1/F_s$, was in f eine periodische Funktion mit Periodenlänge 1 ergibt. Im Grunde bestätigt (2.5.5) für beliebige Signale, was wir bereits für Signale mit einzelnen Frequenzen (siehe (2.1.1)) gefunden hatten.

Andererseits kann man es auch so sehen, dass der Wert $X(f)$ sich aus Summation der Werte $X_a(f \cdot F_s - kF_s)$ für alle $k \in \mathbb{Z}$ ergibt. Also an der Frequenz f erscheinen in $X(f)$ die Werte von X_a an den Stellen $fF_s - kF_s$. Das heißt, wie die Periodifizierung vonstatten geht hängt maßgeblich von der Samplingfrequenz F_s ab!

Man sieht nun auch wieder schön, dass unsere obige Argumentation zum Verhältnis von Samplingfrequenz F_s und maximaler Frequenz $F_{\max}$ im abgetasteten Signal immernoch gilt, da *alle* belegten Frequenzen von x_a sich im Intervall $[-F_s/2, +F_s/2]$ befinden müssen, wenn sich sonst die Kopien von X_a in (2.5.5) nicht überlappen sollen. Denn nur wenn diese sich nicht überlappen, kann man das Spektrum des analogen Signals X_a von den Werten von X ablesen.

Dies ist äquivalent zur Forderung, dass

$$|X_a(F)| = 0 \quad \text{für} \quad |F| > F_s/2$$

gelten muss. In diesem Fall spricht man davon, dass X_a *bandbegrenzt* ist. Das kleinste $F \in \mathbb{R}$, was obige Ungleichung erfüllt, nennen wir $F_{\max}$.

Wir sind nun beinahe am Ziel angelangt, dass wir das Samplingtheorem formulieren können. Wir benötigen lediglich noch eine Möglichkeit aus den Abtastwerten $x[\cdot]$ das ursprüngliche Signal x_a zu rekonstruieren. Wenn wir wissen, dass kein Aliasing vorliegt, also $F_{\max} < F_s/2$, dann können wir X_a aus X berechnen, indem wir

$$X_a(F) = \begin{cases} \frac{X(F/F_s)}{F_s}, & |F| \leq F_s/2, \\ 0 & \text{sonst} \end{cases}$$

setzen. Wir wissen außerdem, dass sich X durch die DTFT durch

$$X(f) = \sum_{n \in \mathbb{Z}} x[n] \exp(-j2\pi nF/F_s)$$

berechnen lässt. Weiter können wir nun auch x_a durch Integration über einen endlichen Bereich durch

$$x_a(t) = \int_{-F_s/2}^{+F_s/2} X_a(F) \exp(j2\pi Ft) dF.$$

erhalten. Wir setzen nun einfach alles ein und erhalten

$$x_a(t) = \frac{1}{F_s} \int_{-F_s/2}^{+F_s/2} \sum_{n \in \mathbb{Z}} x[n] \exp(-j2\pi nF/F_s) \exp(j2\pi Ft) dF.$$

Toll, aber wir müssen dies nun noch ein wenig umformen. Zuerst vertauschen wir Integration und Summation, weil wir es dürfen und nutzen $x[n] = x_a(n/F_s)$. Dies ergibt

$$x_a(t) = \frac{1}{F_s} \sum_{n \in \mathbb{Z}} x_a(n/F_s) \int_{-F_s/2}^{+F_s/2} \exp(-j2\pi nF/F_s) \exp(j2\pi Ft) dF.$$

Wir sind nicht wirklich schlauer, aber wir wissen aus unserer Erfahrung von Integration von komplexen Funktionen, dass

$$\frac{1}{F_s} \int_{-F_s/2}^{+F_s/2} \exp(-j2\pi nF/F_s) \exp(j2\pi Ft) dF = \frac{\sin(\pi F_s(t - n/F_s))}{\pi F_s(t - n/F_s)} = \text{sinc}(F_s(t - n/F_s)).$$

Wir nutzen nun wiederum die Interpretation von Integration eines Produktes von Funktionen als deren Skalarprodukt. Wir bilden also das Skalarprodukt von zwei Funktionen mit Periode F_s , d.h. es genügt das Integral im Intervall $[-F_s/2, +F_s/2]$ zu bilden. Sehen wir genauer hin, bilden wir also das Skalarprodukt (aber diesmal von Signalen im *Frequenzbereich*!) $\langle k_t, k_{n/F_s} \rangle$, wobei ähnlich wie oben

$$k_t(F) = \exp(j2\pi Ft)$$

definiert wurde. Das heißt nun in der Sprache von Skalarprodukten, dass wir die sinc-Funktion erhalten, als

$$\text{sinc}(F_s(t - n/F_s)) = \langle k_t, k_{n/F_s} \rangle.$$

Wenn man es ein wenig allgemeiner formuliert, kann man sagen, dass die sinc-Funktion nur ein spezieller *Interpolationskern* ist, der sich aus der Definition von k_t und damit eigentlich aus der Art der Abtastung ergibt. Wir werden später noch ein weiteres Beispiel für so einen Kern betrachten. Zusammenfassend für diesen Abschnitt 2 steht nun folgendes

Theorem 2.1 (Samplingtheorem). Gegeben sei ein analoges Signal x_a mit Frequenzen in $[-F_{\max}, +F_{\max}]$ und dessen Abtastwerte $x[\cdot]$ mit $x[n] = x_a(nT)$, wobei $T = 1/F_s$.

Falls $F_s > 2F_{\max}$, dann gilt

$$x_a(t) = \sum_{n \in \mathbb{Z}} x[n] \cdot g_{F_s}(t - nT), \quad (2.5.6)$$

wobei der Interpolationskern $g : \mathbb{R} \rightarrow \mathbb{R}$ gegeben ist durch

$$g_{F_s}(t) = \frac{\sin(\pi F_s t)}{\pi F_s t}.$$

Hinweis: Das Samplingtheorem, wie es in [1] formuliert ist, beinhaltet einen kleinen, aber entscheidenden Fehler. Die Definition des Interpolationskernes g_F ist dort mit

$$g(t) = \frac{\sin(2\pi Bt)}{2\pi Bt}.$$

gegeben. Doch in diesem Falle würde der Interpolationskern von der Bandbreite des Signals x_a , aber nicht von der Abtastrate F_s abhängen. Die angegebene Definition ist nur korrekt, falls $F_s = 2B$, wir also mit der *minimalen* Samplerate abgetastet haben. In diesem Fall spricht man auch von *kritischer Abtastung*.

Es ist weiterhin noch zu erwähnen, dass wir genau genommen immer noch nicht wissen, wie wir überprüfen können, welche maximale Frequenz $F_{\max}$ von einem analogen Signal x_a belegt wird. Dies liegt daran, dass wir im Vorhinein – also vor Abtastung – keinen Zugriff auf das abzutastende Signal haben und deswegen die Bedingung an die Bandbreite nicht überprüfen können. Das heißt, dass wir auch noch nicht überprüfen können, ob wir das Samplingtheorem eingehalten haben, oder einhalten werden, falls wir Abtasten sollten. Man muss sich also für ein gegebenes System, das einem zu digitalisierende Signale produziert, die also abgetastet werden sollen, auf anderem Weg überlegen, wie man die Bedingung $|X_a(F)| = 0$ für $|F| > F_s/2$ überprüfen, oder besser noch im Vorhinein sicherstellen, kann.

```

F_max = 1.0
F = np.random.choice(
    np.linspace(
        -F_max, +F_max, 11, endpoint=True
    ),
    5,
    replace=False,
) # Hz
theta = np.random.uniform(0, 2 * np.pi, 5) # rad
A = np.random.randn(5) # amplitude

```

```

def x_a(t: np.ndarray) -> np.ndarray:
    return np.sum(
        np.cos(
            2 * np.pi * np.outer(t, F) + theta
        )
        * A,
        axis=1,
    )

```

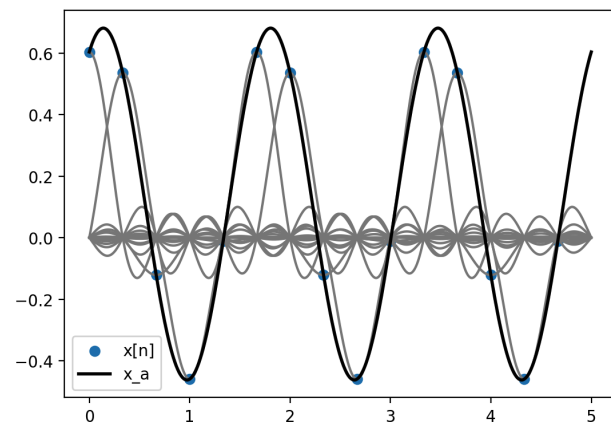
```
t = np.linspace(0, 5, 1024)
```

```
F_s = 3
```

```

def g(t: np.ndarray) -> np.ndarray:
    nenner = np.pi * F_s * t
    nenner[np.isclose(t, 0)] = 1
    result = np.sin(np.pi * F_s * t) / nenner
    result[np.isclose(t, 0)] = 1
    return result

```



```

n = (
    np.arange(int(t[-1] * F_s)).astype(float)
    / F_s
)
x_n = x_a(n)
interpol = np.zeros_like(t)
for nn in np.arange(len(n)):
    interpol += x_n[nn] * g(t - nn / F_s)
plt.plot(
    t,
    x_n[nn] * g(t - nn / F_s),
    color="grey",
)

```

Codeschnipsel 5: Berechnung und Darstellung von Theorem 2.1, siehe [dsv/code/sampling_theorem.py](#)

3 Diskrete Signale und Systeme

3.1 Diskrete Signale

Wir sind nun endlich im Digitalen angekommen. Wir wollen uns als erstes verschiedene Möglichkeiten der Klassifikation von diskreten Signalen $x[\cdot] : \mathbb{Z} \rightarrow \mathbb{C}$ ansehen. Dabei folgend wir weitestgehend [1, Kap. 2.1]. In Abschnitt 1.2.5 haben wir bereits den Einheitsstoß $\delta[\cdot]$ und die Heavy-Side-Funktion $u[\cdot]$ kennengelernt.

- Eine linear ansteigende Version $u_r[\cdot]$ von $u[\cdot]$ (r steht für ramp) ist gegeben durch

$$u_r[n] = n \cdot u[n] = \begin{cases} n, & \text{für } n \geq 0 \\ 0, & \text{sonst} \end{cases}.$$

In Python ist die Funktion auch sehr einfach zu implementieren:

```
def u_r(n: int) -> int:
    return n if n > 0 else 0
```

Will man die Funktion effizienter mittels Numpy [2] implementieren, dann liest sie sich

```
import numpy as np
def u_r_np(n: np.ndarray[int]) -> np.ndarray[int]:
    u_r = n.copy()
    u_r[n <= 0] = 0
    return u_r
```

- Für eine komplexe Zahl $a = r \exp(j\theta) \in \mathbb{C}$ erhält man das zugehörige exponentielle Signal als

$$x[n] = a^n = r^n \exp(j\theta n) = r^n \cos(\theta n) + jr^n \sin(\theta n).$$

Damit gilt $x[0] = 1$ unabhängig von a . Beispielsweise erhalten wir das Signal x_k aus (2.3.1) indem wir $r = 1$ und $\theta = 2\pi k f_0$ setzen. Eine Implementierung von $x[n]$ ist in Codeschnipsel 6 gegeben. Es ist sicher interessant für verschiedene Werte von a und n die Ausgabe zu betrachten.

Beispielsweise kann man sehen, dass für $a \in \mathbb{R}$ gelten muss, dass $\lim_{n \rightarrow \infty} x[n] = 0$, falls $r < 1$, aber $\lim_{n \rightarrow -\infty} |x[n]| = \infty$ andernfalls. Weiterhin gilt für $a \in \mathbb{C}$ und $|a| = 1$, dass dann auch $|x[n]| = 1$ für alle n .

3.1.1 Energie Diskreter Signale

Oft ist es interessant zu bemessen, wie viel Energie in einem Signal vorhanden ist. Für eine physikalisch korrekte Bemessung dieser Energie, müsste man das Signal zwar mit Einheiten versehen, aber diese ergeben nur einen entsprechenden Proportionalitätsfaktor. Hierzu betrachten wir

$$E(x[\cdot]) = \sum_{n \in \mathbb{Z}} |x[n]|^2 = \sum_{n \in \mathbb{Z}} x[n]^* \cdot x[n]. \quad (3.1.1)$$

Wenn gilt $E(x[\cdot]) < \infty$, dann sprechen wir erstaunlicherweise von einem Signal endlicher Energie.

Es ist nun interessant sich eine Menge $\mathcal{E}$ zu definieren, die alle Signale enthält, welche endliche Energie besitzen, also

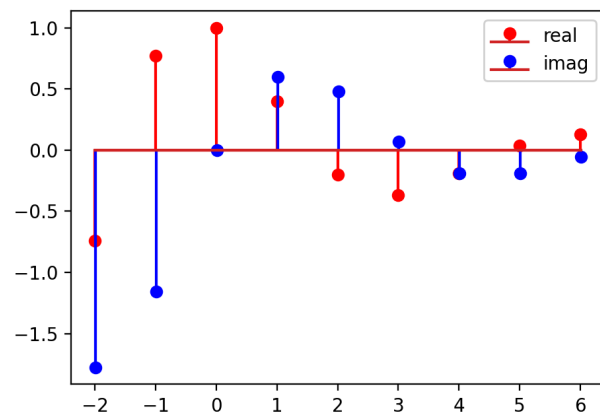
$$\mathcal{E} = \{x : \mathbb{Z} \rightarrow \mathbb{C} \text{ mit } E(x[\cdot]) < \infty\}.$$

```

def complex_exp(
    n: np.ndarray[int], a: complex
) -> np.ndarray[complex]:
    return np.abs(a) ** n * np.exp(
        1j * np.angle(a) * n
    )

n = np.linspace(-2, +10, 13, dtype=int)
a = 0.4 + 1j * 0.6
plt.stem(
    n,
    complex_exp(n, a).real,
    linefmt="r",
    label="real",
)
plt.stem(
    n + 0.1,
    complex_exp(n, a).imag,
    linefmt="b",
    label="imag",
)
plt.legend()
plt.show()

```



Codeschnipsel 6: Berechnung und Darstellung eines komplexen exponentiellen Signals, siehe [dsv/code/c/complex_exp.py](#)

Man kann sich nun überlegen, dass

$$E(\alpha x[\cdot] + \beta y[\cdot]) \leq E(\alpha x[\cdot]) + E(\beta y[\cdot]) = \alpha^2 E(x[\cdot]) + \beta^2 E(y[\cdot]) < \infty$$

gelten muss, falls $E(x[\cdot]), E(y[\cdot]) < \infty$. Das heißt, dass Linearkombinationen von Signalen mit endlicher Energie wieder ein Signal mit endlicher Energie ergeben, sodass die Signale endlicher Energie $\mathcal{S}$ einen *Unterraum* der Signale $\mathcal{S}$ bilden. Wir können noch einen Schritt weiter gehen und wie in (2.5.4) die Summe in (3.1.1) als Skalarprodukt auffassen.

Definieren wir für zwei Signale endlicher Energie die Abbildung $\langle \cdot, \cdot \rangle : \mathcal{E} \times \mathcal{E} \rightarrow \mathbb{C}$ als

$$(x[\cdot], y[\cdot]) \mapsto \langle x[\cdot], y[\cdot] \rangle = \sum_{n \in \mathbb{Z}} x[n]^* \cdot y[n], \quad (3.1.2)$$

dann kann man sich überlegen, dass dies die Bedingungen an ein *Skalarprodukt* erfüllt. Beispielsweise kann man nachrechnen, dass die unendliche Summe in (3.1.2) immer endlich ist, falls $x[\cdot], y[\cdot] \in \mathcal{E}$, da

$$|\langle x[\cdot], y[\cdot] \rangle| \leq E(x[\cdot]) \cdot E(y[\cdot]) < \infty$$

Nun kann man aber auch E durch

$$E(x[\cdot]) = \langle x[\cdot], x[\cdot] \rangle$$

ausdrücken und sich überlegen, dass E auf den signalen endlicher Energie eine *Norm* definiert. Wir können also zusätzlich zum „Winkel“ zwischen Signalen nun auch deren *Länge* bestimmen.

Beispiel 3.1. Betrachten wir $x[n] = a^n \cdot u[n]$ für $a = r \exp j\theta \in \mathbb{C}$. Dann berechnet sich $E(x[\cdot])$ durch

$$E(x[\cdot]) = \sum_{n \geq 0} |a^n|^2 = \sum_{n \geq 0} (r^2)^n.$$

Ist nun $r \geq 1$, dann $E(x[\cdot]) = \infty$, falls aber $r < 1$, dann ergibt sich aus der geometrischen Reihe, dass

$$E(x[\cdot]) = \frac{1}{1 - r^2}$$

gilt. Das heißt auch, dass das Heavy-Side-Signal $u[\cdot]$, also mit $a = 1$, keine endliche Energie besitzt.

3.1.2 Periodische Signale

Gilt für ein Signal $x[\cdot]$, dass $x[n + N] = x[n]$ für ein $N \in \mathbb{N}$ und alle $n \in \mathbb{Z}$, so nennt man $x[\cdot]$ periodisch mit Periodenlänge/Periode N , oder kurz N -periodisch, siehe beispielsweise Gleichung (2.3.1). Falls $x[\cdot]$ nun N -periodisch ist, dann ist $x[\cdot]$ auch kN -periodisch, falls $k \in \mathbb{N}$. Das heißt, dass es sinnvoller ist, das *kleinste* $N \in \mathbb{N}$ zu betrachten, sodass $x[\cdot]$ dann N -periodisch ist. Man nennt N dann Fundamentalperiode. Falls solch ein N nicht existiert, dann nennt man $x[\cdot]$ aperiodisch, oder nicht-periodisch. Falls $x[\cdot] \neq 0$, dann gilt für periodische Signale, dass $E(x[\cdot]) = \infty$. Beispielsweise haben wir bereits in Abschnitt 2.4 gesehen, dass

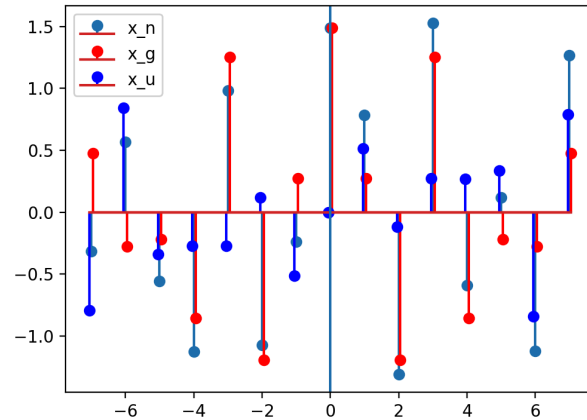
$$x[n] = \exp(j2\pi f n)$$

periodisch mit Periode N ist, falls $f = k/N$, also eine rationale Zahl ist.

```
# assume n=0 in center, size(x_n) = 2K+1
def even(x_n: np.ndarray) -> np.ndarray:
    return 0.5 * (x_n + x_n[::-1])

# assume n=0 in center, size(x_n) = 2K+1
def odd(x_n: np.ndarray) -> np.ndarray:
    return 0.5 * (x_n - x_n[::-1])

K = 7
n = np.linspace(-K, +K, 2 * K + 1)
x_n = np.random.randn(n.size)
```



Codeschnipsel 7: Zerlegung eines Signals in seinen geraden und ungeraden Anteil., siehe [dsv/code/even_odd.py](#)

3.1.3 Symmetrie von Signalen

Gilt für ein Signal $x[n] = x[-n]$, dann nennt man es *symmetrisch* bzw. *gerade*. Gilt andererseits $x[n] = -x[-n]$, so nennt man es *anti-symmetrisch* bzw. *ungerade*.

Ist ein beliebiges Signal $x[\cdot]$ gegeben, so kann man

$$x_g[n] = \frac{1}{2} (x[n] + x[-n]) \quad \text{und} \quad x_u[n] = \frac{1}{2} (x[n] - x[-n])$$

definieren. Dann ist $x_g[\cdot]$ gerade und falls $x[\cdot]$ bereits gerade ist, so gilt $x[\cdot] = x_g[\cdot]$. Genauso ist $x_u[\cdot]$ ungerade und falls $x[\cdot]$ bereits ungerade ist, so gilt $x[\cdot] = x_u[\cdot]$. Außerdem gilt

$$x[n] = x_g[n] + x_u[n].$$

Wir haben das Signal $x[\cdot]$ also in einen geraden und einen ungeraden Teil zerlegt. Dies ist manchmal sinnvoll, wenn man solch ein Signal linear transformiert und weiß, dass die lineare Transformation für gerade oder ungerade Signale gewisse Eigenschaften hat. Wie man an Codeschnipsel 7 gut sehen kann, muss gelten $x_u[0] = 0$, da $x_u[0] = x[0] - x[0] = 0 = -x_u[0]$ und $x_e[0] = x[0]$, da $2x_e[0] = x[0] + x[0]$.

3.2 Diskrete Systeme

Nachdem wir uns nun ein wenig mit diskreten Signalen vertraut gemacht haben, sind wir in der Lage uns mit diskreten Systemen zu befassen. Ganz allgemein kann man fast jeden Prozess, an dessen Anfang ein diskretes Signal steht und dessen Ergebnis wiederum ein diskretes Signal ist, als ein diskretes System auffassen. Sobald man dieses System nun einmal vorliegen hat, will man Techniken und Werkzeuge entwickeln, mit welchen man dieses System systematisch untersuchen kann – eine Systematik der Systeme.

Ein System $\mathcal{T}$ wird mathematisch als Abbildung eines (Eingabe-)Signals $x[\cdot]$ auf ein anderes (Ausgabe-)Signal $y[\cdot]$ aufgefasst. Wir schreiben dafür dann

$$x[\cdot] \mapsto \mathcal{T}(x[\cdot])[\cdot] = y[\cdot]. \quad (3.2.1)$$

Das System $\mathcal{T}$ bildet also die Paare $(x[\cdot], \mathcal{T}(x[\cdot])) = (x[\cdot], y[\cdot])$.

Betrachten wir folgendes

Beispiel 3.2. Gegeben sei das Eingangssignal

$$x[n] = \begin{cases} |n|, & \text{falls } -3 \leq n \leq +3, \\ 0 & \text{sonst.} \end{cases}$$

Wir sind nun an den Werten der Ausgangssignals $y[\cdot]$ interessiert, für

- a) das Einheitssystem $y[n] = x[n]$,
- b) das Einheitsdelay-System $y[n] = x[n - 1]$,
- c) das Einheitsadvance-System $y[n] = x[n + 1]$

interessiert.

Diese Systeme waren noch ein wenig simpel und man ist vielleicht noch nicht überzeugt, dass eine genauere Analyse von diskreten Systemen notwendig sein sollte. Dies liegt vor allem daran, dass die Systeme in Beispiel 3.2 nur „lokal“ gearbeitet haben, da Werte $y[n]$ nur von Werten $x[n - 1]$, $x[n]$ und $x[n + 1]$ abhängen.

Betrachten wir wiederum die Mandelbrot-Iteration³, aber diesmal als System

$$y[n + 1] = y[n]^2 + x[n]$$

für verschiedene Eingangssignale $x[n] = c \in \mathbb{C}$ mit $y[0] = 0$. Wir sind nun an solchen c interessiert für welche das System divergiert, also $|y[n]| > 2$ ab einem gewissen n . Wir wollen aber solche c finden, für welche wir in einem gewissen Bereich $n \in [n_{\text{low}}, n_{\text{high}}]$ divergieren. Das in Codeschnipsel 8 gezeigte Beispiel ist hierbei am anderen Ende des Komplexitätsspektrums, da man bei diesem System eher von einem „chaotischen“ System sprechen sollte. Kleine Veränderungen an $x[n] = c$ haben großen Einfluss auf das Divergenzverhalten der Folge $y[n]$ ⁴.

Um zu sehen, wie Systeme von ihrem Anfangszustand abhängen können, wollen hierfür ein etwas einfacheres Beispiel betrachten. Gegeben ist das System

$$y[n] = \sum_{k=-\infty}^n x[k].$$

Wir sehen hier, dass es für die Berechnung von $y[n]$ nicht ausreicht, den Zustand des Eingangs zum Zeitpunkt n , also $x[n]$ zu kennen. Schließlich müssen wir die gesamte Vergangenheit von $x[\cdot]$ bis zum Zeitpunkt n in die Berechnung einfließen lassen. Wir können das System aber umschreiben in

$$y[n] = \sum_{k=-\infty}^{n-1} x[k] + x[n] = y[n - 1] + x[n],$$

wobei wir nun auch sehen, warum dieses System *Akkumulator* genannt wird, da $y[\cdot]$ im Prinzip die Werte von $x[\cdot]$ „aufsammelt“.

³siehe dsv/code/mandelbrot.py

⁴<https://erleuchtet.org/2010/07/ridiculously-large-buddhabrot.html>

```

def sample(
    x_0: complex, low: int, hgh: int
) -> tuple[bool, int]:
    x_n: int = x_0
    counter: int = 0
    while (
        x_n.real**2 + x_n.imag**2
    ) < 4 and counter < hgh:
        x_n = x_n**2 + x_0
        counter += 1

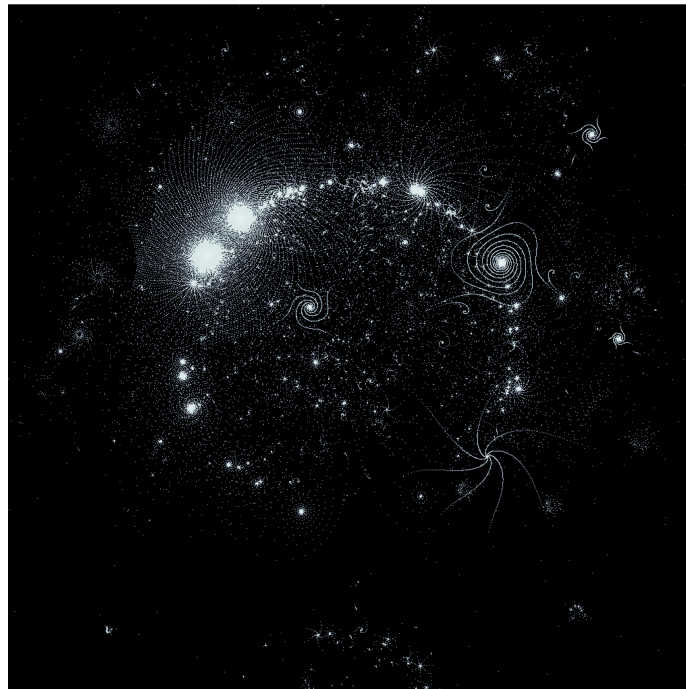
    return (
        (low <= counter) and (counter <= hgh),
        counter,
    )

def trial():
    while True:
        yield np.random.uniform(
            -2, +2
        ) + 1j * np.random.uniform(-2, +2)

success = []
for tt in trial():
    valid, counter = sample(tt, 100, 50000)
    if valid:
        success += [(tt, counter)]
    if len(success) > 400:
        break

image = np.zeros((1024, 1024), dtype=int)
for ss in success:
    x_n = ss[0]
    for ii in range(ss[1]):
        image[
            int(
                image.shape[0]
                * (x_n.real / 2 + 0.5)
            ),
            % image.shape[0],
            int(
                image.shape[1]
                * (x_n.imag / 2 + 0.5)
            ),
            % image.shape[1],
        ] += 1
    x_n = x_n**2 + ss[0]

```



Codeschnipsel 8: Spät divergierende Orbits der Mandelbrot-Iteration, siehe [dsv/code/buddhabrot.py](#)

```

n = np.arange(100)
x_1_n = np.cos(2 * np.pi * 0.1 * n)
x_2_n = np.cos(2 * np.pi * 0.5 * n) * np.exp(
    -n / 20
)

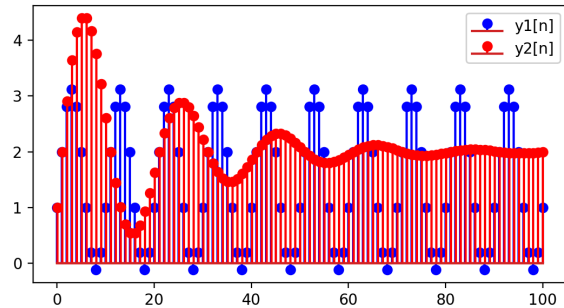
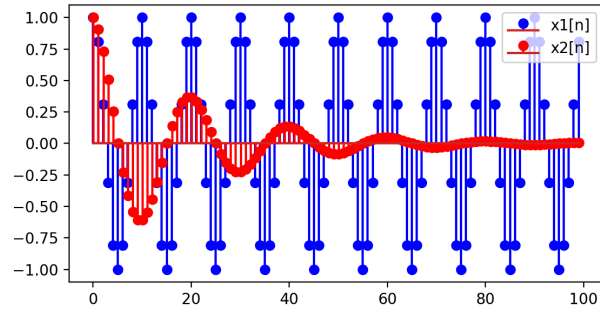
y_0 = -1

```

```

y_1_n = np.concatenate(
    [[y_0], y_0 + np.cumsum(x_1_n)]
)
y_2_n = np.concatenate(
    [[y_0], y_0 + np.cumsum(x_2_n)]
)

```



Codeschnipsel 9: Akkumulator für zwei verschiedene Eingangssignale., siehe [dsv/code/accumulator.py](#)

Stellen wir uns nun vor, dass wir dieses System modellieren/simulieren wollen für $n \leq n_0$, so benötigen wir entweder die Werte $x[n]$ für $n < n_0$, oder die sogenannte *Anfangsbedingung* $y[n_0] = y_0$. Dies erinnert an das Lösen einer Differentialgleichung

$$\dot{y}(t) = x(t),$$

wonach dann gilt, dass

$$y(t) = \int_{-\infty}^t x(s) \, ds \quad \text{oder} \quad y(t) = y_0 + \int_{t_0}^t x(s) \, ds,$$

damit $y(t_0) = y_0$. Ein Beispiel für die Wirkung von einem Akkumulator ist in Codeschnipsel 9 gegeben. Man sieht sehr gut, welchen Einfluss die Anfangsbedingung auf $x_2[\cdot]$ hat, da dies den Grenzwert $\lim_{n \rightarrow \infty} y[n]$ maßgeblich beeinflusst. Falls gilt, dass $y_0 = 0$, so spricht man vom Ruhezustand, beziehungsweise dem Nullzustand in dem sich das System zum Zeitpunkt $n = n_0$ befindet.

Statisch vs. Dynamisch Man kann Systeme nun auf verschiedene Arten klassifizieren. Beispielsweise nennen wir Systeme *statisch*, wenn der Wert $y[n]$ nur von $x[n]$ abhängt, aber nicht von vergangenen oder gar zukünftigen Werten (entweder von $y[\cdot]$ oder $x[\cdot]$). Die Systeme

$$y[n] = ax[n], \quad \text{oder} \quad y[n] = \sqrt{x[n]} + x^4[n]$$

sind statisch. Hängt nun $y[n]$ von seiner eigenen Vergangenheit oder der von $x[n]$ ab, so nennen wir diese Systeme dynamisch, oder Systeme mit Gedächtnis. Die Systeme

$$y[n] = x[n] + ax[n-1] \quad , \quad y[n] = \sum_{k=0}^N x[n-k]$$

sind dynamisch und haben jeweils Gedächtnis der Länge $g = 1$, beziehungsweise $g = N - 1$. Man sieht bereits an Codeschnipsel 9, dass dynamische Systeme im Allgemeinen interessanter sein werden.

Kausal vs. Akausal Hängen die Werte $y[n]$ nur von $x[n], x[n-1], \dots$ ab, also nicht auch von $x[n+1], x[n+2], \dots$, so nennen wir das System kausal. Intuitiv bedeutet dies die intuitiv bekannte Kausalität in dem Sinne, dass nur die Zeitliche Vergangenheit notwendig ist, um den aktuellen Zustand des Systems zu bestimmen. Ist die nicht gegeben, so nennt man das System *akausal* oder *nicht-kausal*.

Was die Realisierung von akausalen Systemen angeht, wird es nicht möglich sein, diese in Echtzeit umzusetzen, da man diese nur mit einer Verzögerung, oder gar nicht für sequentiell verfügbares $x[\cdot]$ implementieren kann. Sind die Werte $x[n]$ jedoch beispielsweise durch Messung oder Simulation entstanden und „offline“ verfügbar, so können solche Systeme durchaus angewandt werden und nützlich sein, wie wir in Abschnitt 7 sehen werden.

Stabil vs. Instabil Eine der zentralen Eigenschaften, die auch bei analogen Systemen eine wichtige Rolle spielt, ist Stabilität. Zwar hat man normalerweise bereits einen intuitiven Begriff für Stabilität im Sinn, doch formal gibt es hiervon verschiedene Ausprägungen. Wir beschränken uns auf die Version der **Bounded-Input-Bounded-Output (BIBO)**-Stabilität, welche fordert, dass für beschränktes Eingangssignal $x[\cdot]$ der Ausgang $y[\cdot]$ ebenfalls beschränkt bleibt. Formal fordern wir also, dass falls

$$|x[n]| \leq M_x \quad \text{für alle } n \in \mathbb{Z}$$

für eine Konstante $M_x \in \mathbb{R}$ gilt, dann auch

$$|y[n]| \leq M_y \quad \text{für alle } n \in \mathbb{Z}$$

gelten muss. Man sieht, dass M_x und M_y an $x[\cdot]$ und $y[\cdot]$ gebunden sind, also keine „universellen“ Konstanten sind. Andersherum reicht es also für Instabilität nur *ein* beschränktes Eingangssignal $x[\cdot]$ konstruiert werden muss, für welches $y[\cdot]$ nicht beschränkt bleibt. Betrachten wir folgendes

Beispiel 3.3. Gegeben sei das System

$$y[n] = C \cdot y[n-1]^2 + x[n]$$

und wir nutzen als Eingang den Einheitsstoß $x[\cdot] = C\delta[\cdot]$, welcher beschränkt ist, mit $M_x = |C|$ für $C \in \mathbb{R}$. Doch dieser produziert für den Startwert $y[n] = 0$ für $n \leq -1$ die Folge

$$y[n] = \{0, \underset{\uparrow}{C}, C^2, C^4, \dots, C^{2^n}\},$$

welche für $|C| > 1$ gegen ∞ divergiert. Es gibt also einen beschränkten Eingang für welchen das Ausgangssignal nicht beschränkt bleibt. Also ist das System für $C > 1$ nicht stabil.

Zeitvariant vs. Zeitinvariant Systeme deren Eingabe-Ausgabe-Verhalten nicht zeitlich konstant ist, nennt man *zeitvariant*. Systeme, die für zeitlich verzögerte Eingaben, die um den gleichen Zeitraum verzögerte Ausgaben produzieren, nennt man *zeitinvariant*, siehe Abschnitt 1.2.6. Formal fordern wir für Zeitinvarianz, dass wenn für beliebige Eingabe $x[\cdot]$

$$x[\cdot] \xrightarrow{\mathcal{T}} y[\cdot]$$

gilt, dass dann für jedes $k \in \mathbb{Z}$ auch

$$x[\cdot - k] \xrightarrow{\mathcal{T}} y[\cdot - k]$$

erfüllt ist. In der Schreibweise von (3.2.1) heißt dies, dass für

$$y[n] = \mathcal{T}(x[\cdot])[n]$$

auch gelten muss, dass

$$y[n - k] = \mathcal{T}(x[\cdot - k])[n]$$

und zwar für alle $x[\cdot]$ und k .

Was erst einmal relativ abstrakt daherkommt, ist eigentlich eine sehr intuitive Sache. Wenn wir beispielsweise an ein Audiorecordinginterface denken, so hätten wir schon gerne, dass es egal ist, zu welchem Zeitpunkt jemand ins Mikrophon singt – unabhängig vom Zeitpunkt sollte die Aufnahme „gleich klingen“. Das System, welches die Aufnahme und eventuelle Audioverarbeitung realisiert, sollte keine zeitlichen Veränderungen zeigen. Auch in der umgekehrten Richtung, beim Abspielen von Ton, sollte es irrelevant sein, zu welchem Zeitpunkt man ein gewisses Stück hören möchte – das Hörerlebnis sollte davon nicht beeinflusst sein.

Im Grunde ist Zeitinvarianz also etwas, das wir normalerweise von einem System „erwarten“ oder implizit annehmen und nicht extra untersuchen beziehungsweise überprüfen wollen.

Linear vs. Nicht-Linear Kommen wir zum Schluss dieser Klassifikationen zu einer der wichtigsten Unterscheidungen. Ein System, das für Eingänge $x_1[\cdot]$ und $x_2[\cdot]$ die Antworten

$$y_1[n] = \mathcal{T}(x_1[\cdot])[n] \quad \text{und} \quad y_2[n] = \mathcal{T}(x_2[\cdot])[n]$$

produziert, nennen wir *linear*, falls die Antwort des Systems auf den Eingang

$$x[\cdot] = a_1 x_1[\cdot] + a_2 x_2[\cdot]$$

sich durch

$$y[n] = \mathcal{T}(x[\cdot])[n] = \mathcal{T}(a_1 x_1[\cdot] + a_2 x_2[\cdot])[n] = a_1 \mathcal{T}(x_1[\cdot])[n] + a_2 \mathcal{T}(x_2[\cdot])[n]$$

ausdrücken lässt.

Abbildung 3 zeigt die beiden möglichen Interpretationen dieser Eigenschaft. Man kann sich also vorstellen, dass die Eingänge *erst* skaliert und addiert werden und dann das System durchlaufen, oder man prozessiert beide Eingänge durch $\mathcal{T}$ und skaliert und addiert die *Ausgänge nachdem* das System im Grund „zweifach“ angewandt wurde.

Anders gesagt „passen“ lineare Systeme genau zu der linearen Struktur von Signalen, wenn wir sie als Vektoren in einem Vektorraum auffassen. Als Konsequenz ergibt sich, dass sich lineare Systeme deutlich einfacher analysieren lassen, weil wir Werkzeuge der linearen Algebra benutzen können. Diese Eigenschaft ist so attraktiv, dass man oft versucht nichtlineare Systeme durch geeignete lineare System zu approximieren (Bspw.: $\sin(x) \approx x$). Man nimmt also Fehler in der Analyse in Kauf⁵, ist damit aber immerhin in der Lage überhaupt Aussagen treffen zu können.

⁵https://en.wikipedia.org/wiki/Hartman-Grobman_theorem

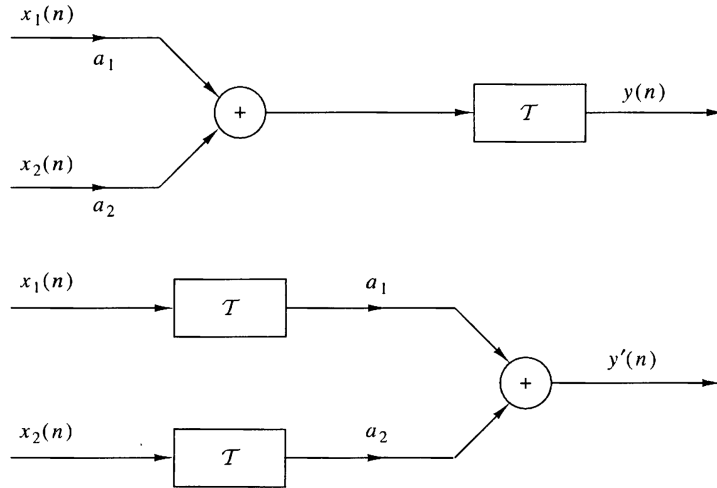


Abbildung 3: zeigt zwei verschiedene systemtheoretische Interpretation von linearen Systemen. Quelle: [1]

3.3 Diskrete LTI-Systeme

Wir schränken nun die Menge der Systeme ein, die wir betrachten wollen, indem wir fordern, dass das System $\mathcal{T}$ gleichzeitig linear und zeitinvariant ist. Das heißt, es gilt einerseits, dass ein Eingang $x[\cdot]$ zum System $\mathcal{T}$ mit Ausgang $y[\cdot]$ bei Verzögerung zu $x[\cdot - k]$ den entsprechend verzögerten Ausgang $y[\cdot - k]$ zur Folge hat. Gleichzeitig kann der Ausgang des Systems $\mathcal{T}$ für Eingänge, die lineare Superpositionen sind als lineare Superposition von den entsprechenden Ausgängen ausgedrückt werden, siehe Abbildung 3.

3.3.1 Faltungsformel

Wir wollen die Struktur von LTI-Systemen ausnutzen, um eine allgemeine und einfache Formel für das Eingangs-Ausgangsverhalten von jenen angeben zu können. Als Erstes verallgemeinern wir hierzu Abbildung 3 zu beliebigen, aber endlichen Summen, also gegeben ist ein Eingang $x[\cdot]$ der Form

$$x[n] = \sum_{k=1}^K a_k \cdot x_k[n], \quad (3.3.1)$$

wobei wir annehmen zu wissen, wie die Ausgänge von jedem $x_k[\cdot]$ zu berechnen sind, also

$$y_k[n] = (\mathcal{T}x_k[\cdot])[n].$$

Dann wissen wir wegen der Linearität und (3.3.1), dass

$$y[n] = \left[\mathcal{T} \left(\sum_{k=1}^K a_k \cdot x_k[\cdot] \right) \right] [n] = \sum_{k=1}^K a_k (\mathcal{T}x_k[\cdot])[n] = \sum_{k=1}^K a_k y_k[n] \quad (3.3.2)$$

gelten muss. Es lohnt sich einige Zeit über diese Sache zu meditieren und es gibt verschiedene Interpretationen.

- Wie bereits erwähnt passt dies zur linearen Struktur des Systems $\mathcal{T}$.

- Ist ein Eingangssignal aus anderen Signalen zusammengesetzt, dann setzt sich die Reaktion des Systems auf dieses zusammengesetzte Signal aus den Reaktionen auf die Signalbausteine zusammen. Wichtig ist hierbei, dass die Art der Zusammensetzung sich nicht ändert. Die a_k in (3.3.2) sind die gleichen, wie in (3.3.1).

Wir wollen nun einen Schritt weiter gehen und eine Menge von $x_k[\cdot]$ angeben, die es erlauben *alle* möglichen Signale darzustellen. Dazu betrachten wir, was geschieht, wenn wir den Einheitsstoß $\delta[\cdot]$ mit einem beliebigen Signal multiplizieren. Wir rechnen demzufolge für ein beliebiges Signal

$$x[n] \cdot \delta[n] = \begin{cases} x[0] & \text{für } n = 0 \\ 0 & \text{sonst.} \end{cases}$$

Wenn wir nun die Einheitsstöße verschieben um $k \in \mathbb{Z}$ erhalten wir

$$x[n] \cdot \delta[n - k] = \begin{cases} x[k] & \text{für } n = k \\ 0 & \text{sonst.} \end{cases}$$

Im Grunde „pickt“ $\delta[\cdot - k]$ bei Multiplikation den Wert von $x[\cdot]$ an der Stelle k heraus. Deshalb können wir nun schreiben

$$x[n] = \sum_{k \in \mathbb{Z}} x[k] \cdot \delta[n - k], \quad (3.3.3)$$

was nach Definition von $x_k[\cdot] = \delta[\cdot - k]$ genau die Form von (3.3.1) mit $a_k = x[k]$ annimmt.

Die Werte des Eingangssignals werden demnach zu den Faktoren in der Linearkombination der $x_k[\cdot] = \delta[\cdot - k]$. Die Kernbeobachtung ist also, dass jedes $x_k[\cdot]$ eine verschobene Kopie von $\delta[\cdot]$ ist. Das heißt, dass wir nun die LTI-Eigenschaft ausnutzen können, weil (3.3.2) impliziert, dass wir nur $(\mathcal{T}\delta[\cdot])[\cdot]$ berechnen müssen und $(\mathcal{T}\delta[\cdot - k])[\cdot]$ sich wegen der Zeitinvarianz als $(\mathcal{T}\delta[\cdot])[\cdot - k]$ ergibt – und das ohne Rechnen!

Wir geben dem Kind nun einen Namen, also definieren wir $h : \mathbb{Z} \rightarrow \mathbb{C}$ als die Antwort des Systems $\mathcal{T}$ auf den Eingang $\delta[\cdot]$, also

$$h[n] = (\mathcal{T}\delta[\cdot])[n]. \quad (3.3.4)$$

Man nennt $h[\cdot]$ die *Impulsantwort* des Systems. Dann können wir mit (3.3.2) folgern, dass

$$y[n] = (\mathcal{T}x[\cdot])[n] = \sum_{k \in \mathbb{Z}} x[k]h[n - k] \quad (3.3.5)$$

gelten muss. Doch diese Art der Operation kennen wir sehr gut – es ist eine gewöhnliche Faltung von zwei diskreten Signalen!

Das heißt, dass sich die Antwort $y[\cdot]$ eines diskreten LTI-Systems aus der *Faltung* des Einganges $x[\cdot]$ mit der Impulsantwort $h[\cdot]$ ergibt. Wir schreiben in Kurzform

$$y[n] = (x * h)[n] = (h * x)[n].$$

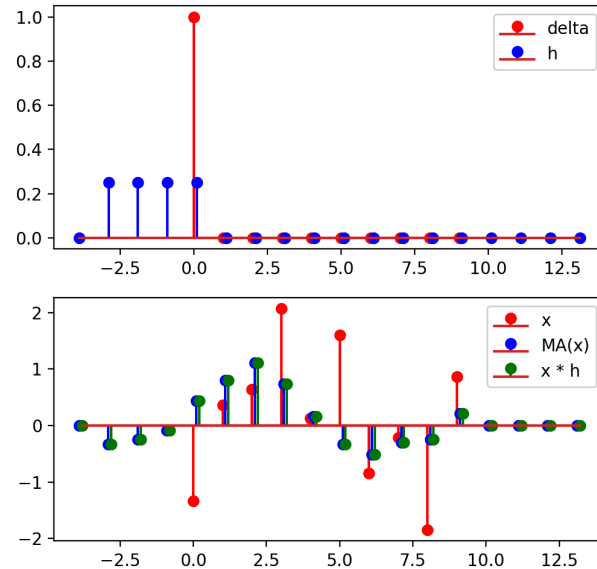
Es wird sich zeigen, dass sich viele Eigenschaften des Systems $\mathcal{T}$ aus oft signifikant einfacher zu prüfenden Eigenschaften der Impulsantwort $h[\cdot]$ ergeben. Das heißt, dass $h[\cdot]$ in gewisser Weise das System $\mathcal{T}$ *vollständig repräsentiert*.

In Codeschnipsel 10 zeigen wir das Verhalten eines gleitenden Mittelwertes (*moving average*), welches sich durch

$$y[n] = \frac{1}{\ell} \sum_{k=0}^{k=\ell} x[n - k]$$

```
def MA(x, l):
    xpad = np.pad(x, (l, l))
    y = np.zeros_like(xpad)
    for n in range(x.size + l):
        y[n] = np.mean(xpad[n : n + l])
    return y
```

```
delta = np.eye(N)[0, :]
h = MA(delta, l)
x = np.random.randn(N)
y1 = MA(x, l)
y2 = np.convolve(x, h[2 * l + 1:])
```



Codeschnipsel 10: Gleitendes Mittel mit Länge $\ell = 4$. Wir vergleichen die direkte Berechnung mit der Berechnung über die Faltung, siehe [dsv/code/moving_average.py](#)

ergibt. Man sieht schön, dass durch das Mitteln die (in diesem Fall) zufällige Eingabe-Sequenz am Ausgang geglättet erscheint. Wichtig bei diesem Beispiel ist die Tatsache, dass wir direkt einsehen, dass Anwendung der direkten Formel für das gleitende Mittel aus eine Eingabe $x[\cdot]$ denselben Effekt hat, wie die Faltung mit $h[\cdot]$, das sich aus der Anwendung der Mittelung auf $\delta[\cdot]$ ergibt.

Es lohnt sich, sich einige Eigenschaften der Faltung zu merken. Diese sind:

- **Bi-Linearität:** Es gilt $(a_1 x_1[\cdot] + a_2 x_2[\cdot]) * h[\cdot] = a_1 (x_1 * h)[\cdot] + a_2 (x_2 * h)[\cdot]$. Dies ist die „normale“ Linearität in den Eingängen, die sich aus der Linearität des Systems $\mathcal{T}$ ergibt. Es gilt aber auch $(x * (a_1 h_1 + a_2 h_2))[\cdot] = a_1 (x * h_1)[\cdot] + a_2 (x * h_2)[\cdot]$. Das heißt, wenn wir ein System $\mathcal{T} = a_1 \mathcal{T}_1 + a_2 \mathcal{T}_2$ gegeben haben, dann ist die Impulsantwort des Systems $\mathcal{T}$ die gleiche Linearkombination der Impulsantworten $h_1[\cdot]$ und $h_2[\cdot]$ der beiden Systeme $\mathcal{T}_{1,2}$. Das heißt wiederum, dass **LTI**-Systeme selbst ein linearer Raum sind! Wir sprechen hier von *Bi*-Linearität, weil die Faltung eben linear in zwei Argumenten ist.
- **Die Faltung ist assoziativ:** Es gilt also, dass $(x * h_1) * h_2 = x * (h_1 * h_2)$. Dies impliziert, dass die Verkettung von zwei **LTI**-Systemen wieder ein **LTI**-system ergibt, wobei sich die Impulsantwort der Verkettung durch Faltung der beiden Impulsantworten der verketteten Systeme ergibt.
- **Kommutativität:** Es gilt $h_1 * h_2 = h_2 * h_1$, demnach auch, dass $x * (h_1 * h_2) = x * (h_2 * h_1)$. Das heißt erstaunlicherweise, dass man verkettete **LTI**-Systeme in ihrer Reihenfolge vertauschen kann, ohne das Eingangs-Ausgungsverhalten des Gesamtsystems zu beeinflussen.

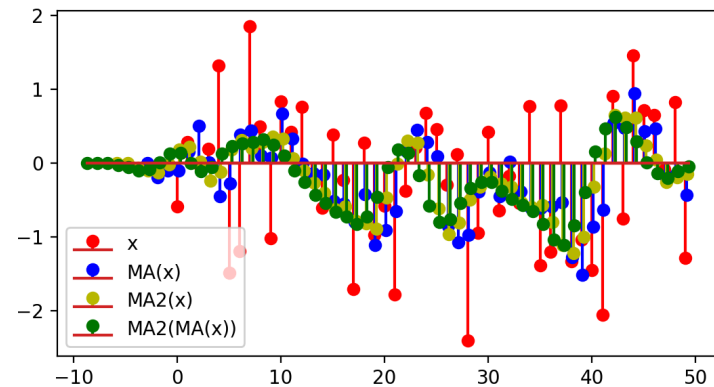
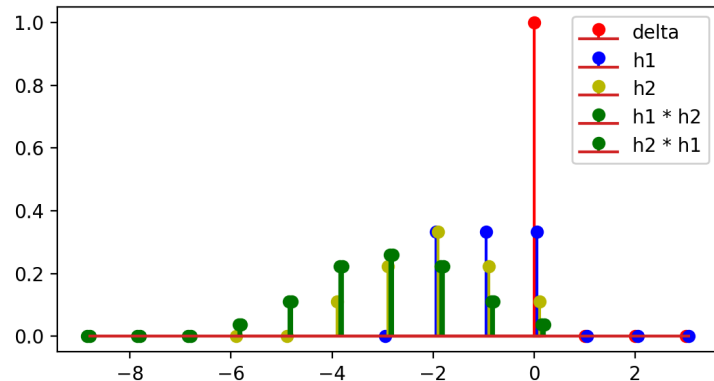
In Codeschnipsel 11 untersuchen wir einige der oben genannten Eigenschaften. Einerseits sehen wir, dass die Impulsantwort von $\text{MA}(\text{MA2}(\cdot))$ mit der von $\text{MA2}(\text{MA}(\cdot))$ identisch ist. Wir haben also Kommutativität nachgeprüft. Wir sehen außerdem, dass sich die Impulsantwort der Verkettung aus Faltung der einzelnen

```
def MA(x, l):
    xpad = np.pad(x, (l, l))
    y = np.zeros_like(xpad)
    for n in range(x.size + l):
        y[n] = np.mean(xpad[n : n + l])
    return y[:-l]
```

```
def MA2(x, l):
    return MA(MA(x, l), l)
```

```
delta = np.eye(N)[0, :]
h1 = MA(delta, l)
h2 = MA2(delta, l)
h3 = MA2(MA(delta, l), l)
h4 = MA(MA2(delta, l), l)
```

```
x = np.random.randn(N)
y1 = MA(x, l)
y2 = MA2(x, l)
y3 = MA(MA2(x, l), l)
```



Codeschnipsel 11: Verkettung von mehreren Moving Averages der Länge $\ell = 3$, siehe [dsv/code/ramp_ma.py](#)

Impulsantworten ergibt. Aus dem abgetasteten „Rechteck“ wird nach nochmaliger Anwendung ein abgetastetes, aber breiteres, „Dreieck“, was schlussendlich zu einer abgetasteten, stückweise quadratischen, Impulsantwort wird. Generell kann man das komplette System als eine dreifache Verkettung gleitender Mittel der Länge $\ell = 3$ verstehen. Außerdem bestätigt Codeschnipsel 11 bei Vergleich der verschiedenen Ausgänge, dass wiederholtes Mitteln am Ausgang mit Anzahl der Mittelungen zunehmend „glattere“ Signale erzeugt.

3.3.2 Eigenschaften von LTI-Systemen

BIBO-Stabilität Wir hatten vorher schon diesen Begriff der Stabilität eingeführt und formulieren nun eine Bedingung an $h[\cdot]$, die es uns erlaubt auf Stabilität zu prüfen. Wir nennen einen Eingang $x[\cdot]$ beschränkt, falls ein $M_x < \infty$ existiert, sodass

$$|x[n]| \leq M_x \quad \text{für alle } n \in \mathbb{Z}$$

erfüllt ist. Dann können wir mit der Faltungsformel (3.3.5) berechnen, dass

$$|y[n]| = \left| \sum_{k \in \mathbb{Z}} x[k]h[n-k] \right| \leq \sum_{k \in \mathbb{Z}} |x[k]h[n-k]| \leq M_x \sum_{k \in \mathbb{Z}} |h[n-k]|$$

gilt. Damit nun $|y[n]| < \infty$, muss also gelten, dass

$$\sum_{k \in \mathbb{Z}} |h[n-k]| < \infty.$$

Das heißt, wenn $h[\cdot]$ absolut summierbar ist, dann ist das System mit $h[\cdot]$ als Impulsantwort **BIBO**-stabil.

Wie ist es um die umgekehrte Schlussfolgerung bestellt? Impliziert **BIBO**-Stabilität, dass $h[\cdot]$ absolut summierbar sein muss? Erinnern wir uns daran, dass man **BIBO**-Stabilität widerlegen kann, indem man einen beschränkten Eingang $x[\cdot]$ findet, für welchen der Ausgang $y[\cdot]$ nicht beschränkt bleibt.

Nehmen wir an, dass

$$\sum_{k \in \mathbb{Z}} |h[n-k]| = \infty$$

und betrachten wir den Eingang

$$x[n] = \begin{cases} \frac{h[-n]^*}{|h[-n]|}, & \text{für } h[n] \neq 0 \\ 0 & \text{sonst.} \end{cases}$$

Man sieht, dass $|x[n]| \leq 1$, es ist also beschränkt. Dann berechnen wir einfach den ersten Wert am Ausgang mit (3.3.5) und der Definition des Einganges $x[\cdot]$, also

$$y[0] = \sum_{k \in \mathbb{Z}} x[-k]h[k] = \sum_{k \in \mathbb{Z}} \frac{|h[k]|^2}{|h[k]|} = \sum_{k \in \mathbb{Z}} |h[k]| = \infty,$$

wobei das letzte Gleichheitszeichen gilt, weil wir angenommen hatten, dass $h[\cdot]$ nicht absolut summierbar ist. Man kann also schlussfolgern, dass sich Stabilität vollständig durch die Impulsantwort $h[\cdot]$ bestimmen lässt, weil absolute Summierbarkeit der Impulsantwort äquivalent zur **BIBO**-Stabilität ist. Eine tolle Sache!

FIR vs. IIR Systeme Gilt für die Impulsantwort $h[\cdot]$, dass

$$h[n] = 0, \quad \text{für } n < m, M \leq n$$

für zwei Zahlen $m \leq M$, dann bezeichnet man das zugehörige System $\mathcal{T}$ als **Finite Impulse Response (FIR)**-System. Ist obige Bedingung nicht erfüllt, so nennt man $\mathcal{T}$ ein **Infinite Impulse Response (IIR)**-System. Bei **FIR**-Systemen sind für die Berechnung von $y[n]$ nur endlich viele Werte, maximal $M - m$ viele, notwendig. Das System hat also ein endliches Gedächtnis der Länge $M - m$, wobei **IIR**-Systeme ein unendlich langes Gedächtnis haben. Sobald bei einem **FIR**-System $|h[n]| < \infty$ für alle $n \in \mathbb{Z}$ gilt, ist es auch stabil – also sind in der Praxis *alle* **FIR**-Systeme stabil.

Bei einem **IIR**-System ist die Frage nach Stabilität nicht so einfach zu beantworten, doch wir werden im Folgenden noch ein Werkzeug kennenlernen, mit welchem dies einfach nachzuprüfen sein wird. Außerdem haben **IIR**-Systeme noch das Problem, dass man die Faltungsformel (3.3.5) nicht nutzen kann, um ein **IIR**-System zu *implementieren*.

3.3.3 Rekursive IIR-Systeme

Um die Notwendigkeit von (3.3.5) zu umgehen, betrachten wir eine gewisse Klasse von Systemen, deren Ausgang $y[\cdot]$ auch über einen alternativen Weg bestimmt werden kann. Hierzu betrachten wir

$$y[n] = \frac{1}{n+1} \sum_{k=0}^n x[k],$$

was ein kumulatives Mittel von 0 bis n darstellt. Wichtig ist hier, dass dies ein Beispiel für ein sogenanntes **Linear Time-Variant (LTV)**-System. Wegen der Linearität existiert immernoch eine Impulsantwort, die durch Faltung mit dem Eingang $x[\cdot]$ den Ausgang $y[\cdot]$ ergibt, diese ist aber zeitvariant, also $h_n[\cdot]$. So wie es hier scheint, muss man für die Berechnung von $y[n]$ alle vergangenen Werte von $x[\cdot]$ bereitliegen haben. Doch mit einer einfachen Umformung findet man, dass

$$(n+1)y[n] = ny[n-1] + x[n] \Leftrightarrow y[n] = \frac{n}{n+1}y[n-1] + \frac{1}{n+1}x[n].$$

Wir können also alternativ $y[n]$ aus $x[n]$ und $y[n-1]$ berechnen. Da also $y[n]$ von $y[n-1]$ abhängt, nennen wir das System *rekursiv*. Man sieht, dass diese Umformulierung deutlich macht, dass wir nur $y[n-1]$ im Speicher halten müssen und dieses mit einer Art *zeitverzögertem Feedback* ausstatten müssen. Hierbei ist natürlich die Zeitverzögerung essenziell, denn müssten wir $y[n]$ in Abhängigkeit von $y[n]$ berechnen, würde uns das vor ein halbwegs unlösbares Problem stellen. In Codeschnipsel 12 sehen wir die beiden unterschiedlichen Implementierungen. Es ist hier zu beachten, dass `y[nn] = np.sum(x[:nn]) / (nn + 1)` immer auf die komplette Vergangenheit von $x[\cdot]$ zugreift und eben *nicht* auf Werte von $y[\cdot]$.

Für rekursive Systeme entsteht noch das Problem, dass wir beispielsweise für den Beginn der rekursiven Berechnung von $y[n]$ ab einem gewissen $n_0 \in \mathbb{Z}$ den Wert $y[n_0 - 1]$ kennen müssen. Das heißt wir benötigen einen *Startwert* für die Rekursion. Dieser beeinflusst auch maßgeblich das Verhalten des Systems.

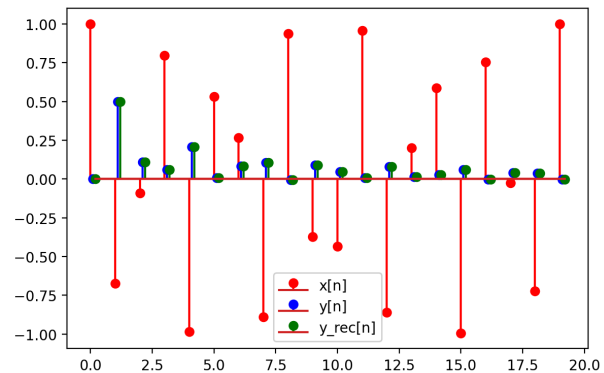
Eine Möglichkeit, wie man das produktiv ausnutzen kann, ist für die Berechnung der Quadratwurzel einer positiven reellen Zahl A . Für die Herleitung definieren wir erst die Funktion $f: \mathbb{R} \rightarrow \mathbb{R}$ via

$$f(x) = x^2 - A.$$

```
def cumulative_sum(
    x: np.ndarray, n: np.ndarray
) -> np.ndarray:
    y = np.zeros(n.size)
    for nn in n:
        y[nn] = np.sum(x[:nn]) / (nn + 1)

    return y

def cumulative_sum_rec(
    y: float, x: float, n: int
):
    return (n / (n + 1.0)) * y + 1.0 / (
        n + 1
    ) * x
```



Codeschnipsel 12: Die beiden möglichen Implementierungen eines kumulativen Mittelwertes, siehe [dsv/code/cumulative_sum.py](#)

Wie man leicht sieht, hat diese Funktion Nullstellen $x_{1,2} = \pm\sqrt{A}$. Die sogenannte Newton-Iteration⁶ berechnet iterativ via

$$y[n] = y[n-1] - \frac{f(y[n-1])}{f'(y[n-1])}$$

eine Nullstelle der Funktion f . Setzen wir unsere Funktion ein und definieren $x[\cdot] = Au[\cdot]$, dann erhalten wir

$$y[n] = \frac{1}{2} \left(y[n-1] + \frac{x[n]}{y[n-1]} \right)$$

Nutzen wir einen geeigneten Startwert, beispielsweise $y[-1] = 1$, so erhalten wir eine *sehr schnell* konvergierende Folge von $y[n]$.

Die Folge konvergiert so schnell, dass bereits nach 5 Iterationen der Fehler an der endlichen Genauigkeit von Gleitkommaarithmetik kratzt. Es ist auch darauf hinzuweisen, dass das System nur sehr einfache arithmetische Operationen benutzt, um die numerisch nicht ganz triviale Berechnung von $\sqrt{A}$ beliebig genau und sehr schnell zu approximieren.

Abschließend ist noch anzumerken, dass man rekursive Systeme auch „lösen“ kann, siehe [1, Kap. 2.4.3], indem man aus der rekursiven Vorschrift eine explizite entwickelt. Die dort entwickelte Theorie erinnert der Lösung von Differenzialgleichungen nach Funktionen, nur werden stattdessen Differenzengleichungen nach diskreten Folgen gelöst.

3.3.4 Approximation von IIR-Systemen durch FIR-Systeme

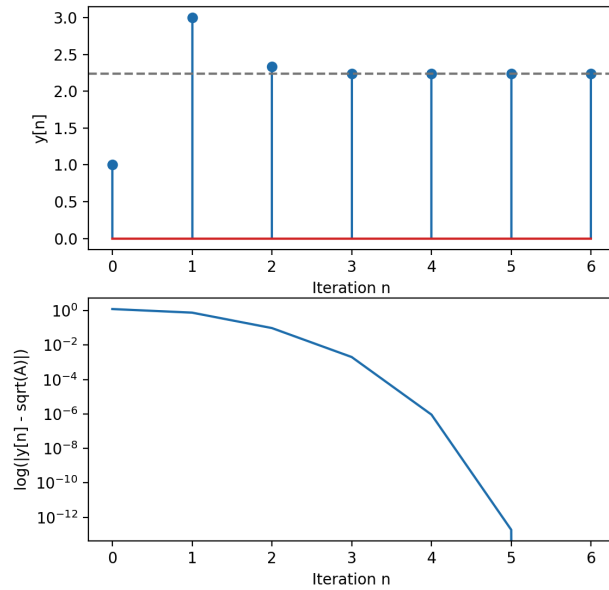
Um die Faltungsformel doch anwenden zu können, kann man für ein *stabiles* IIR-System mit Impulsantwort $h[\cdot]$ eine FIR-Approximation finden. Ein weiterer Vorteil ergibt sich dann, dass auch dieses resultierende

⁶https://encyclopediaofmath.org/index.php?title=Newton_method

```
def root(A: float, y0: float, n):
    y = [y0]
    for ii in range(n):
        y.append((y[-1] + A / y[-1]) / 2)

    return y
```

```
A = 5.0
rootA = np.sqrt(A)
y = root(A, 1.0, 6)
```



Codeschnipsel 13: Iteratives Verfahren zur Berechnung von $\sqrt{A}$, siehe [dsv/code/square_root.py](#)

System stabil sein muss. Für $n_{\text{low}} < n_{\text{high}}$ definieren wir

$$h_{\text{FIR}}[n] = \begin{cases} h[n], & \text{falls } n_{\text{low}} \leq n \leq n_{\text{high}}, \\ 0 & \text{sonst.} \end{cases}$$

Da aus der Stabilität von dem System mit Impulsantwort $h[\cdot]$ folgt, dass

$$\lim_{k \rightarrow \infty} \sum_{n=k}^{\infty} (|h[n]| + |h[-n]|) = 0,$$

da sonst

$$\sum_{n \in \mathbb{Z}} |h[n]|$$

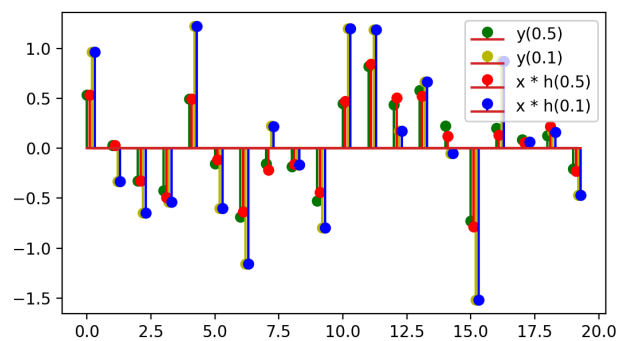
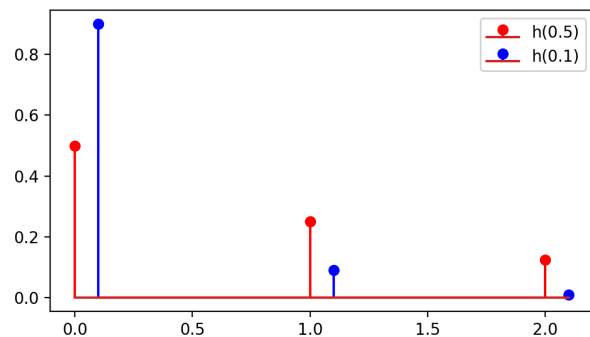
nicht endlich sein könnte. Das heißt, dass die Werte der Impulsantwort $h[\cdot]$ gegen 0 konvergieren müssen und deshalb kann man $n_{\text{low}} < n_{\text{high}}$ so wählen, dass der Unterschied zwischen $h[\cdot]$ und $h_{\text{FIR}}[\cdot]$ beliebig klein wird.

Je nachdem wie schnell $h[\cdot]$ gegen 0 konvergiert, benötigen wir mehr oder weniger von 0 verschiedene Einträge in h_{FIR} , was sich auf den Implementierungsaufwand auswirkt.

Ein Beispiel für einen exponentiellen Mittelwert-Filter findet man in Codeschnipsel 14. Man sieht, dass für $\alpha = 0.5$ der Abschneidefehler deutlicher zutage tritt, da die Faltung mit $h_{\text{FIR}}[\cdot]$ in diesem Fall ein deutlich schlechteres Ergebnis liefert als im Falle von $\alpha = 0.1$. Dies liegt eben daran, dass bei $\alpha = 0.1$ der IIR-Filter *effektiv* ein FIR-Filter wird. Wir wollen aber darauf hinweisen, dass es eine ganz eigene Theorie für die Approximation von Filtern durch andere Filter gibt, die wir hier nicht einmal anreißen wollen.

```
def exp_mean(
    x: np.ndarray, alpha: float
) -> np.ndarray:
    y = np.zeros_like(x)
    y[0] = 0
    for ii in range(1, y.size):
        y[ii] = (
            alpha * y[ii - 1]
            + (1 - alpha) * x[ii]
        )
    return y
```

```
alpha1 = 0.5
alpha2 = 0.1
N = 20
L = 3
n = np.arange(N)
l = np.arange(L)
delta = np.eye(L)[0]
h1 = exp_mean(
    np.pad(delta, (L, L), mode="constant"),
    alpha1,
)[L:-L]
h2 = exp_mean(
    np.pad(delta, (L, L), mode="constant"),
    alpha2,
)[L:-L]
```



Codeschnipsel 14: Approximation eines exponentiellen Mittelwertes, also eines IIR-Systems, durch ein FIR-System, siehe [dsv/code/exp_mean.py](https://dsv.code/exp_mean.py)

3.4 z-Transformation

Wir wollen nun eines der wichtigsten Werkzeuge für die Analyse von diskreten **LTI**-Systemen einführen und untersuchen. Die z-Transformation nimmt für diskrete **LTI**-Systeme dieselbe Rolle ein, wie die Laplace-Transformation für zeitstetige **LTI**-Systeme. Wir werden sehen, dass die Faltungs-Formel (3.3.5) im z-Bereich eine einfachere Form annimmt. Außerdem erlaubt uns die Transformation eine neue Sicht auf diskrete Systeme zu gewinnen, mit welcher es eventuell einfacher sein sollte gewisse Eigenschaften von Systemen nachzuprüfen. Wir werden auch sehen, dass sich manche Signale oder Systeme im z-Bereich kompakter aufschreiben lassen.

Die z-Transformation $X_z : \mathbb{C} \rightarrow \mathbb{C}$ eines Signals $x[\cdot] : \mathbb{N} \rightarrow \mathbb{C}$ ist die komplexe Potenzreihe⁷

$$z \mapsto X_z(z) = \sum_{n \in \mathbb{Z}} x[n]z^{-n}. \quad (3.4.1)$$

Wegen der Summation über ganz $\mathbb{Z}$ ist $X_z(z)$ für jedes z ein Grenzwert, der nicht existieren muss. Das heißt, es ist notwendig auch *immer* den Konvergenzbereich, oder Englisch die **Region of Convergence (ROC)**, von X_z anzugeben.

Betrachten wir beispielsweise

$$x[\cdot] = \{5, 6, \underset{\uparrow}{3}, 4, -\pi\},$$

dann ergibt sich

$$X_z(z) = 5z^2 + 6z + 3 + 4z^{-1} - \pi z^{-2} \quad \text{mit ROC } \mathbb{C} \setminus \{0\}.$$

Im Fall von $x[\cdot] = \delta[\cdot - k]$, ergibt sich $X_z(z) = z^{-k}$ mit **ROC** $\mathbb{C} \setminus \{0\}$, wobei sich bei $x[\cdot] = \delta[\cdot + k]$, sich $X_z(z) = z^k$ mit **ROC** $\mathbb{C}$ ergibt. Man sieht nun auch, weshalb das System, das ein Signal um einen Abtastwert verzögert, oft mit z^{-1} bezeichnet wird, weil dies die z-Transformation dessen Impulsantwort ist. Außerdem kann man bei endlichen Signalen im z-Bereich deren Werte im Zeitbereich einfach ablesen. Will man den Wert $x[n]$, also an Sample n , ermittelt man diesen, indem man den Koeffizienten vom passenden z^{-n} betrachtet.

Aber auch für unendlich lange Signale, wie beispielsweise $x[n] = a^n u[n]$, ergibt sich

$$X_z(z) = 1 + az^{-1} + a^2 z^{-2} + \dots = \sum_{n=0}^{\infty} a^n z^{-n} = \sum_{n=0}^{\infty} (az^{-1})^n,$$

was eine geometrische Reihe über az^{-1} darstellt. Dann folgt, dass

$$X_z(z) = \frac{1}{1 - az^{-1}},$$

wobei diese Potenzreihe nur konvergiert, wenn $z > a$. Das heißt, dass die **ROC** hier alle $z \in \mathbb{C}$ umfasst mit $|z| > |a|$. Wir sehen, dass sich eigentlich unendlich lange Signale im z-Bereich durch Berechnen des Grenzwertes knapper darstellen lassen. Gleichzeitig können wir so auch die **ROC** bestimmen, indem wir ermitteln, wann der ermittelte Grenzwert existiert.

Wenn wir $z = r \exp(j\phi)$ in die Definition (3.4.1) einsetzen und $|X_z(z)|$ betrachten, finden wir

$$|X_z(z)| = \left| \sum_{n \in \mathbb{Z}} x[n]z^{-n} \right| \leq \sum_{n \in \mathbb{Z}} |x[n]z^{-n}| = \sum_{n \in \mathbb{Z}} |x[n]r^{-n}| |\exp(-jn\phi)| = \sum_{n \in \mathbb{Z}} |x[n]r^{-n}|.$$

⁷<https://mathworld.wolfram.com/LaurentSeries.html>

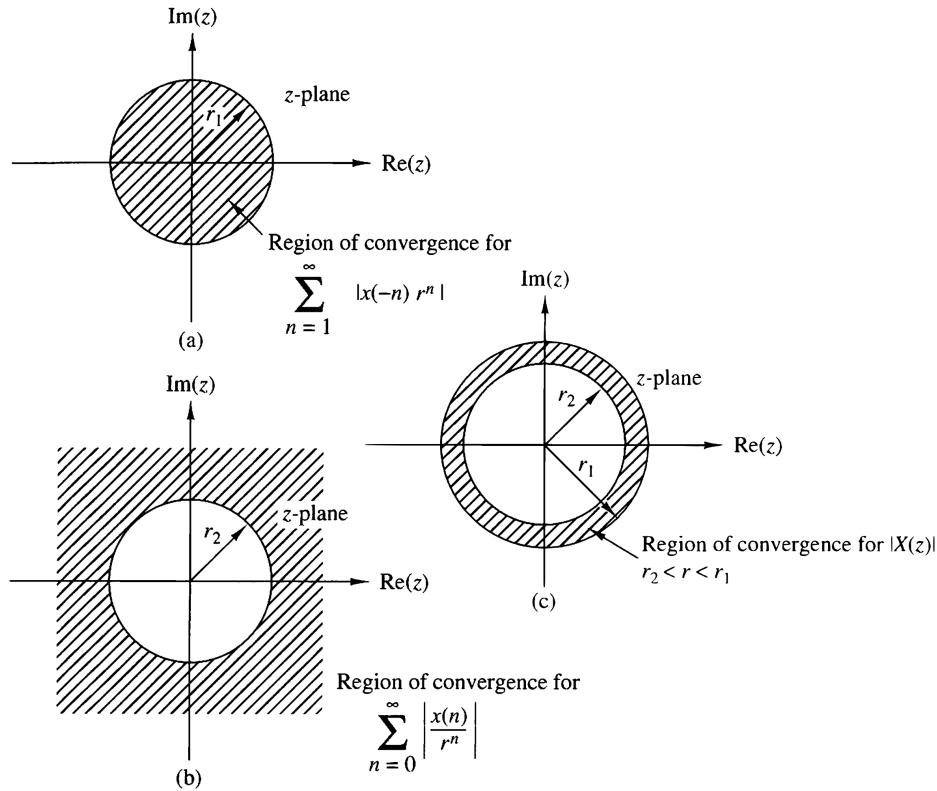


Abbildung 4: Veranschaulichung der Entstehung der ROC aus den Konvergenzkriterien. Quelle: [1]

Betrachten wir den letzten Ausdruck genauer indem wir ihn in zwei Summationen aufteilen, also

$$\sum_{n \in \mathbb{Z}} |x[n] r^{-n}| = \sum_{n=-1}^{-\infty} |x[n] r^{-n}| + \sum_{n=0}^{+\infty} |x[n] r^{-n}| = \sum_{n=1}^{+\infty} |x[-n] r^n| + \sum_{n=0}^{+\infty} \left| \frac{x[n]}{r^n} \right|.$$

Das ergibt dann also, dass

$$|X_z(z)| \leq \sum_{n=1}^{+\infty} |x[-n] r^n| + \sum_{n=0}^{+\infty} \left| \frac{x[n]}{r^n} \right|$$

Damit X_z existiert, müssen beide Grenzwerte existieren. Der erste Grenzwert existiert, wenn es ein r_1 gibt, sodass r_1^n die Werte $x[-n]$ so gewichtet, dass sich Konvergenz einstellt. Dieses r_1 ist ausschließlich durch die Werte in der „Vergangenheit“ von $x[\cdot]$ bestimmt! Wenn sich Konvergenz für ein r_1 einstellt, dann auch für alle $r < r_1$. Der Konvergenzbereich ergibt sich also als ein Kreis in der komplexen Ebene mit Radius r_1 . Der zweite Grenzwert existiert, falls für ein $r_2 > 0$ die Werte $1/r_2^n$ so die „Zukunft“ von $x[n]$ so wichten, dass die Summe konvergiert. Das heißt, falls so ein r_2 existiert, dann konvergiert der zweite Summand für alle $r > r_2$.

Die **ROC** ist also die Schnittmenge von der Menge $\{|z| \leq r_1\}$ mit der Menge $\{|z| \geq r_2\}$. Dies ist in Abbildung 4 zusammengefasst. Wir sehen auch, dass im Falle von $r_1 > r_2$ die **ROC** die leere Menge ist und deshalb die z -Transformation nicht existiert.

3.4.1 Eigenschaften

Wir wollen nun einige Eigenschaften der z -Transformation auflisten, die wir immer wieder verwenden werden. Besonderes Augenmerk legen wir hierbei auf die Behandlung der **ROC**.

Linearität Gegeben zwei diskrete Signale $x_{1,2}[\cdot]$ und komplexe Zahlen $a_{1,2} \in \mathbb{C}$, dann gilt für $x[\cdot] = a_1 x_1[\cdot] + a_2 x_2[\cdot]$, dass

$$X_z(z) = a_1 X_{z,1}(z) + a_2 X_{z,2}(z),$$

wobei die **ROC** von X_z der Schnittmenge der **ROCs** von $X_{z,1}$ und $X_{z,2}$ entspricht.

Zeitversatz Ist X_z die z -Transformation von $x[\cdot]$, so ist $z^{-k}X(z)$ die z -Transformation von $x[\cdot - k]$. Hierbei ist die **ROC** von X_z , falls $k \leq 0$. Im Falle $k > 0$ müssen wir aus der **ROC** von X_z den Wert $z = 0$ entfernen. Intuitiv sollte man diese Eigenschaft leicht verifizieren können, da man durch das Verschieben nur die Potenz des z ändert und man diese Änderung einfach ausklammern kann.

Um beispielsweise die z -Transformation von

$$x[n] = \begin{cases} 1, & \text{für } 0 \leq n < N \\ 0 & \text{sonst} \end{cases},$$

zu bestimmen, könnten wir einerseits die Definition bemühen, oder die beiden schon bekannten Eigenschaften ausnutzen, um zu folgern, dass

$$X_z(z) = \begin{cases} N & \text{für } z = 1, \\ \frac{1-z^{-N}}{1-z^{-1}} & \end{cases}$$

gilt. Dazu nutzen wir, dass $x[n] = u[n] - u[n - N]$ gilt. Da außerdem $x[\cdot]$ nur endlich viele Werte verschieden von 0 hat, ist die **ROC** von X_z die ganze z -komplexe Ebene, außer $z = 0$.

Skalierung im z -Bereich Ist X_z die z -Transformation von $x[\cdot]$ mit **ROC** $r_1 < |z| < r_2$, dann ist $a^n x[n]$ die inverse z -Transformation von $X(a^{-1}z)$ mit **ROC** $|a|r_1 < |z| < |a|r_2$. Die Intuition hierbei ist, dass die Multiplikation mit der Folge a^n mit $a = r \exp(j\omega)$ eine Skalierung um r und eine Rotation um ω im z -Bereich bewirkt. Das heißt gewisse „geometrische“ Eigenschaften von X_z transformieren sich analog mit.

Zeitumkehrung Ist X_z die z -Transformation von $x[\cdot]$ mit **ROC** $r_1 < |z| < r_2$, dann ist $X_z(z^{-1})$ die z -Transformation von $x[-\cdot]$ mit **ROC** $1/r_1 < |z| < 1/r_2$. Mit anderen Worten entspricht Umkehrung im Zeitbereich Reflexion am Einheitskreis im z -Bereich.

Faltung im Zeitbereich Kommen wir nun zu einer der wichtigsten Eigenschaften und einem der Gründe, weshalb die z -Transformation als Analogie zur Laplace-Transformation verstanden wird. Gegeben zwei diskrete Signale $x_{1,2}[\cdot]$ und $x[\cdot] = x_1 * x_2$ deren Faltung, dann gilt

$$X_z(z) = X_{z,1}(z) \cdot X_{z,2}(z), \quad (3.4.2)$$

wobei die **ROC** mindestens die Schnittmenge der **ROCs** von $X_{z,1}(z)$ und $X_{z,2}(z)$ ist. Der Grund für das „mindestens“ ist die Tatsache, dass für gewisse z die Divergenz von $X_{z,1}$ durch rasches Tendieren von $X_{z,2}$ gegen 0 kompensiert werden kann.

Wegen ihrer Bedeutung, wollen wir uns kurz überlegen, warum die obige Eigenschaft gilt. Wir nutzen im Grunde nur die Definition (3.4.1) und die Definition der Faltung, um zu sehen, dass

$$X_z(z) = \sum_{n \in \mathbb{Z}} x[n]z^{-n} = \sum_{n \in \mathbb{Z}} \sum_{k \in \mathbb{Z}} x_1[k]x_2[n-k]z^{-n} = \sum_{k \in \mathbb{Z}} x_1[k] \sum_{n \in \mathbb{Z}} x_2[n-k]z^{-n}.$$

Jetzt nutzen wir die Eigenschaft, die sich aus dem Zeitversatz gibt zusammen mit (3.4.1), und erhalten schlussendlich

$$X_z(z) = \sum_{k \in \mathbb{Z}} x_1[k] \sum_{n \in \mathbb{Z}} x_2[n-k]z^{-n} = \sum_{k \in \mathbb{Z}} x_1[k]X_{z,2}(z)z^{-k} = X_{z,2}(z) \sum_{k \in \mathbb{Z}} x_1[k]z^{-k} = X_{z,1}(z)X_{z,2}(z).$$

Wir können also zusammenfassend folgende Herangehensweise bei der Berechnung der Faltung angeben:

1. Transformation von $x_1[\cdot]$ und $x_2[\cdot]$ zu $X_{z,1}$ und $X_{z,2}$
2. Punkweise Multiplikation von $X_z(z) = X_{z,1}(z)X_{z,2}(z)$
3. Rücktransformation von $X_z(z)$ um damit $x[\cdot] = x_1 * x_2$ zu erhalten

Inverse z-Transformation Obige Berechnung der Faltung verlangt im letzten Schritt die Invertierung der z-Transformation von X_z zu $x[\cdot]$. Im allgemeinen Fall ist man darauf angewiesen den Cauchyschen Integralsatz⁸ zu bemühen. Am Ende erlaubt dieser die Herleitung von

$$x[n] = \frac{1}{j2\pi} \oint_{C \subset \text{ROC}} X_z(z)z^{n-1}dz \quad (3.4.3)$$

als Möglichkeit der Bestimmung von $x[n]$ aus X_z . Oft sind diese Integrale jedoch nicht leicht zu Berechnen und man sollte sich auf geeignete Tabellen (siehe [1, Tabelle 3.2, Tabelle 3.3]) und die restlichen obigen Eigenschaften beziehen, wenn man z-Transformationspaare bestimmen möchte.

3.4.2 Anwendung bei diskreten LTI-Systemen

Wir kombinieren nun (3.3.5) mit (3.4.2), um eine alternative Beschreibung von LTI-Systemen zu erhalten. Wir wissen aus (3.3.5), dass sich das Eingabe-Ausgabe-Verhalten von einem LTI-System $\mathcal{T}$ durch

$$y[\cdot] = h * x$$

ausdrücken lässt, wobei $h[\cdot] = y[\cdot]$ für $x[\cdot] = \delta$. Im z-Bereich transformiert sich dies wegen (3.4.2) zu

$$Y_z(z) = H_z(z)X_z(z). \quad (3.4.4)$$

Man nennt dann H_z als z-Transformation der Impulsantwort die *Transferfunktion* des Systems $\mathcal{T}$. Wir haben so eine alternative Sichtweise auf LTI-Systeme gewonnen, weil man eben durch die z-Transformation die Impulsantwort $h[\cdot]$ mit der Transferfunktion H identifiziert. Das heißt, dass beide Beschreibungen äquivalent sind. Wir werden uns wie in Abschnitt 3.3.2 überlegen, wie man gewissen Eigenschaften von LTI-Systemen an H „ablesen“ kann.

Aus diesem multiplikativen Zusammenhang kann man bei Kenntnis von zwei Größen in der Regel die fehlende Dritte bestimmen. Kennt man beispielsweise H_z und X_z , dann kann man sehr einfach Y_z und damit

⁸https://de.wikipedia.org/wiki/Cauchyscher_Integralsatz

auch $y[\cdot]$ bestimmen. In bestimmten Anwendungen der Messtechnik ist man eher daran interessiert durch geschickte Messungen $h[\cdot]$ zu bestimmen. Dies kann unter anderem interessant sein, wenn in $h[\cdot]$ gewisse Parameter eines Systems „versteckt“ sind, die einen aber interessieren. Dann ist es oft geschickt das System durch $x[\cdot]$ anzuregen, $y[\cdot]$ zu beobachten und dann die Transferfunktion durch

$$H_z(z) = \frac{Y_z(z)}{X_z(z)}$$

zu bestimmen. Man muss sich im Klaren hierbei sein, dass es hierzu notwendig ist das Signal $x[\cdot]$ so zu wählen, dass obige Division ein hinreichend genaues Bild von $h[\cdot]$ erlaubt.

Rekursive Systeme Ist allgemein die Vorschrift von einem System gegeben durch

$$y[n] = -\sum_{k=1}^N a_k y[n-k] + \sum_{k=0}^M b_k x[n-k],$$

dann ergibt sich durch die Linearität, Zeitversatz und geschicktes Umstellen, dass

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}.$$

Das heißt, dass Systeme, die durch lineare Differenzgleichungen charakterisiert sind, besitzen eine Transferfunktion, die die Struktur einer rationalen Funktion

$$H_z(z) = \frac{B(z)}{A(z)}$$

hat, wenn wir fordern, dass A und B Polynome in z sind. Allgemein schreiben wir

$$H(z) = \frac{b_0 + b_1 z^{-1} + \dots + b_M z^{-M}}{a_0 + a_1 z^{-1} + \dots + a_N z^{-N}} = \frac{\sum_{k=1}^M b_k z^{-k}}{\sum_{k=1}^N a_k z^{-k}} = G z^{N-M} \frac{(z - z_1) \dots (z - z_M)}{(z - p_1) \dots (z - p_N)}.$$

Dann nennen wir die $z_1, \dots, z_M$ die „Nullstellen“, oder *zeros*, des Systems $\mathcal{T}$ und die Menge $p_1, \dots, p_N$ die *Pole*, oder *Polstellen*, von $\mathcal{T}$. Tritt ein p_i oder z_i mehrfach auf, so sprechen wir von einer mehrfachen Polstelle, bzw. Nullstelle. Die Anzahl an gleichen Null-/Polstellen nennt man ihre Vielfachheit.

Es gibt nun zwei Spezialfälle, die wir betrachten können. Erstens, es gilt $a_1 = \dots = a_N = 0$. In diesem Fall gilt

$$H(z) = z^{-M} \sum_{k=1}^M b_k z^{M-k}$$

und H besitzt als Polynom vom Grad M dann M Nullstellen⁹ und eine M -fache Polstelle bei $z = 0$. Tritt eine Polstelle bei $z = 0$ auf, so nennen wir diese *trivial*, da sie keinen Einfluss auf das System hat. Schließlich korrespondiert z^{-M} nur zu einer Verschiebung im Zeitbereich, was bei **LTI**-Systemen per definitionem nur eine globale Verschiebung der ganzen Systems bedeutet. Da keine nicht-trivialen Polstellen auftreten, hat das

⁹https://de.wikipedia.org/wiki/Fundamentalsatz_der_Algebra

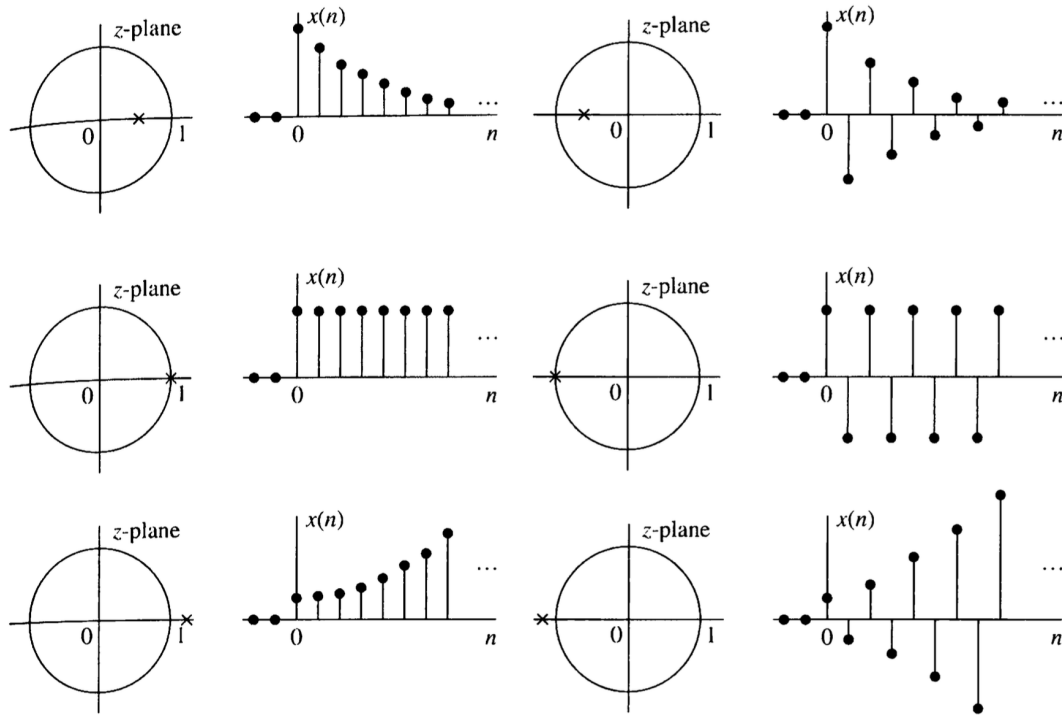


Abbildung 5: Impulsantwort $h[n] = a^n u[n]$ für verschiedene Werte von a , Quelle: [1]

System eine *endliche* Impulsantwort. Das können wir daran sehen, dass wir die Werte von $h[\cdot]$ direkt an den Koeffizienten von H ablesbar sind.

Zweitens, kann es vorkommen, dass $b_1 = \dots = b_N = 0$, was wiederum H zu einer rationalen Funktion macht, die keine Nullstellen und dementsprechend ausschließlich Polstellen besitzt. Um zu bestimmen, welche Implikation dies hat, betrachten wir ein einfaches System mit einem reellen Pol bei $a \in \mathbb{R}$, also

$$H(z) = \frac{1}{1 - az^{-1}}, \text{ROC} : |z| > |a|.$$

Wie wir weiter oben schon gesehen haben, ist die zugehörige Impulsantwort $h[\cdot]$ gegeben durch

$$h[n] = a^n u[n].$$

Das Verhalten von diesem System ist in Abb. 5 dargestellt. Man sieht, dass der Einheitskreis in der komplexen Ebene eine entscheidende Rolle spielt. Liegt a innerhalb des Einheitskreises, so ergibt sich eine exponentiell abfallende Impulsantwort. Falls $a < 0$, stellt sich auch eine Oszillation ein. Für $|a| = 1$ entspricht der Betrag der Impulsantwort der Heavyside-Funktion $u[\cdot]$. Man sieht, dass im Fall von $|a| < 1$ die Impulsantwort absolut summierbar ist, also

$$\sum_{n \in \mathbb{Z}} |h[n]| < \infty,$$

was impliziert, dass das System stabil ist. Entsprechend, ist das System instabil, falls $|a| > 1$. Man sieht also, dass die Position der Pole im z -Bereich direkt interpretierbar ist. Betrachtet man nochmals die ROC $|z| > |a|$,

zusammen mit der Information, dass sich Stabilität einstellt, falls $|a| < 1$. Geometrisch bedeutet dies, dass das System stabil ist, falls der Einheitskreis in der ROC enthalten ist.

Diese Resultate lassen sich auf mehrere Polstellen verallgemeinern, da H nur mehr Pole enthält und die ROC dann sich nach dem Betrag der „größten“ Polstelle richtet. Das heißt ultimativ, dass sich für ein System H Stabilität einstellt, falls *alle* Polstellen sich im Einheitskreis befinden, *oder* äquivalent, dass sich der Einheitskreis in der ROC befinden muss. Später werden wir noch eine Bedingung für Stabilität in Bezug auf die Fourier-Transformation finden.

4 Fourier-Transformation

Wir wollen unsere Werkzeuge zur Analyse von Signalen und Systemen nun um das wahrscheinlich wichtigste erweitern. Hierbei zerlegen wir Signale, beziehungsweise Systemantworten/Impulsantworten, in ihre „harmonischen“ Anteile – wir transformieren in den *Frequenzbereich*. Man diese Art der *Analyse* auch oft *Fourier-Analyse*. Da wir verschiedene Arten von Signalen bereits kennengelernt haben, muss diese Zerlegung auch auf verschiedene Weisen durchgeführt werden. Für diskrete Signale, ergibt es beispielsweise wenig Sinn, eine Integraltransformation zu definieren, Aperiodische Signale wiederum können nicht in einer Fourier-Reihe entwickelt werden, da die inhärent der Periodizität widerspricht. Das heißt, dass für jede „Art“ von Signal und dessen Eigenschaften, die passende Transformation existiert. Weiterhin ändert sich auch immer die *Interpretation* dieser Zerlegung in harmonische Komponenten.

Darüber hinaus werden wir auch den umgekehrten Weg gehen. Es ist auch möglich, Signale aus dem Frequenzbereich in den jeweils richtigen Signalbereich zu transformieren. In diesem Fall spricht von von *Fouriersynthese*, da wir ein Signal aus dessen Information über harmonische Anteile erzeugen/synthetisieren. Wir liefern nun also die Definitionen und Zusammenhänge der FT, die wir in Abschnitt 2 ohne Erläuterungen ausgenutzt haben.

4.1 Fourier-Transformation kontinuierlicher Signale

4.1.1 Fourier-Transformation kontinuierlicher periodischer Signale

Wir haben bereits in Abschnitt 2.2 mit (2.2.1) gesehen, dass man durch Linearkombination der komplexen Sinus-Funktionen

$$\{\exp(j2\pi k F_0 t), \text{ für } k \in \mathbb{Z}\}$$

eine $1/F_0 = T_0$ -periodische Funktion $x : \mathbb{R} \rightarrow \mathbb{C}$ durch

$$x(t) = \sum_{k \in \mathbb{Z}} c_k \exp(j2\pi k F_0 t)$$

erhält. Dies ist demnach ein Fall von *Fourier-Synthese*, da wir aus den Gewichten c_k in der Linearkombination eine Funktion x erhalten. Wir wollen nun den umgekehrten Weg gehen, auf welchem wir für eine gegebene Funktion x die Koeffizienten c_k bestimmen können. Wir starten dazu mit

$$x(t) = \sum_{k \in \mathbb{Z}} c_k \exp(j2\pi k F_0 t) \quad (4.1.1)$$

und multiplizieren beide Seiten mit $\exp(-j2\pi \ell F_0 t)$ und integrieren über eine Periode $[0, T_0]$. Dann erhalten wir

$$\int_0^{T_0} x(t) \exp(-j2\pi \ell F_0 t) dt = \int_0^{T_0} \left(\sum_{k \in \mathbb{Z}} c_k \exp(j2\pi k F_0 t) \right) \exp(-j2\pi \ell F_0 t) dt$$

und nach Vertauschung von Summation und Integration, dass

$$\sum_{k \in \mathbb{Z}} c_k \int_0^{T_0} \exp(j2\pi(k - \ell) F_0 t) dt = \sum_{k \in \mathbb{Z}} c_k \left[\frac{\exp(-j2\pi(k - \ell) F_0 t)}{j2\pi F_0(k - \ell)} \right]_0^{T_0}.$$

Da die Funktion $\exp(-j2\pi(k - \ell)F_0t)$ im Fall $k \neq \ell$ Periode T_0 besitzt, sind alle Summanden in der rechten Summation identisch 0, außer wenn $k = \ell$. Dann ergibt sich für $\exp(j2\pi(k - \ell)F_0t) = 1$, also

$$\int_0^{T_0} \exp(j2\pi(k - \ell)F_0t) dt = \int_0^{T_0} 1 dt = T_0.$$

Final erhalten wir für die Berechnung der Fourier-Koeffizienten $c : \mathbb{N} \rightarrow \mathbb{C}$ als Berechnungsvorschrift

$$c[\ell] = \frac{1}{T_0} \int_0^{T_0} x(t) \exp(-j2\pi\ell F_0t) dt.$$

Wir haben in diesem Fall also *Fourier-Analyse* betrieben. Außerdem haben wir absichtlich die Schreibweise von c_ℓ auf $c[\cdot]$ angepasst, um zu verdeutlichen, dass man nun die $c[\cdot]$ als *komplexes diskretes Signal* auffassen können. Dieses diskrete Signal können wir nun durch die Fourier-Reihe mit dem Signal $x : \mathbb{R} \rightarrow \mathbb{C}$ identifizieren. Sowohl x als auch $c[\cdot]$ sind also Darstellungen desselben Sachverhaltes – im „Zeitbereich“ und im zugehörigen „Frequenzbereich“. Wir sehen auch, dass sich für kontinuierliche und periodische Signale ein *diskreter* Frequenzbereich ergibt.

Obige Herleitung verschleiert aber einen viel tiefer liegenden Zusammenhang. Definieren wir wie in Abschnitt 2.2 die Funktionen $x_k : \mathbb{R} \rightarrow \mathbb{C}$ als

$$x_k(t) = \exp(j2\pi k F_0t),$$

dann können wir obige Rechnung auch wie folgt auffassen. Wir betrachten Multiplikation von x mit x_ℓ^* und anschließende Integration als Skalarprodukt $\langle x, x_\ell \rangle$. Außerdem haben wir aus der Fourier-Reihe gegeben, dass

$$x = \sum_{k \in \mathbb{Z}} c_k x_k$$

Wir haben also in dieser Denkweise lediglich auf beiden Seiten der Fourierreihe in (4.1.1) das Skalarprodukt mit x_ℓ gebildet. Also

$$\langle x, x_\ell \rangle = \left\langle \sum_{k \in \mathbb{Z}} c_k x_k, x_\ell \right\rangle.$$

Das Skalarprodukt ist linear, also können wir auch

$$\langle x, x_\ell \rangle = \sum_{k \in \mathbb{Z}} c_k \langle x_k, x_\ell \rangle$$

schreiben. Wir müssen also nur $\langle x_k, x_\ell \rangle$ bestimmen. Dies haben wir aber oben schon berechnet, denn das waren die Ausdrücke

$$\langle x_k, x_\ell \rangle = \int_0^{T_0} x_k(t) x_\ell(t)^* dt = \int_0^{T_0} \exp(j2\pi k F_0t) \exp(-j2\pi \ell F_0t) dt = \int_0^{T_0} \exp(j2\pi(k - \ell)F_0t) dt$$

Von oben wissen wir, dass

$$\langle x_k, x_\ell \rangle = \left[\frac{\exp(-j2\pi(k - \ell)F_0t)}{j2\pi F_0(k - \ell)} \right]_0^{T_0} = \begin{cases} T_0 & \text{für } k = \ell \\ 0, & \text{sonst.} \end{cases}$$

Die x_k stehen also paarweise *senkrecht/orthogonal* zu einander und es gilt $\langle x, x_\ell \rangle = T_0$. Sie bilden ein sogenanntes *Orthogonalsystem*. Hätten wir $\hat{x}_k = x_k / \sqrt{T_0}$ definiert, würde sogar gelten $\langle \hat{x}_k, \hat{x}_k \rangle = 1$ und die $\hat{x}_k$ bilden eine *Orthonormalsystem*.

Wenn wir nun noch einmal obige Gleichung für $\langle x, x_\ell \rangle$ betrachten, finden wir

$$\langle x, x_\ell \rangle = \sum_{k \in \mathbb{Z}} c_k \langle x_k, x_\ell \rangle = T_0 c_\ell,$$

weil alle Summanden durch die Orthogonalität verschwinden und nur im Falle von $k = \ell$ eben T_0 übrig bleibt. Der Fourier-Koeffizient c_ℓ ergibt sich also als Skalarprodukt von x mit dem zugehörigen x_ℓ . Es ist wichtig zu sehen, dass wir nur für die Berechnung von $\langle x_k, x_\ell \rangle$ die spezielle Form der x_k eingesetzt haben und sonst nur allgemein mit den Eigenschaften von Skalarprodukten gearbeitet haben. Das heißt, dass man ganz allgemein Signale „transformieren“ beziehungsweise *analysieren* kann, indem man sie als Summe von paarweise orthogonalen Signalen ausdrückt. Die Fourier-Reihe ist nur ein Spezialfall von diesem allgemeinen Konzept!

In Codeschnipsel 15 wird eine Rechteckfunktion $\text{rect}_{[0,T]}$ in ihre Fourier-Reihe entwickelt. In diesem Fall müssen wir natürlich die Reihenentwicklung abbrechen, da kein $K_{\max}$ existiert, sodass $c[k] = 0$ für $k > K_{\max}$. Wir könnten die Reihe zwar analytisch ausrechnen, da wir jedes x_k nur auf $[0, T]$ integrieren, also dem Bereich, auf dem die von uns definierte $\text{rect}_{[0,T]}$ -Funktion Werte ungleich 0 annimmt. Der Einfachheit halber nutzen wir `scipy.integrate.quad`, was in der Lage ist numerische Integration relativ präzise durchzuführen.

Es lohnt sich mit dem Wert von $K_{\max}$ zu experimentieren. Man sieht hierbei, dass größere Werte von $K_{\max}$ an den Unstetigkeitsstellen 0 und T und in deren Nähe nicht zu einer besseren Übereinstimmung der Fourierreihe mit x führen. Die Fourier-Reihe muss also nicht immer gegen x konvergieren. Im Falle von $\text{rect}_{[0,T]}$ ergibt sich das Problem genau aus dem Verhalten an 0 und T – also Unstetigkeit, was ein generelles Problem bei der Entwicklung von Signalen in Fourier-Reihen darstellt.

Im oberen Plot von Codeschnipsel 15 sieht man auch, dass der Realteil der Koeffizienten $\Re c[\cdot]$ ein gerades diskretes Signal ist, also $c[k] = c[-k]$. Der Imaginärteil $\Im c[\cdot]$ hingegen ist ein ungerades Signal, also $c[k] = -c[-k]$ ¹⁰. Dies liegt daran, dass das Signal lediglich reelle Werte annimmt, weshalb die Symmetrie-Eigenschaften der $c[\cdot]$ allgemein für reelle Signale gelten.

In Abbildung 6 sind noch mehr Eigenschaften der Fourier-Reihe deutlich gemacht. Generell stellen wir fest, dass die beiden Signale jeweils gut durch eine endliche Fourier-Reihe approximiert werden können, da sie keine Unstetigkeiten aufweisen. Im oberen Plot sieht man außerdem, dass Achsen-Symmetrie des Signals x dazu führt, dass die Imaginärteile von $c[\cdot]$ verschwinden. Wie man in Abbildung 6 unten sieht, ist es bei Anti-Symmetrie des Signals x der Realteil von $c[\cdot]$, der verschwindet. Dies liegt daran, dass der Realteil der x_k eine gerade Funktion ist und der Imaginärteil respektive eine ungerade Funktion. Da wir in Codeschnipsel 7 schon gesehen haben, dass ungerade und gerade Anteile eines Signals im Sinne von Skalarprodukten orthogonal sind, gilt dies auch für die resultierenden Fourier-Koeffizienten. In Abbildung 6 unten sieht man auch, dass man für gewisse Signale die Fourier-Reihe direkt angeben kann. Im Falle des Beispiels gilt nämlich

$$x(t) = \sin(10 \cdot 2\pi t) = \frac{1}{2j} (\exp(10 \cdot 2\pi t) - \exp(-10 \cdot 2\pi t)) = \frac{1}{2j} x_{10} - \frac{1}{2j} x_{-10}.$$

¹⁰siehe Codeschnipsel 7

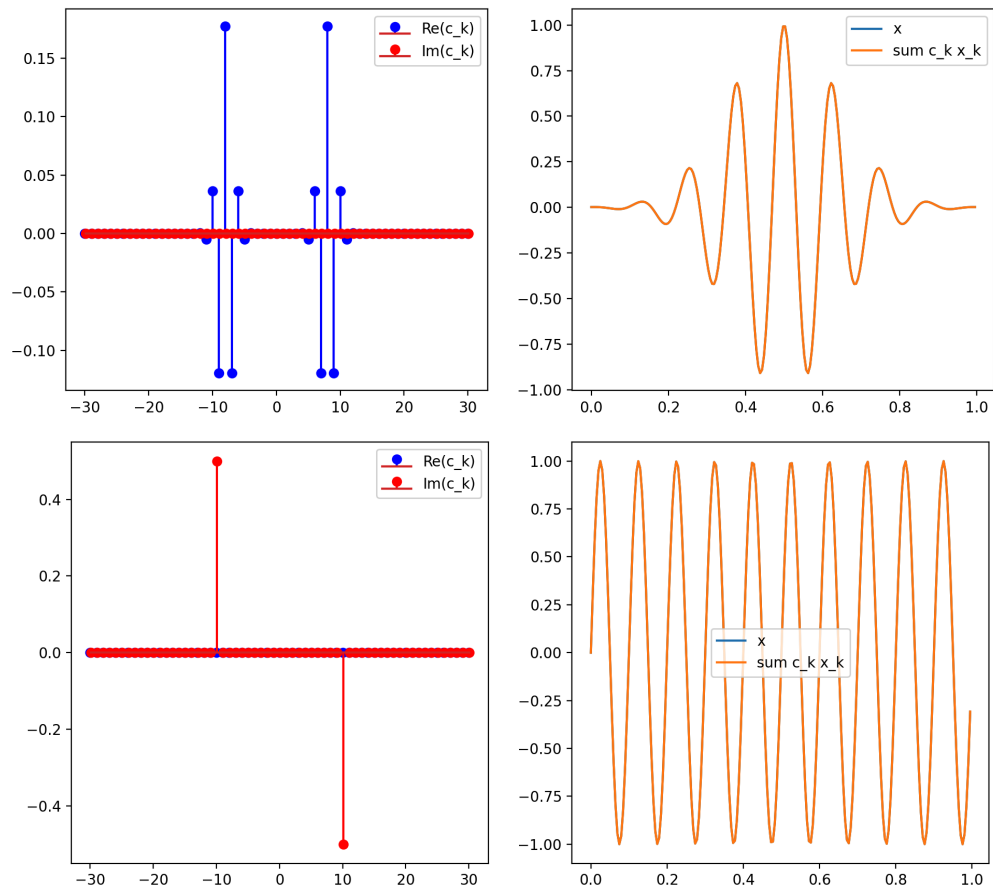


Abbildung 6: Mehr Versionen von Codeschnipsel 15; Oben: $x(t) = \exp(-25(t - 0.5)^2) \cos(16\pi t)$; Unten: $x(t) = \sin(20\pi t)$;

```

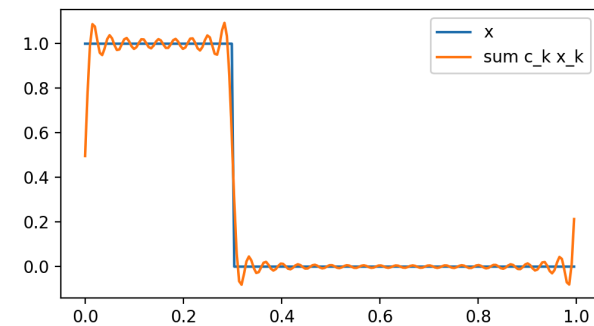
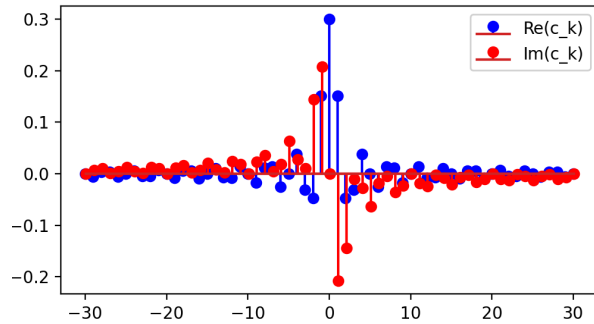
def rect(t: float) -> float:
    if (t < 0.0) or (t > 0.8):
        return 0
    else:
        return 1

def x_k(t: float, k: int) -> complex:
    return np.exp(1j * 2 * np.pi * F_0 * k * t)

def synth(
    t: float, c_k: list[complex], allK: list[int]
) -> complex:
    return np.sum(
        c_kk * x_k(t, kk)
        for kk, c_kk in zip(allK, c_k)
    )

allK = np.linspace(
    -Kmax, +Kmax, 2 * Kmax + 1, dtype=int
)
c_k = np.array(
    [
        quad(
            lambda t: x_k(t, kk).conj()
            * rect(t),
            0,
            T_0,
            complex_func=True,
        )[0]
        for kk in allK
    ]
)

```



Codeschnipsel 15: Berechnung und Darstellung von (4.1.1), siehe [dsv/code/fourier_series.py](#)

Das heißt, dass wir x *direkt* in seine Fourier-Reihe entwickelt haben, da wir es als Linearkombination der x_k dargestellt haben. Das heißt es sind nur x_{10} und x_{-10} notwendig, um x darzustellen und beide Koeffizienten haben ausschließlich imaginäre Anteile.

Ähnlich wie bei der z-Transformation kann man also an der Fourier-Reihe Eigenschaften des Signals x direkt ablesen, oder umgekehrt von Eigenschaften des Signals x auf Eigenschaften der Fourier-Koeffizienten $c[\cdot]$ schließen. Außerdem werden wir für die noch folgenden Versionen der FT sehr analoge Zusammenhänge finden.

4.1.2 Leichtungsdichte-Spektrum periodischer Signale

Das Leichtungsdichtespektrum eines T_0 -periodischen Signals $x : \mathbb{R} \rightarrow \mathbb{C}$ ist gegeben durch

$$P(x) = \frac{1}{T_0} \int_0^{T_0} |x(t)|^2 dt = \frac{1}{T_0} \int_0^{T_0} x(t)x(t)^* dt = \frac{1}{T_0} \langle x, x \rangle. \quad (4.1.2)$$

Wir wollen nun $P(\cdot)$ in Abhängigkeit der Fourier-Koeffizienten $c[\cdot]$ berechnen. Wir entwickeln also

$$x(t) = \sum_{k \in \mathbb{Z}} c_k \exp(j2\pi k F_0 t)$$

und setzen dies in P ein, um

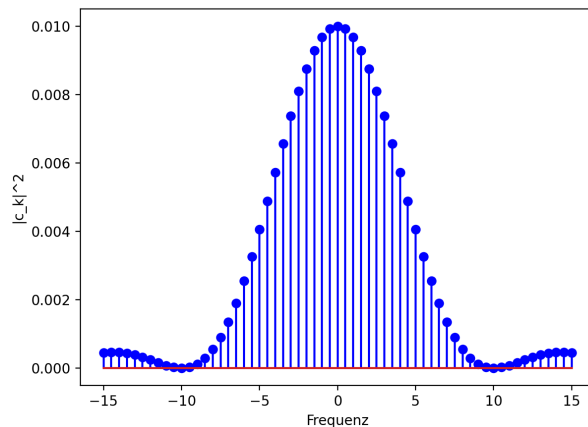
$$\frac{1}{T_0} \int_0^{T_0} x(t)x(t)^* dt = \frac{1}{T_0} \int_0^{T_0} x(t) \sum_{k \in \mathbb{Z}} c_k^* \exp(-j2\pi k F_0 t) dt = \sum_{k \in \mathbb{Z}} c_k^* \frac{1}{T_0} \int_0^{T_0} x(t) \exp(-j2\pi k F_0 t) dt = \sum_{k \in \mathbb{Z}} c_k^* c_k \quad (4.1.3)$$

als den *Satz von Parseval*¹¹ zu erhalten. Die physikalische Interpretation ist, dass $|c[k]|$ der Leistung des Signals bei der Frequenz kF_0 entspricht. Jeder Index k hat also eine physikalische Größe, die mit ihm assoziiert ist.

Da nur die Frequenzen kF_0 für $k \in \mathbb{Z}$ auftreten, also Frequenzen wie $0.1F_0$ nicht vorhanden sind, sprechen wir von einem *diskreten Spektrum*. Es ergibt sich also direkt folgender wichtiger Zusammenhang: Periodische Signale im Zeitbereich besitzen ein *diskretes* Spektrum. Andersherum ergibt sich auch: Signale mit diskretem Spektrum sind periodisch. Beide Argumente ergeben sich aus den Eigenschaften der Fourier-Reihe. Damit haben wir das duale Ergebnis zu (2.5.5) gefunden. Dort führte Diskretisierung im Zeitbereich, also Sampling/Abtastung, zu einer Periodifizierung im Frequenzbereich.

```
T_0 = 2
F_0 = 1.0 / T_0
Kmax = 30

def x(t: float) -> float:
    if (t < 0.0) or (t > 0.1):
        return 0
    else:
        return 1
```



Codeschnipsel 16: Modifiziertes Codeschnipsel 15 und Plot von P im Frequenzbereich., siehe [dsv/code/period_psd.py](#)

In Codeschnipsel 16 zeigen wir eine Modifikation von Codeschnipsel 15, in welcher wir $T_0 = 2$ setzen, also $F_0 = 1/2$ erhalten. Auf der Frequenzachse sehen wir, dass demnach diese für $K_{\max} = 30$ also von -15 bis $+15$ reicht.

¹¹https://de.wikipedia.org/wiki/Satz_von_Parseval

4.1.3 Fourier-Transformation von kontinuierlichen aperiodischen Signalen

Um die Einschränkung auf periodische Signale zu vermeiden, nutzen wir die **Fourier Transform**, wie wir sie bereits in (2.5.1) für ein Signal $x : \mathbb{R} \rightarrow \mathbb{C}$ durch

$$X(F) = \int_{-\infty}^{+\infty} x(t) \exp(-j2\pi Ft) dt \quad (4.1.4)$$

definiert haben. Im Unterschied zu (4.1.1) integrieren wir nun über ganz $\mathbb{R}$, da sich das Signal nicht mehr periodisch wiederholt. Aus diesem Grund ergibt sich auch ein *kontinuierliches* Spektrum, da wir uns nicht mehr auf eine abzählbare Menge von diskreten Frequenzen zurückziehen können. Deshalb ergibt sich auch für die Synthese des Signals, dass wir auch in diesem Fall integrieren anstatt summieren müssen, es gilt also

$$x(t) = \int_{-\infty}^{+\infty} X(F) \exp(j2\pi Ft) dF. \quad (4.1.5)$$

Da sich für die meisten Signale, die wir betrachten werden, Integration und Summation „ähnlich“ verhalten, finden wir auch die obigen Eigenschaften bezüglich Symmetrien, etc., von (4.1.1) wieder.

Es gibt aber eine Verbindung zur Fourier-Reihe, die wir im Folgenden kurz erläutern wollen. Nehmen wir an, es existiert ein $T > 0$, sodass $|x(t)| = 0$ für alle t mit $|t| > T$. Dann können wir das Signal x periodifizieren mit Periode T , indem wir

$$x_p(t) = \sum_{k \in \mathbb{Z}} x(t - 2kT)$$

setzen. Dies haben wir bereits in ähnlicher Form in (2.5.5) gesehen. Dort hat es sich aber aus Berechnungen ergeben und hier *setzen* wir diesen Zusammenhang explizit. Dann können wir x_p in seine Fourier-Reihe

$$x_p(t) = \sum_{k \in \mathbb{Z}} c_k \exp(j2\pi kt/T) \quad \text{mit} \quad T c_k = \int_{-T/2}^{+T/2} x_p(t) \exp(-j2\pi kt/T) dt = \int_{-\infty}^{+\infty} x(t) \exp(-j2\pi kt/T) dt$$

entwickeln. Aus der Definition in (4.1.4) und der letzten Gleichung sehen wir, dass sich die Koeffizienten der Fourier-Reihe finden lassen durch

$$c_k = \frac{1}{T} X\left(\frac{k}{T}\right), \quad (4.1.6)$$

diese sich also auch aus der Fourier-Transformation ablesen lassen. Das heißt, dass wir auch

$$x_p(t) = \sum_{k \in \mathbb{Z}} \frac{1}{T} X\left(\frac{k}{T}\right) \exp(j2\pi kt/T) = \sum_{k \in \mathbb{Z}} X(k\Delta F) \exp(j2\pi kt\Delta F) \Delta F$$

schreiben können. Dabei haben wir im letzten Schritt $1/T = \Delta F$ gesetzt. Dies können wir intuitiv (aber nicht rigoros!) so interpretieren, dass im Falle von $T \rightarrow \infty$ gilt, dass $\Delta F \rightarrow 0$. Je länger der Bereich der Funktion x , auf welchem gilt $x \neq 0$, desto kleiner ΔF . Der nicht-periodische Fall $T = \infty$ ergibt sich also als Grenzfall, bei welchem obige Summation zu einer Integration wird und ΔF zu einem dF . Man kann die **Fourier Transform** also als Grenzfall der Fourier-Reihe für $T \rightarrow \infty$ betrachten.

4.1.4 Leistungsdichte-Spektrum aperiodischer Signale

Analog zum Satz von Parseval für periodische Signale in (4.1.3) können wir auch hier wieder definieren und folgern, dass

$$E(x) = \int_{-\infty}^{+\infty} |x(t)|^2 dt = \int_{-\infty}^{+\infty} |X(F)|^2 dF \quad (4.1.7)$$

auch wieder ein Parseval Theorem für die **Fourier Transform** ergibt, was aber in diesem Fall als Plancherel Theorem¹² bekannt ist.

Andererseits kann man X auch in Betrag und Phase zerlegen, da es im Allgemeinen eine komplexe Größe ist, also

$$X(F) = |X(F)| \exp(j\angle(X(F))).$$

Man nennt dann $|X(F)|^2$ das *Leistungsdichte-Spektrum* von x .

In Codeschnipsel 17 zeigen wir das Vorgehen zur Fourier-Analyse mittels numerischer Integration. Es ist hier anzumerken, dass wir „nur“ die Funktion `rect` definieren müssen und der Rest, also die Funktionen `kernel`, `analyse` und `synthese` unabhängig hiervon sind. Wir haben hier ein Codeschnipsel, welches sich Methoden der Funktionalen Programmierung bedient. Die Funktionen `textttanalyse` und `synthese` haben als Eingabewert die entweder die Funktion, oder die **Fourier Transform** einer Funktion. Außerdem liefern sie als Ausgabewert wieder *Funktionen*, die wir einfach „aufrufen“ können.

¹²https://en.wikipedia.org/wiki/Plancherel_theorem

```

def rect(t: float) -> float:
    if np.abs(t) < 0.75:
        return 1
    else:
        return 0

def kernel(t: float, F: float) -> complex:
    return np.exp(1j * 2 * np.pi * F * t)

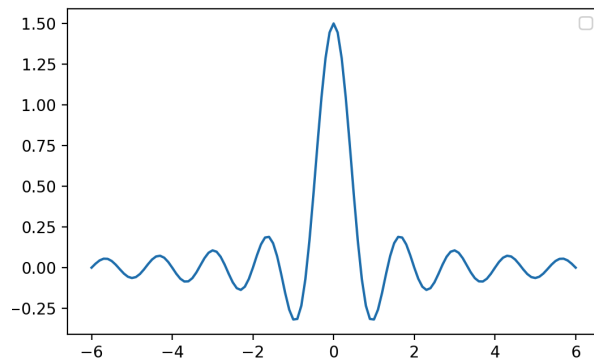
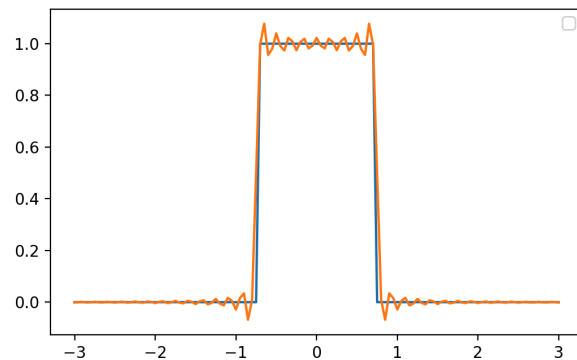
def analyse(x, Trange: list[float]):
    def x_ft(F: float):
        return quad(
            lambda t: kernel(-t, F) * x(t),
            np.min(Trange),
            np.max(Trange),
            complex_func=True,
        )[0].real

    return x_ft

def synthese(X, Frange: list[float]):
    def x(t: float):
        return quad(
            lambda F: kernel(t, F) * X(F),
            np.min(Frange),
            np.max(Frange),
            complex_func=True,
        )[0].real

    return x

```



```

Trange = np.linspace(-3, +3, 121, endpoint=True)
Frange = np.linspace(-6, +6, 121, endpoint=True)

rect_ft = analyse(rect, Trange)
rect_synth = synthese(rect_ft, Frange)

```

Codeschnipsel 17: Berechnung und Darstellung von (4.1.4), siehe [dsv/code/fourier_trafo.py](#)

4.2 Fourier-Transformation diskreter Signale

Wir nähern uns langsam der Fourier-Analyse von diskreten Signalen. Schließlich wollen wir etwaige spektrale Analysen und dergleichen im Digitalen durchführen, um die Vorzüge von digitalen Rechenwerken dabei nutzen zu können. Die vorher eingeführten Transformationen sind zwar hilfreich für theoretische Argumentation, wie beispielsweise beim Sampling-Theorem 2.1, aber nicht für praktische Anwendungen. Deshalb wenden wir uns nun der spektralen Analyse von diskreten Signalen zu. Wir werden aber im Verlauf auch wieder ähnliche Zusammenhänge wie in (4.1.6) finden.

4.2.1 Fourier-Transformation diskreter periodischer Signale

Wir beginnen mit diskreten Signalen, die gleichzeitig periodisch sind, also ein N existiert, sodass

$$x[n] = x[n + kN] \quad \text{für alle } n, k \in \mathbb{Z}$$

gilt. Aus vorherigen Diskussionen in Abschnitt 2 wissen wir einerseits, dass diskrete Signale ein periodisches Spektrum auf $(0, 1)$ besitzen. Andererseits wissen wir aus Abschnitt 4.1.1, dass periodische Signale ein *diskretes* Spektrum besitzen. Wir finden also nun intuitiv, dass das Spektrum von diskreten periodischen Signalen *ebenfalls* diskret und periodisch ist. Denken wir zurück an Abschnitt 2.3 so haben wir bereits alles Notwendige betrachtet. Ein N -periodisches diskretes Signal ergibt sich aus der Linearkombination der diskreten Signale $x_k[\cdot] : \mathbb{Z} \rightarrow \mathbb{C}$ definiert durch

$$x_k[n] = \exp\left(j2\pi \frac{k}{N}n\right) \quad \text{mit } k = 0, \dots, N-1.$$

Das heißt, für $x[\cdot]$ setzen wir mit

$$x[\cdot] = \sum_{k=0}^{N-1} c[k]x_k[\cdot]$$

an. Damit sind bereits beide Eigenschaften (Diskretheit und Periodizität) des Signals „eingepreist“. Denn man erkennt in den $x_k[\cdot]$ die vorher erwähnte *zweifache* Periodizität wieder, weil sowohl $x_{k+N}[\cdot] = x_k[\cdot]$ für alle k als auch $x_k[n + N] = x_k[n]$ für alle n gilt.

Es ist nun unser Ziel für gegebene Werte $x[n]$ von $x[\cdot]$ die Werte des *ebenfalls periodischen und diskreten* Signals $c[\cdot]$ zu bestimmen. Dies verläuft ganz analog zu Abschnitt 4.1.1. Wir definieren das Skalarprodukt $\langle \cdot, \cdot \rangle$ für N -periodische und diskrete Signale via

$$\langle x_1[\cdot], x_2[\cdot] \rangle = \sum_{n=0}^{N-1} x_1[n]x_2[n]^*.$$

Das heißt, dass wir das periodische Signal $x[\cdot]$ mit dem *endlich-dimensionalen* Vektor $\mathbf{x} \in \mathbb{C}^N$ identifizieren, wir setzen also die Einträge des Vektors als $\mathbf{x}_i = x[i-1]$. Dann können wir auch für das entsprechende Skalarprodukt der Vektoren $\mathbf{x}_{1,2} \in \mathbb{C}^N$ als

$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = (\mathbf{x}_2^*)^T \cdot \mathbf{x}_1 = \mathbf{x}_2^H \cdot \mathbf{x}_1$$

schreiben. Analog identifizieren wir die periodische und diskrete Sequenz $c[\cdot]$ mit dem Vektor $\mathbf{c} \in \mathbb{C}^N$.

Wie in Abschnitt 4.1.1 müssen wir nur

$$\langle \mathbf{x}_k, \mathbf{x}_\ell \rangle = \sum_{i=0}^{N-1} x_k[i]x_\ell[i]^* = \sum_{i=0}^{N-1} \exp\left(j2\pi \frac{i(k-\ell)}{N}\right) = \begin{cases} N & \text{falls } k = \ell \\ \frac{1 - \exp\left(j2\pi \frac{(k-\ell)}{N}\right)^N}{1 - \exp\left(j2\pi \frac{(k-\ell)}{N}\right)} & \text{sonst} \end{cases} = \begin{cases} N & \text{falls } k = \ell \\ 0 & \text{sonst} \end{cases}$$

berechnen. Hierbei nutzen wir, dass $\exp(j2\pi k) = 1$ für alle $k \in \mathbb{Z}$. Wie in Abschnitt 4.1.1 finden wir, dass also $\langle \mathbf{x}_k, \mathbf{x}_\ell \rangle = 0$ gilt, falls $k \neq \ell$ und $\langle \mathbf{x}_k, \mathbf{x}_k \rangle = N$. Damit ergibt sich bei Anwendung auf das eigentliche Signal $\mathbf{x}$ für den Vektor $\mathbf{c}$, dass

$$\langle \mathbf{x}, \mathbf{x}_\ell \rangle = \left\langle \sum_{k=1}^N \mathbf{c}_k \mathbf{x}_k, \mathbf{x}_\ell \right\rangle = \sum_{k=1}^N \mathbf{c}_k \langle \mathbf{x}_k, \mathbf{x}_\ell \rangle = N \mathbf{c}_\ell \Rightarrow \mathbf{c}_\ell = \frac{1}{N} \langle \mathbf{x}, \mathbf{x}_\ell \rangle.$$

Zusammenfassend, kann man also sagen, dass sich folgende Analyse- und Synthesegleichungen ergeben:

$$x[n] = \sum_{k=0}^{N-1} c[k] \exp(j2\pi kn/N), \quad \text{und} \quad c[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \exp(-j2\pi kn/N). \quad (4.2.1)$$

In Vektorschreibweise lässt sich der erste Teil durch

$$\mathbf{x} = \sum_{k=0}^{N-1} \langle \mathbf{x}, \mathbf{x}_k \rangle \mathbf{x}_k \quad (4.2.2)$$

ausdrücken.

Weiterhin, können wir eine Matrix $\mathbf{F}_N \in \mathbb{C}^{N \times N}$ definieren, deren k -te Spalte den Vektor $\mathbf{x}_k$ beinhaltet:

$$\mathbf{F}_N = [\mathbf{x}_1, \dots, \mathbf{x}_k, \dots, \mathbf{x}_N].$$

Dann können wir noch einen Schritt weitergehen und sehen, dass

$$\mathbf{c} = \frac{1}{N} \mathbf{F}_N^H \mathbf{x}, \quad \text{und} \quad \mathbf{x} = \mathbf{F}_N \mathbf{c}$$

gilt. Das heißt, dass sich Fourier-Analyse und Fourier-Synthese von diskreten und periodischen Signalen durch eine Matrix-Vektor-Multiplikation durchführen lässt! Das sind erst einmal gute Nachrichten, denn damit wissen wir, dass digitale Rechenwerke sehr gut darin sind, diese Transformation durchzuführen. Vor allem moderne **Central Processing Units (CPUs)** und **Graphical Processing Units (GPUs)** sind auf diese Art von Operation sehr stark optimiert indem spezielle Teile der Architektur nur für diese Operation bereitgestellt werden.

Die hier vorgestellte Transformation nennen wir **Discrete Fourier Transform (DFT)** und oft definiert man ein diskretes Signal $X[\cdot]$ durch

$$X[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \exp(-j2\pi kn/N) = c[k] = \mathbf{c}_k \quad (4.2.3)$$

und nennt dieses dann die **DFT** von $x[\cdot]$. Es ist anzumerken, dass der Vorfaktor N^{-1} in manchen Lehrbüchern oder Veröffentlichungen auch bei der Synthese anstatt der Analyse auftaucht, oder aber *beide* Gleichungen erhalten einen Faktor $N^{-1/2}$.

In Codeschnipsel 18 zeigen wir ein einfaches Beispiel für $x[\cdot] = u[\cdot] - u[\cdot - k]$. Außerdem zeigen wir auch die beiden Berechnungsmethoden der $c[\cdot]$ – einerseits mit der Definition und andererseits über ein Matrix-Vektor-Produkt.

4.2.2 Leichtungsdichte-Spektrum diskreter periodischer Signale

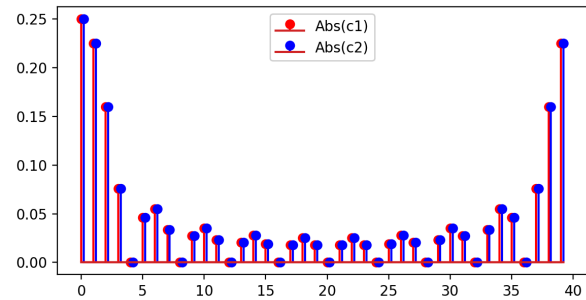
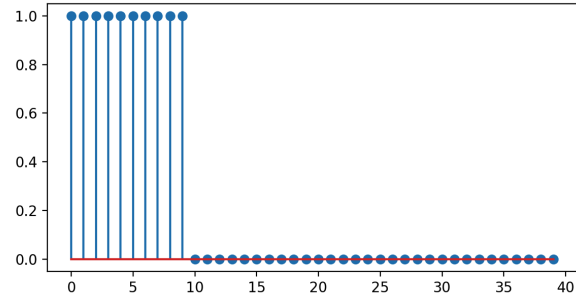
Da periodische Signale keine endliche Energie besitzen, betrachten wir die durchschnittliche Leistung über eine Periode. Wir betrachten also für ein N -periodisches Signal $x[\cdot]$ das Funktional

$$P(x[\cdot]) = \frac{1}{N} \sum_{i=0}^{N-1} |x[n]|^2. \quad (4.2.4)$$

```
def x_k(k: int, N: int) -> np.ndarray[complex]:
    return np.exp(
        1j * 2 * np.pi * k * np.arange(N) / N
    )
```

```
N = 40
x = np.zeros(N)
k = N // 4
x[:k] += np.ones(k)

c1 = np.array(
    [
        (x * x_k(kk, N).conj()).sum() / N
        for kk in range(N)
    ]
)
F = np.array([x_k(kk, N) for kk in range(N)]).T
c2 = F.conj().T.dot(x) / N
```



Codeschnipsel 18: Berechnung und Darstellung von (4.2.1), siehe [dsv/code/dft_1.py](#)

Dann können wir analog zu vorher herleiten, dass für

$$x[\cdot] = \sum_{i=0}^{N-1} c[k] x_k[\cdot]$$

und dessen Leistung gilt, dass

$$P(x[\cdot]) = \sum_{i=0}^{N-1} |c[k]|^2.$$

Also auch hier finden wir wieder, dass sich die durchschnittliche Leistung einer Periode im Zeitbereich auf die Leistung einer Periode im Frequenzbereich überträgt. Auch hier können wir die Folge $|c[\cdot]|^2$ als Leistungsdichte-Spektrum interpretieren.

4.3 Fourier-Transformation diskreter aperiodischer Signale

Wiederum analog zu Abschnitt 4.1 benötigen wir noch eine Transformation für diskrete Signale, die keine Periodizität aufweisen. Hierzu definieren wir die zugehörige FT, die wir DTFT nennen, durch

$$X(f) = \sum_{n \in \mathbb{Z}} x[n] \exp(-j2\pi f n), \quad (4.3.1)$$

genau wie in Abschnitt 2 bei der Herleitung von (2.1). Im Unterschied zur „analogen“ FT (4.1.4) sehen wir, dass X nur für Frequenzen $f \in (0, 1]$ definiert ist, da das diskrete Signal $x_f[n] = \exp(j2\pi f n)$ periodisch in f ist. Es gilt also $x_{f+k}[\cdot] = x_f[\cdot]$ für alle $k \in \mathbb{Z}$. Dies „passt“ auch zur Natur von diskreten Signalen, da deren Frequenzbereich immer periodisch sein muss.

Wir können nun für die inverse Transformation (4.3.1) mit (4.1.1) vergleichen. Bis auf das Vorzeichen in der Funktion $\exp(\cdot)$ gleicht (4.3.1) einer Fourier-Reihe der periodischen Funktion X . In der Tat, können wir

die Folge $x[\cdot]$, also das ursprüngliche Signal, als Fourier-Koeffizienten der **DTFT** X durch

$$x[n] = \int_{-1/2}^{+1/2} X(f) \exp(j2\pi f n) df \quad (4.3.2)$$

wiederfinden. Wir sind hier also den umgekehrten Weg gegangen und haben X als eine periodische Funktion im Frequenzbereich in ihre Fourier-Reihe entwickelt, welche dann wiederum Informationen über den entsprechend diskreten Zeitbereich liefert.

Diese Synthese-Operation nennen wir dann **Inverse Discrete Time Fourier Transform (iDTFT)**. Es ist wiederum anzumerken, dass die **DTFT** „nur“ ein theoretisches Werkzeug ist, da sich im Allgemeinen die unendliche Summe in (4.3.1) praktisch nicht realisieren lässt, genauso wenig wie deren Resultat, eine kontinuierliche Funktion.

In Codeschnipsel 19 zeigen wir dennoch, wie man die **DTFT** einer aperiodischen Folge approximieren kann. Wir studieren hierzu das Signal

$$x[n] = \begin{cases} \frac{\omega}{\pi} & \text{für } n = 0, \\ \frac{\omega}{\pi} \frac{\sin(\omega n)}{\omega n} & \text{sonst} \end{cases}.$$

Dessen analytisch bestimmte **DTFT** X ist

$$X(f) = \text{rect}(f / (2\pi\omega)),$$

welche wir durch eine endliche Summe über die von uns verfügbaren Werte von $x[\cdot]$ bestimmen.

Nun treten hierbei mehrere Effekte zutage.

- Die Approximation der **DTFT** mittels des „abgeschnittenen“ Signals x stimmt noch nicht sehr gut mit der analytischen Lösung überein. Verändert man den Wert der Variable N im Skript, tritt dieser Effekt stärker oder schwächer zutage.
- Auch bei Erhöhung von N stellt sich immernoch keine Konvergenz von X_{approx} gegen X_{true} ein. Dies liegt daran, dass die **DTFT** von $x[\cdot]$ nicht konvergiert, wie wir in Codeschnipsel 15 bereits gesehen hatten – zumindest nicht in dem üblichen Sinn. Diesen Effekt bezeichnet man allgemein als Gibbsches Phänomen¹³.
- Augenscheinlich ist es besser die **iDTFT** aus X_{approx} zu berechnen, als aus X_{true} . Doch dies liegt lediglich daran, dass die numerische Integration der rect-Funktion nicht genau genug ist. Erhöht man die Anzahl der Punkte im Array F , dann stimmen im dritten Plot die drei Graphen besser überein.

¹³https://en.wikipedia.org/wiki/Gibbs_phenomenon

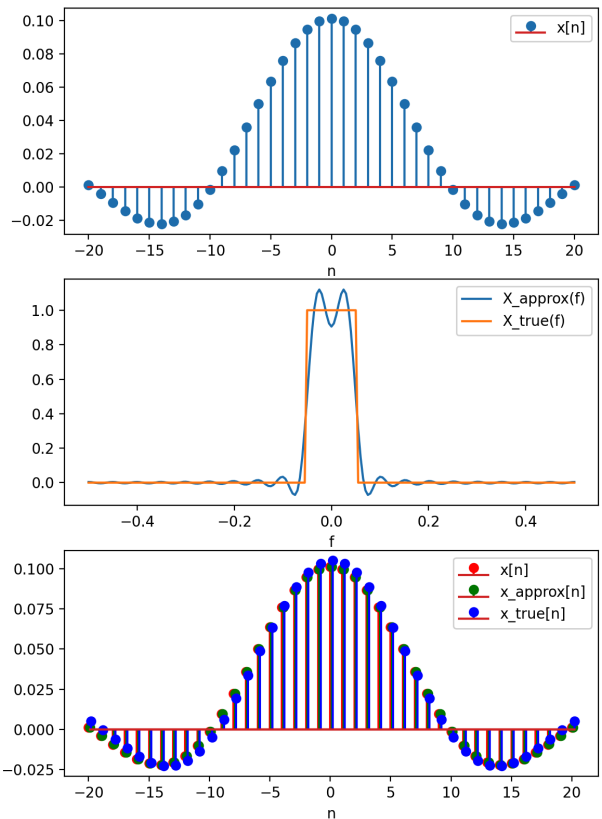
```
def dtft(
    x: np.ndarray[complex],
    n: np.ndarray[int],
    F: np.ndarray[float],
) -> np.ndarray[complex]:
    return np.array(
        [
            (
                x * np.exp(-2j * np.pi * n * f)
            ).sum()
            for f in F
        ]
    )
```

```
def idtft(
    X: np.ndarray[complex],
    F: np.ndarray[float],
    n: np.ndarray[int],
):
    return np.array(
        [
            (F[1] - F[0])
            * (
                X * np.exp(2j * np.pi * nn * F)
            ).sum()
            for nn in n
        ]
    )
```

```
N = 20
n = np.linspace(-N, +N, 2 * N + 1)
omega = 1 / np.pi

x = np.sin(omega * n) / (np.pi * n)
x[N] = omega / np.pi
F = np.linspace(-0.5, +0.5, 201)
X_approx = dtft(x, n, F)
X_true = np.zeros_like(F)
X_true[np.abs(F) <= omega / (2 * np.pi)] += 1
```

```
x_approx = idtft(X_approx, F, n)
x_true = idtft(X_true, F, n)
```



Codeschnipsel 19: Berechnung und Darstellung von (4.3.1), siehe [dsv/code/dtft.py](#)

4.3.1 Leichtungsdichte-Spektrum diskreter aperiodischer Signale

Die Energie eines diskreten Signals haben wir in (3.1.1) durch

$$\mathcal{E}(x[\cdot]) = \sum_{n \in \mathbb{Z}} |x[n]|^2$$

definiert. Wiederum können wir uns überlegen, dass dann für die DTFT X gilt, dass

$$\mathcal{E}(x[\cdot]) = \int_{-1/2}^{+1/2} |X(f)|^2 df.$$

Schlussendlich bezeichnet man dann

$$S_{xx}(f) = |X(f)|^2$$

als das Leichtungsdichte-Spektrum des Signals $x[\cdot]$. In manchen Anwendungen ist es auch wieder sinnvoll Betrag und Phase von $X(f)$ zu analysieren.

4.4 Eigenschaften der Fourier-Transformationen

Nachdem wir nun die notwendigen Definitionen gesammelt haben, wollen wir uns einige wichtige Eigenschaften der definierten Transformationen ansehen und eventuell auch für „praktische“ Dinge verwenden.

4.4.1 Beziehung der Fourier-Transformationen und der z-Transformation

Wie wir in Abschnitt 3.4 gesehen haben, ist die z-Transformation für ein diskretes Signal $x[\cdot]$ definiert durch

$$X_z(z) = \sum_{n \in \mathbb{Z}} x[n]z^{-n} \quad \text{mit ROC} \quad r_2 < |z| < r_1.$$

Wenn wir nun z in Polarform $z = r \exp(j2\pi f)$ ausdrücken und annehmen, dass $r_2 < r < r_1$, dann sehen wir, dass die z-Transformation von $x[\cdot]$ nichts anderes ist als die DTFT von $x[n]r^{-n}$ ist. Mit anderen Worten ist die z-Transformation an einer Stelle $z = r \exp(j2\pi f)$ die DTFT X eines Signals gewichtet mit der Folge $w[n] = r^{-n}$ an der Stelle f . Ist nun $1 \in \text{ROC}$ (und damit der gesamte Einheitskreis der komplexen Ebene), dann gilt

$$X_z(\exp(j2\pi f)) = X(f) \tag{4.4.1}$$

Wir haben uns zwar bis jetzt wenig Gedanken über die Existenz der Fourier-Transformation gemacht, doch hier finden wir einen Hinweis. Die DTFT existiert, beziehungsweise *konvergiert*, falls $1 \in \text{ROC}$. Da die „Form“ der ROC immer kreisförmig ist, ist die gleichbedeutend mit der Tatsache, dass sich der Einheitskreis der komplexen Ebene in der ROC befindet. Außerdem kann man auch eine Fourier-Analyse von LTI-Systemen betreiben. Dann ist die BIBO-Stabilität von einem solchen System äquivalent zur Tatsache, dass der Einheitskreis in der ROC liegt. Wie wir gerade gesehen haben, folgt dann auch, dass BIBO-Stabilität ebenfalls durch die Existenz der DTFT charakterisiert wird.

In Codeschnipsel 20 zeigen wir diesen Zusammenhang für das gleiche Signal, wie in Codeschnipsel 18, interpretieren es hier aber einmal als aperiodisches Signal bei der Berechnung der DTFT und als periodisches Signal für die Berechnung der DFT. Wir wissen aus den Eigenschaften der z-Transformation, dass das

Signal $x[n] = u[n] - u[n - k]$ im z -Bereich einen k -fachen Pol bei $z = 0$ hat und $k - 1$ Nullstellen auf dem Einheitskreis.

Wir können in dem Plot von Codeschnipsel 20 direkt sehen, dass die DTFT auf dem Einheitskreis in der Tat mit der z -Transformation übereinstimmt. Außerdem sehen wir gut den Pol bei $z = 0$ und die 9 Nullstellen auf dem Einheitskreis. In den Plots der DTFT und der DFT sehen wir auch, dass sich die DFT durch Auswertung der DTFT ergibt. Wir hatten vorher in Gleichung (4.1.6) schon gesehen, dass sich die Fourier-Koeffizienten $c[k]$ eines periodischen kontinuierlichen Signals aus der Auswertung der Fourier-Transformation an den richtigen Stellen ergeben. Denselben Zusammenhang finden wir hier wieder, denn zwischen der DFT und der DTFT besteht derselbe Zusammenhang.

```

N = 40
x = np.zeros(N)
k = N // 4
x[:k] += np.ones(k)

res = 128
real = np.linspace(-2, +2, res, endpoint=True)
imag = np.linspace(-2, +2, res, endpoint=True)
zValues = np.concatenate(
    [
        gg.flatten()[None, :]
        for gg in np.meshgrid(
            real,
            imag,
        )
    ]
).T

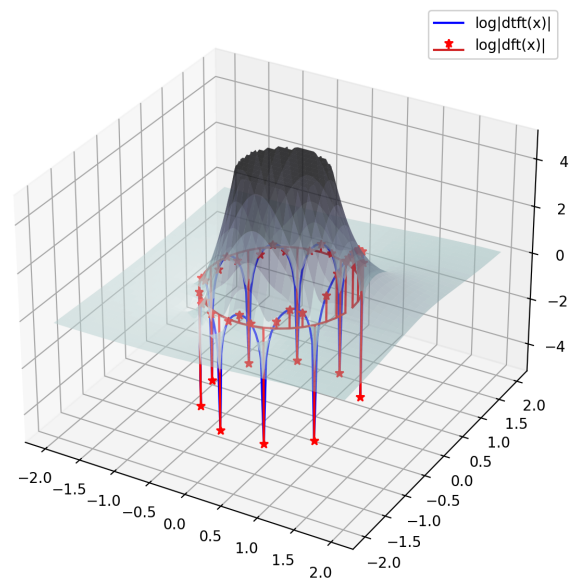
zTrafo = np.array(
    [
        (
            x
            * (zz[0] + 1j * zz[1])
            ** (-np.arange(N))
        ).sum()
        for zz in zValues
    ]
).reshape(res, res)

dtftValues = np.exp(
    1j
    * 2
    * np.pi
    * np.linspace(0, 1, 201, endpoint=True)
)

dtft = np.array(
    [
        (x * zz ** (-np.arange(N))).sum()
        for zz in dtftValues
    ]
)

dft = np.fft.fft(x)

```



Codeschnipsel 20: Berechnung und Darstellung von (4.4.1), siehe [dsv/code/dtft_z.py](#)

4.4.2 Weitere Eigenschaften der DFT

Wir wollen analysieren, welche Eigenschaften die Beziehung zwischen einem diskreten N -periodischen Signal $x[\cdot]$ und dessen **DFT** $X[\cdot]$ besitzt.

Für die meisten Eigenschaften macht es Sinn die **DFT** eines Signals der Länge N und die entsprechende **Inverse Discrete Fourier Transform (iDFT)** zu definieren durch

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{k \cdot n} \quad \text{und} \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-k \cdot n}, \quad (4.4.2)$$

wobei wir hier

$$W_N = \exp(-j2\pi/N) \quad (4.4.3)$$

definiert haben.

DFT ist periodisch: Wir wissen bereits aus der Definition und den Eigenschaften von komplexen diskreten Harmonischen, dass

$$X[k + \ell \cdot N] = X[k]$$

für alle $\ell \in \mathbb{Z}$ gilt.

Übereinstimmung mit DTFT: Aus Codeschnipsel 20 wissen wir, dass

$$X(k/N) = X[k]$$

gilt – wir die Werte der **DFT** also aus Auswertung der **DTFT** von $x[\cdot]$ erhalten. Es ist wichtig zu erwähnen, dass die Frequenzen an denen die **DTFT** X ausgewertet wird harmonische „Verwandte“ sind, da wir X an den Stellen $f = k/N$ auswerten.

Abgetastete Signale Ist $x[\cdot]$ aus Abtastung eines analogen Signals $x : \mathbb{R} \rightarrow \mathbb{C}$ entstanden, so erhält die Einheit von f bei der **DTFT** eine Bedeutung, da dann „Perioden pro Sample“ eine physikalische Interpretation zulässt. Tasten wir x mit Sampling-Rate F_s ab, so ist der Abstand zwischen zwei Samples in $x[\cdot]$ genau $1/F_s$. Demzufolge erhält die **DTFT** $X(f)$ die Interpretation, dass wir das periodifizierte Spektrum von x an den Stellen $f \cdot F_s$ betrachten. Das heißt beispielsweise bei einer Sample-Rate von 44 kHz entspricht der Bereich $f \in [-1/2, +1/2]$ dem *physikalischen* Frequenzbereich von $F \in [0 \text{ Hz}, 44 \text{ kHz}]$. Es ist jedoch zu beachten, dass wir bei der Betrachtung der **DTFT** *immer* nur das periodifizierte Spektrum erhalten. Das heißt also, nur wenn wir Aliasing-frei abgetastet haben, also F_s groß genug gewählt haben, können wir aufschlussreiche Aussagen über das Spektrum von x basierend auf der **DTFT** von $x[\cdot]$ treffen. Wie bereits erwähnt, ergibt dann die **DFT** des Signals $x[\cdot]$ eben die periodifizierten spektralen Informationen von x an den diskreten Frequenzen kF_s/N .

Reelle Signale Intuitiv sind reelle Signale „einfacher“ als komplexe Signale. Deshalb mag es nicht verwunderlich sein, dass die **DFT** von reellen Signalen Struktur hat, in dem Sinne, dass wir einige Koeffizienten aus anderen schnell berechnen können. Ist ein Signal reell, so gilt $x[n] = x[n]^*$ für alle n . Für W_N gilt

$$W_N^\ell = \left(W_N^{-\ell}\right)^* \quad \text{und} \quad W_N^{N-\ell} = \left(W_N^\ell\right)^*$$

also gilt

$$X[N-k] = \sum_{n=0}^{N-1} x[n] W_N^{(N-k)n} = \sum_{n=0}^{N-1} x[n]^* \left(W_N^{kn}\right)^* = \sum_{n=0}^{N-1} x[n]^* W_N^{-kn} = \left(\sum_{n=0}^{N-1} x[n] W_N^{kn}\right)^* = X[k]^* = X[-k].$$

Das heißt, dass die **DFT** von reellen Signalen „konjugiert symmetrisch“ um $k = 0$ sind. Dies hat zur Folge, dass man nur die Koeffizienten von $k = 0$ bis $k = \lceil N/2 \rceil$ berechnen und speichern muss, was den Rechenaufwand halbiert. Außerdem wird hierdurch auch die **IDFT** weniger komplex. In Python ist es in solchen Fällen deshalb ratsam auf `np.fft.rfft` und `np.fft.irfft` zurückzugreifen. Es sei noch angemerkt, dass sich noch viele weitere Redundanzen und Symmetrien ausnutzen lassen. Siehe hierzu [1, Kap. 7.2.1].

DC-Komponente und Nyquist-Sequenz Der Wert $X[0]$ wird oft als DC-Komponente, also Gleichanteil, bezeichnet, weil $W_N^0 = 1$, also ergibt sich speziell für $X[0]$, dass

$$X[0] = \sum_{n=0}^{N-1} x[n],$$

was eben dem um Faktor N skalierten Mittelwert des Signals $x[\cdot]$ entspricht. Eine andere Interpretation ergibt sich aus der Betrachtung der **DFT** als Skalarprodukt, wobei wir die „Ähnlichkeit“ von $x[\cdot]$ mit der konstanten Sequenz $x_0[\cdot] = 1$ berechnet haben. Ist ein Signal reell, so ergibt sich aus der Symmetrie des Spektrums, dass der DC-Anteil immer ebenfalls reell sein muss. Glück gehabt!

Der andere Extremfall tritt auf, wenn $k = N/2$. Hierbei wird vorausgesetzt, dass N eine gerade Zahl ist. Dann nennt man $x_{N/2}$ die Nyquist-Sequenz, denn wie wir in **Abgetastete Signale** gesehen haben, entspricht $k = N/2$ der physikalischen Frequenz $F = (N/2)F_s/N = F_s/2$, also genau der *maximalen* Frequenz, die ein Signal beinhalten darf, sodass es noch mit der gewählten Sampling-Rate ohne Aliasing beobachtbar ist.

Man sieht in Codeschnipsel 21 gut, dass die beiden Sequenzen wirklich „Gegensätze“ darstellen, da $x_0[\cdot]$ die Anteile mit „minimaler“ Variation/Frequenz aufammelt, während die Werte von $x_{N/2}[\cdot]$ genau so liegen, dass ein analoges Signal (und dessen Aliase), siehe Codeschnipsel 4, genau Frequenz $F_s/2$ (und Amplitude 1) besitzen muss, um $x_{N/2}[\cdot]$ als Abtastwerte zu erhalten.

Zyklische Faltung im Zeitbereich Kommen wir nun zur wahrscheinlich wichtigsten Eigenschaft der **DFT** und dem Grund, warum digitale Signalverarbeitung überhaupt ermöglicht wurde. Wir haben bereits gesehen, dass **LTI**-Systeme durch eine Faltung realisiert werden können. Gegeben zwei N -periodische Signale $x_{1,2}[\cdot]$ und betrachten wir deren *zyklische* Faltung $\circledast$ definiert durch

$$x_3[m] = \sum_{n=0}^{N-1} x_1[n] \cdot x_2[(m-n) \bmod N] = (x_1 \circledast x_2)[m], \quad (4.4.4)$$

dann kann man zeigen, dass

$$X_3[k] = X_1[k] \cdot X_2[k]$$

gilt. Das heißt, dass die zyklische Faltung von zwei periodischen Signalen wieder ein periodisches Signal ergibt, dessen **DFT** das Produkt der **DFTs** der gefalteten Signale ist. Wenn wir an **LTI**-Systeme denken und uns erinnern, dass viele Filter als **LTI**-Systeme aufgefasst werden können, so ist klar, warum diese Eigenschaft wichtig ist. Die digitalen Filter können also im Frequenzbereich sehr einfach angewendet werden, weil dort nur eine einfache punktweise Multiplikation notwendig ist. Doch dies allein ist nicht ausreichend, denn

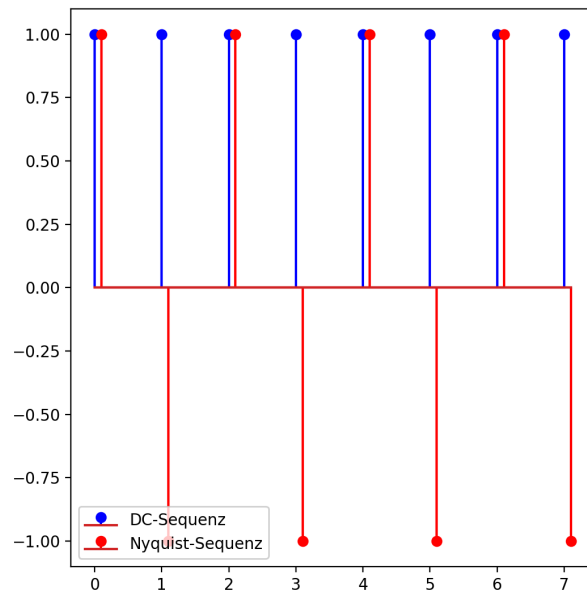
```

N = 8

X = np.fft.fft(np.eye(N))
x_DC = X[:, 0]
x_Nyquist = X[:, N // 2]

plt.stem(
    np.arange(N),
    x_DC,
    linefmt="b",
    label="DC-Sequenz",
)
plt.stem(
    np.arange(N) + 0.1,
    x_Nyquist,
    linefmt="r",
    label="Nyquist-Sequenz",
)
plt.legend()
plt.show()

```



Codeschnipsel 21: Darstellung der DC-Sequenz und der Nyquist-Sequenz für $N = 8$, siehe [dsv/code/nyquist_seq.py](#)

wir benötigen noch einen effizienten Weg, um zwischen Zeit- und Frequenzbereich wechseln zu können. Dies wird uns durch die **Fast Fourier Transform (FFT)** ermöglicht werden. In Codeschnipsel 18 hatten wir bereits erfahren, dass man die **DFT** als Matrix-Vektor-Produkt formulieren kann, wozu etwa **Floating Point Operations (FLOPs)** in der Größenordnung N^2 notwendig sind. Die **FFT** erlaubt es aber eine **DFT** in der Größenordnung von nur $N \log N$ **FLOPs** durchzuführen.

Dann beläuft sich eine effiziente Implementierung eines **FIR** Filters mit Impulsantwort $h[\cdot]$ auf folgende Schritte:

1. Transformiere $x[\cdot]$ und $h[\cdot]$ in den Frequenzbereich
2. Punktweise Multiplikation von $Y[\cdot] = X[\cdot] \cdot H[\cdot]$ (NICHT die z -Transformation H_z)
3. Rücktransformation von $Y[\cdot]$ zu $y[\cdot]$.

Dieser Prozess wurde in Codeschnipsel 22 nachgestellt und wird mit der zyklischen Faltung im Zeitbereich verglichen. Als Beispiel nehmen wir hier eine abgetastete Sinusfunktion, deren Werte mit Rauschen belegt sind. Um das Signal ein wenig vom Rauschen zu befreien, implementieren wir ein gleitendes Mittel, siehe Codeschnipsel 10, dessen Mittelungsdauer wir mit einem „cut-off“ einstellen können. Wie man sieht, ist das Ergebnis der Faltung im Zeitbereich mit dem im Frequenzbereich identisch. Außerdem stellt sich durch die Mittelung eine Glättung der Werte ein, die mehr dem Verlauf des ursprünglichen Signals ähnelt. Jedoch sieht man auch, dass sich die Phase des Resultates von der des Eingangs unterscheidet. Diese Phase wird vom Filter h aufgeprägt und ist somit eine Konsequenz der gewählten Signalverarbeitung. Es gibt Möglichkeiten diesen Einfluss zu reduzieren, die aber über unser aktuelles Interesse hinausgehen.

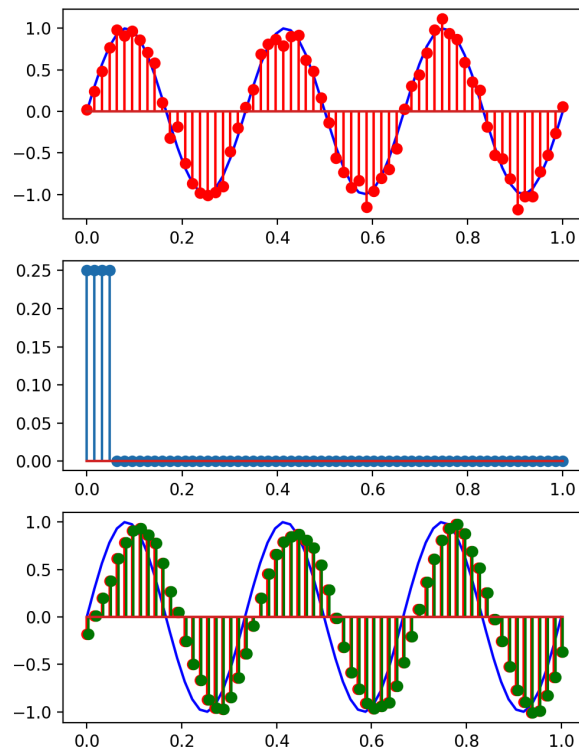
Es sei hier angemerkt, dass dies der einfachste Fall ist, den man sich für die sogenannte schnelle Faltung vorstellen kann. Es ist beispielsweise nicht direkt klar, was zu tun ist, wenn $x[\cdot]$ und $h[\cdot]$ nicht dieselbe Länge

haben, was dazu führt, dass die resultierenden DFTs ebenfalls von verschiedener Länge sind. Darüber hinaus ist unsere Implementierung eine „offline“-Version in dem Sinne, dass die zu verarbeitenden Daten bereits vollständig im Speicher liegen müssen. Doch wenn man an Echtzeitanwendungen, beispielsweise im Audibereich, denkt so wird es notwendig sein, diese Operation auf Daten anzuwenden, die nicht vollständig im Zugriff liegen, sondern sich fortlaufend aktualisieren. Außerdem will man natürlich nicht immer eine zyklische Faltung, sondern eine „normale“ Faltung implementieren, besonders im Echtzeitfall ist dies besonders wichtig.

All diese Probleme können aber mit einigen Tricks so umformuliert werden, dass man an einer Stelle in der Verarbeitung wieder eine zyklische Faltung rechnen kann. Oft müssen hierzu das Signal oder die Impulsantwort zielführend manipuliert werden, oder das Ergebnis nach Faltung noch prozessiert werden.

```
N = 64
F = 3
T = np.linspace(0, 1, N)
cutOff = 0.05
sin = np.sin(2 * np.pi * 3 * T)
x = sin + 0.1 * np.random.randn(N)
h = np.zeros_like(T) + 1 * (T < cutOff)
h /= np.sum(h)

# zirkulare faltung im zeitbereich
# Gleichung (4.4.4)
y1 = np.convolve(x, np.tile(h, 2))[N : 2 * N]
y2 = np.fft.ifft(
    np.fft.fft(x) * np.fft.fft(h)
).real
```



Codeschnipsel 22: Zyklische/periodische Faltung im Zeitbereich und im Frequenzbereich., siehe [dsv/code/dft_conv.py](#)

Zyklische Faltung im Frequenzbereich Als Dual zu **Zyklische Faltung im Zeitbereich** gilt auch, dass die zyklische Faltung im Frequenzbereich eine Multiplikation der jeweiligen Signale im Zeitbereich zur Folge hat. Es gilt also, für ein Signal $x_3[\cdot]$ mit

$$x_3[n] = x_1[n] \cdot x_2[n],$$

dass

$$X_3[k] = \frac{1}{N} (X_1 \otimes X_2)[k]. \quad (4.4.5)$$

Wir haben nun genügend Eigenschaften der DFT gesammelt, sodass wir uns nun Anwendungen zuwenden können.

5 Anwendung der Disreten Fourier-Transformation

5.1 Spektralanalyse mit der Short Time Fourier Transformation

Einleitung Als erstes wollen wir uns mit der Fourier-Analyse von Signalen beschäftigen. Hierbei ist das Ziel das Verhalten eines Signals im Frequenzbereich zu charakterisieren. Wir wollen jedoch in unserer Analyse davon ausgehen, dass wir ein Signal $x[\cdot]$ vorliegen haben, dessen spektrale Eigenschaften *zeitvariant* sind. Hiermit ist *nicht* gemeint, dass wir davon ausgehen, dass das Signal zeitvariant ist, denn das ist es natürlich. Es geht darum, dass der Anteil der verschiedenen Frequenzen in einem Signal sich mit der Zeit ändert. Natürlich ist hier eine der am einfachsten zugänglichen Anwendungen die Analyse von Audiosignalen.

Beginnen wir mit einer intuitiven Betrachtung. Zunächst wollen wir einen möglichst großen Bereich im Frequenzbereich abdecken – zumindest den für die jeweilige Anwendung relevanten. Da sich beispielsweise im Audibereich oft an der menschlichen Hörcharakteristik orientiert wird, die jenseits von 20 kHz nichts mehr leistet, sind Frequenzen jenseits dieser Grenze nicht mehr relevant. Das heißt, dass sinnvollerweise ein Audiosignal mit einer Sample-Rate von 40 kHz bereits absolut ausreichend aufgenommen wurde. Für andere Anwendungen lassen sich oft ähnliche Argumente finden, da die meisten physikalischen Systeme eine Tief- oder Bandpasscharakteristik „versteckt“ haben, und man somit nur sehr selten mit Signalen mit immensen Bandbreiten konfrontiert ist. Zusammenfassend muss also die Sample-Rate so angepasst werden, dass sie entsprechend unserer Anforderungen ausreicht.

Darüber hinaus ist uns aber auch gut daran getan, die Frequenzen, die in dem Signal vorkommen, sehr gut unterscheiden zu können. Wir wollen also möglichst *genau* wissen, welche einzelnen Frequenzen im Signal vorhanden sind. Das heißt, dass wir für eine tatsächlich im Signal vorhandene Frequenz F aus unserer Analyse eine Frequenz erhalten wollen, die im Intervall $[F - \Delta_1, F + \Delta_1]$ für möglichst kleines $\Delta_1 > 0$ liegt. In diesem Sinne wollen wir also möglichst große *Auflösung*. Wenn wir daran denken, dass $\Delta_1 = F_s / N$ gilt, so scheint es zu helfen, dass wir ein Signal möglichst lange „beobachten“ müssen, also N sehr groß wählen müssen.

Doch es gibt noch einen weiteren Auflösungsbegriff, der vom ersten strikt zu unterscheiden ist. Wir wollen außerdem eine maximale Trennschärfe zwischen zwei unterschiedlichen Frequenzen, die gleichzeitig im Signal vorhanden sind, erreichen. Wir wollen also in unserer Analyse zwei Frequenzen F_1 und F_2 mit $|F_1 - F_2| = \Delta_2$ immernoch unterscheiden und beider Existenz erkennen können. Wiederum ist hier das minimale $\Delta_2 > 0$ von Interesse, für das diese Unterscheidbarkeit noch gilt, da dieses ein weiteres Auflösungslimit unseres Systems ist. Ähnlich wie bei Δ_1 ist auch hier der Beobachtungszeitraum N , welchen wir als Grundlage für unsere Analyse verwenden, von Bedeutung. Längere Zeiträume erlauben eine bessere Auflösung *zwischen* Frequenzen.

Außerdem ist es noch oft interessant, dass man einen hohen Dynamikbereich abdecken kann. Dies meint, dass man Frequenzanteile mit Frequenzen F_1 und F_2 und Amplituden A_1 und A_2 noch als zwei wahrnimmt, beziehungsweise beide noch erkennt, solange $\log(A_1 / A_2) > \Delta_3$. Je größer das maximale Δ_3 ist, desto besser, da prominente Frequenzanteile nicht weitere weniger stark ausgeprägte Anteile maskieren.

Schlussendlich haben wir es auch mit zeitvarianten Frequenzanteilen zu tun. Das heißt, dass sich zum Zeitpunkt T_1 die Zusammensetzung der Frequenzen eines Signals x von denen zum Zeitpunkt T_2 unterscheidet. Für $|T_1 - T_2| > \Delta_4$ wollen wir also erkennen, dass $x[T_1]$ andere spektrale Charakteristik hat als $x[T_2]$ und die für möglichst kleine Δ_4 . Diese zeitliche Evolution des Spektrums ist natürlich von immenser Bedeutung, weil man möglichst genau wissen möchte, *wann* sich Frequenzen im Signal verändern. Ideal

wäre es also, wenn wir eine Art „instantane“ Frequenzanalyse durchführen könnten, die trennscharf in einem infinitesimalen Intervall die jeweiligen Frequenzanteile aus dem Signal extrahiert. Doch Schwingungen sind inhärent *ausgedehnte* Effekte. Ein kleines Δ_4 steht also in direktem Widerspruch zu den Anforderungen an unsere Analyse für, gute (lies: kleine) Δ_1 und Δ_2 , da wir dort gesehen haben, dass wir das Signal *lange* beobachten müssen. Für diesen Beobachtungszeitraum nehmen wir aber das Signal als „konstant“ im Frequenzbereich – also genau kontraproduktiv für ein kleines Δ_4 . Dieser Widerspruch ergibt also eine Art Unschärferelation zwischen Frequenz- / und Zeitauflösung.

Die zugehörige Transformation nennen wir **Short Time Fourier Transformation (STFT)**. Wir wollen uns hier lediglich mit der diskreten Version der **STFT** befassen, weil wir diese uns als Analysetool von abgetasteten Daten vorstellen, die mit Rate F_s aufgenommen wurden. Uns liegt als das Signal $x : \mathbb{R} \rightarrow \mathbb{C}$ vor, das wir zum diskreten Signal $x[\cdot]$ abgetastet haben.

Definition Dann definieren wir eine Fensterbreite $W \in \mathbb{N}$ und eine Verzögerung $S \in \mathbb{N}$ und bilden die W -dimensionalen Vektoren $\mathbf{x}_s$ als

$$\mathbf{x}_n^\ell = [x[\ell \cdot S + n]]_{n=0}^{W-1} \in \mathbb{C}^W$$

und mit diesen eine zweidimensionale Datenstruktur (keine Matrix!)

$$\mathbf{X} = [\mathbf{x}^1, \dots, \mathbf{x}^L] \in \mathbb{C}^{W \times L},$$

welche aus L vielen „Fenstern“ in $x[\cdot]$ der Länge W besteht. Dann berechnen wir spaltenweise die **DFT** auf $\mathbf{X}$ und erhalten $\mathbf{X}_{\text{stft}}$.

Interpretation Im Licht der Überlegungen aus **Einleitung** können wir nun diese Definition interpretieren und noch etwas aufbohren, um bessere Eigenschaften beziehungsweise mehr Information aus der **STFT** zu erhalten. Dabei machen wir uns zunutze, dass eine diskrete **STFT** „nur“ eine blockweise angewandte **DFT** ist. Das heißt, dass deren Eigenschaften sich ausnutzen lassen, um die **STFT** zu beeinflussen.

Zunächst stellen wir fest, dass wir in einem Block von $\mathbf{X}_{\text{stft}}$ eine Frequenzauflösung von $\Delta_1 = F_s / NW$ erhalten, da die Periodendauer eines Blockes im Zeitbereich die Länge WN / F_s hat. Jetzt könnte man auf die Idee kommen, dass man diese Auflösung verbessern will. Eine Möglichkeit besteht darin die Fensterbreite künstlich zu verlängern. Eine beliebte Herangehensweise ist an $\mathbf{X}$ eine bestimmte Anzahl von Nullen anzuhängen.

Dieses sogenannte *zero-padding* wird in Codeschnipsel 23 gezeigt. Man sieht, dass man bei Hinzufügen von $2N$ Nullen an ein Signal der Länge N eine *Interpolation* der **DFT** erreicht. Diese Technik kann dann „blockweise“ bei der **STFT** angewandt werden, um einen besseren Überblick des Spektrums zu bekommen. Beispielsweise erlaubt dies auch eine genauere Bestimmung des Maximums der **DFT**, was der im Signal „prominentesten“ Frequenz entspricht. Da wir aber nur Nullen im Zeitbereich anhängen, entsteht keine neue Information, sondern wir erhalten nur zusätzliche Zwischenwerte. Für die **STFT** heißt das, dass wir lediglich ein schöner aufgelöstes Bild erhalten, wenn wir *zero-padding* anwenden.

```

N = 25
F_s = 25

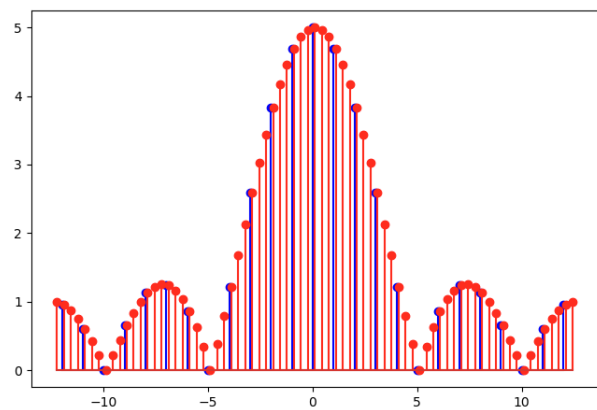
x = np.zeros(N)
x[: N // 5] = 1

freq_x = np.fft.fftshift(
    np.fft.fftfreq(x.size) * F_s
)
X = np.fft.fftshift(np.fft.fft(x))

x_zp = np.pad(x, (0, 2 * N))

freq_x_zp = np.fft.fftshift(
    np.fft.fftfreq(x_zp.size) * F_s
)
X_zp = np.fft.fftshift(np.fft.fft(x_zp))

```



Codeschnipsel 23: Zero-padding im Zeitbereich einer rect-Funktion liefert eine überabtastung der sinc-Funktion im Frequenzbereich, siehe [dsv/code/stft_zp.py](#)

Wir sehen auch noch einen weiteren Effekt von zero-padding, wenn wir ein harmonisches Signal mit der **DFT** analysieren. Die **FT** einer einzelnen Sinusschwingung mit Frequenz F_0 entspricht genau zwei komplexen Dirac-Stößen bei $\pm F_0$. Doch was wir im Plot von Codeschnipsel 24 sehen ist, dass wir dies im Diskreten nicht reproduzieren können. Bei Anwendung der Technik aus Codeschnipsel 23 stellt sich heraus, dass wir eher eine Faltung der Dirac-Stöße bei $\pm F_0$ mit einer (periodischen) sinc-Funktion sehen. Deren Auftreten lässt sich dadurch erklären, dass zero-padding eines Signals der Länge N auf Länge M im Zeitbereich der Multiplikation eines Signals mit einem diskreten rect-Signal entspricht, welches bei N vielen Werten den Wert 1 annimmt, siehe Codeschnipsel 18. Da dessen **DFT**, wie in Codeschnipsel 23 gezeigt, einem sinc entspricht, sehen wir im Frequenzbereich die genannte Faltung. Wenn wir im Zeitbereich die Länge des Signals auf $2N$ verlängern, so multiplizieren wir mit einer „breiteren“ rect-Funktion im Zeitbereich und erhalten so einen „proportional schmaleren“ sinc im Frequenzbereich.

Für die **STFT** heißt das, genau das, was wir oben schon intuitiv analysiert haben. Beobachten wir das Signal in einem längeren Block, so können wir benachbarte Frequenzen besser auseinanderhalten. Dies gilt jedoch nur, wenn sich diese Zusammensetzung im Frequenzbereich nicht stark ändert, während man diese Analyse durchführt, also innerhalb eines Blocks.

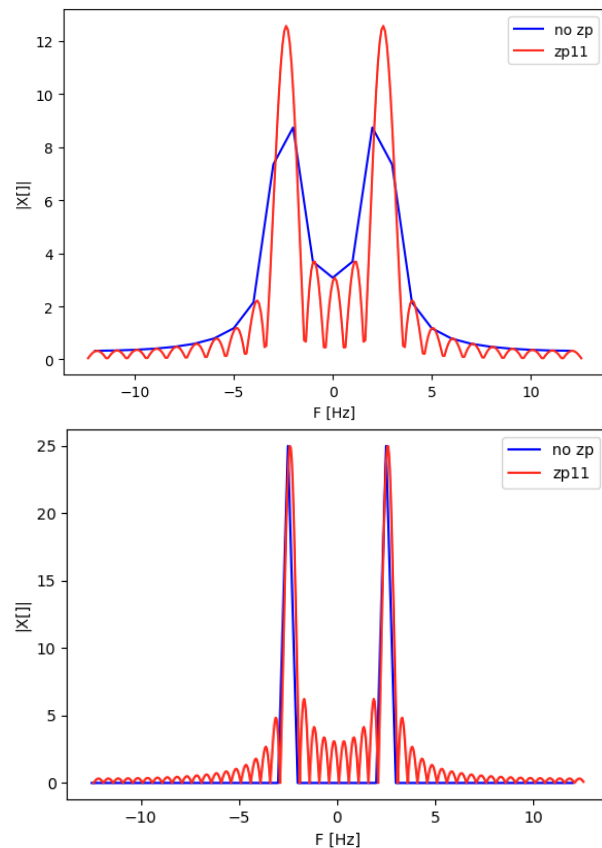
```
N = 25
F_s = 25

x = np.sin(2 * np.pi * 0.1 * np.arange(N))

freq_x = np.fft.fftshift(
    np.fft.fftfreq(x.size) * F_s
)
X = np.fft.fftshift(np.fft.fft(x))

x_zp = np.pad(x, (0, 10 * N))

freq_x_zp = np.fft.fftshift(
    np.fft.fftfreq(x_zp.size) * F_s
)
X_zp = np.fft.fftshift(np.fft.fft(x_zp))
```



Codeschnipsel 24: Einfluss der Fensterbreite bei der Analyse von harmonischen Schwingungen, siehe [dsv/code/stft_harm.py](#)

Man kann sich diesen Effekt aber auch zunutze machen und somit das Ergebnis der **DFT** beeinflussen, indem man das Signal vor der **DFT** noch zusätzlich fenstert, wobei man die Fensterfunktion genau so konzipiert, wie man es gerne hätte¹⁴.

Wenn wir uns beispielsweise an die Diskussion bezüglich der möglichen Dynamik in einem Signal erinnern, so kann es sein, dass man eher an Dynamik als an Auflösung interessiert ist. In solch einem Fall, kann man beispielsweise auf ein Hann-Fenster¹⁵ zurückgreifen und anstatt einem Vektor x transformieren wir mit der **DFT** den Vektor $w \odot x$, wobei $\odot$ die punktweise Multiplikation repräsentiert. Dies führt dazu, dass wir im Frequenzbereich eine Faltung mit der **DFT** von w erhalten anstatt einem sinc.

In Codeschnipsel 25 sehen wir nun, dass dies erlaubt Frequenzanteile zu finden, die sich ohne explizite Fensterung in den sogenannten Nebenwellen der sinc-Funktion „verstecken“. Die Nebenwellen der sinc-Funktion fallen ab gemäß

$$|\text{sinc}(x)| \leq \frac{1}{x},$$

wobei andere Fensterfunktionen im Frequenzbereich deutlich schneller abfallen können. Wir sehen aber auch, dass sich hier wiederum ein Trade-Off aufzeigt, da die sogenannte Hauptkeule der **DFT** von w deutlich breiter ist als die der sinc-Funktion. Das heißt, dass man durch Auswahl der Fensterfunktion zwischen Auflösung im Sinne von $\Delta_{1,2}$ und Dynamikbereich wählen kann, beziehungsweise *muss*. Denn, wie wir gesehen haben, impliziert eine **DFT** *immer* eine Multiplikation mit der entsprechenden rect-Funktion und man kann diese nur gegen eine andere ersetzen, aber nicht vermeiden! Für die **STFT** bedeutet dies, dass wir hier mit derselben Fragestellung konfrontiert sind und entsprechend den Anforderungen der speziellen Anwendung wählen müssen.

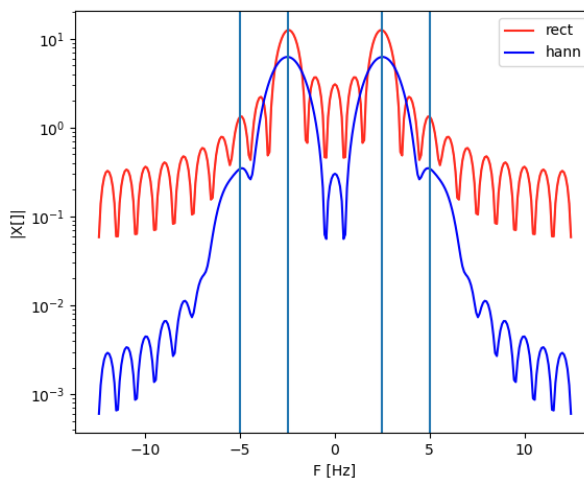
```
N = 25
F_s = 25

x = np.sin(
    2 * np.pi * 0.1 * np.arange(N)
) + 0.05 * np.sin(2 * np.pi * 0.2 * np.arange(N))

x_1 = np.pad(x, (0, 10 * N))

freq_x = np.fft.fftshift(
    np.fft.fftfreq(x_1.size) * F_s
)
X_1 = np.fft.fftshift(np.fft.fft(x_1))

x_2 = np.pad(
    x * hann(x.size, sym=False), (0, 10 * N)
)
X_2 = np.fft.fftshift(np.fft.fft(x_2))
```



Codeschnipsel 25: Geschickte Fensterung kann den Dynamikbereich erhöhen, siehe [dsv/code/stft_win.py](#)

¹⁴<https://docs.scipy.org/doc/scipy/reference/signal.windows.html>

¹⁵<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.windows.hann.html>

Als nächstes wollen wir mit Codeschnipsel 26 den Punkt machen, dass bei der **STFT** direkt Auflösung im Frequenzbereich gegen Auflösung im Zeitbereich getauscht wird. Wollen wir Änderungen im Frequenzbereich präzise in der Zeit verorten, brauchen wir entsprechend kurze Fenster, doch opfern dafür Genauigkeit bei der Bestimmung des eigentlichen Frequenzwertes. Andererseits können wir ein Signal länger, mit einem breiten Fenster, beobachten und dann haben sehr gute Auflösung im Frequenzbereich, was aber impliziert, dass wir nicht genau sagen können, *wann* sich Dinge im Frequenzbereich ändern.

```
def f(t):
    if t < 5:
        return np.cos(2 * np.pi * t * 10)
    elif t < 10:
        return np.cos(2 * np.pi * t * 25)
    elif t < 15:
        return np.cos(2 * np.pi * t * 50)
    else:
        return np.cos(2 * np.pi * t * 100)

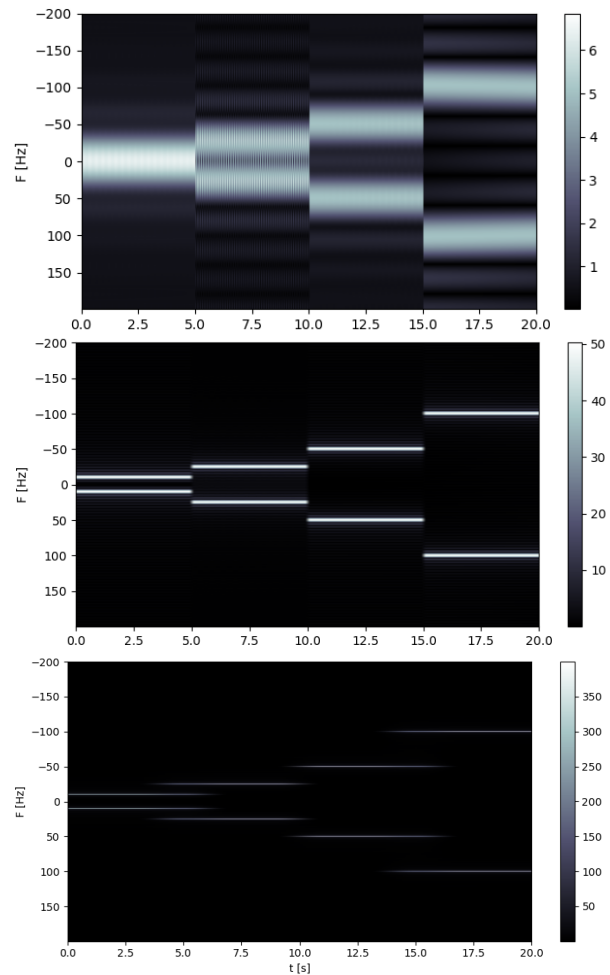
F_s = 400.0 # Hz
T_max = 20.0 # s
T = np.linspace(0, T_max, int(T_max * F_s)) # s

W = 2.0 # s
W_N = int(F_s * W) # samples

x = np.array([f(tt) for tt in T])
X = x.reshape((-1, W_N)).T

F = np.fft.fftshift(np.fft.fftfreq(1024)) * F_s

STFT = np.fft.fftshift(
    np.fft.fft(X, n=1024, axis=0), axes=0
)
```



Codeschnipsel 26: STFT für verschiedene Fensterlängen, siehe [dsv/code/stft_length.py](#)

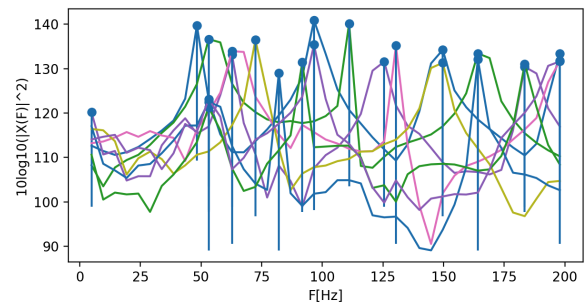
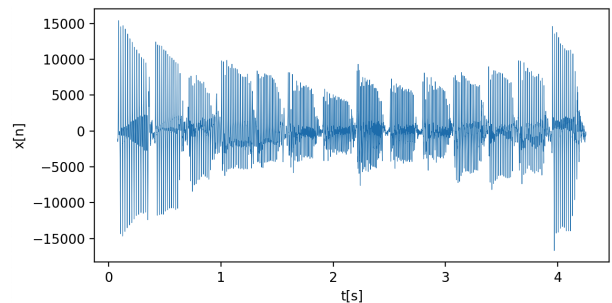
Als letztes wollen wir ein Beispiel für eine Anwendung der **STFT** zeigen. In Codeschnipsel 27 wurde mit einer Bassgitarre eine G-Dur-Tonleiter eingespielt und in entsprechende Blöcke zerteilt, die in etwa den einzelnen Tönen entsprechen. Im Frequenzbereich wurde dann eine „peak search“ genutzt, um die lokalen Maxima der einzelnen **DFT**-Blöcke zu finden. Die ermittelten Frequenzen der einzelnen Töne passen sehr gut zu denen, die man online in Tabellen¹⁶ finden kann.

```
sliceRun = slice(
    int(0.08 * F_s), int(4.25 * F_s)
)
T = np.linspace(
    0, x.size / F_s, x.size, endpoint=False
)

y = x[sliceRun][
    : int(np.floor(x[sliceRun].size / 14) * 14)
].reshape((14, -1))[:, 1000:-1000]
N = y.shape[1]

dfts = np.fft.fft(y, axis=1)
freqs = np.fft.fftfreq(N) * F_s
plotInd = np.logical_and(
    (np.abs(freqs)) < 200, freqs > 0
)

plt.figure()
plt.subplot(211)
plt.plot(T[sliceRun], x[sliceRun], linewidth=0.4)
plt.xlabel("t[s]")
plt.ylabel("x[n]")
plt.subplot(212)
for dd in range(8):
    spectrum_dB = 10 * np.log10(
        np.abs(dfts[dd, :]) ** 2
    )
    plt.plot(
        freqs[plotInd],
        spectrum_dB[plotInd],
    )
    peakSearch = scipy.signal.find_peaks(
        spectrum_dB, height=120
    )
```



```
[ 48.2811  96.5622 149.6715 197.9527]
[ 53.109   111.0466 164.1559]
[  4.8281  62.7654 125.5309 183.4683]
[ 62.7654 130.3590 197.9527]
[ 72.4217 149.6715]
[ 53.109   82.0779 164.1559]
[ 53.109   91.7341 183.4683]
[ 53.109   96.5622 197.95270418]
```

Codeschnipsel 27: Analyse einer „Bassline“ mittels STFT, die eine G-Dur-Tonleiter spielt, siehe [dsv/code/stft_bass.py](#)

5.2 Interpolation mittels DFT

In Abschnitt 2.5 haben wir eine Theorie entwickelt, welche es uns erlaubt aus einer diskreten Menge von Abtastpunkten, das ursprüngliche Signal zu rekonstruieren. Allgemeiner kann man dies auch als ein

¹⁶<https://mixbutton.com/mixing-articles/music-note-to-frequency-chart/>

Interpolationsproblem betrachten, wobei es darum geht, eine Funktion zu finden, deren Werte an bestimmten Punkten vorbestimmt sind. Das heißt, wir suchen nicht unbedingt eine bestimmte Funktion, sondern eine beliebige, die „nur“ die sogenannte Interpolationseigenschaft erfüllt. Eine Möglichkeit, solche eine Funktion zu finden, bietet die **DFT**.

5.2.1 Theoretische Grundlagen

Gegeben sei ein periodisches, analoges Signal $x : \mathbb{R} \rightarrow \mathbb{C}$, mit Periode $T_p = 1/F_0$. Wir beobachten dieses Signal auf einem uniformen Raster von Punkten, via $x[n] = x(nT)$ und wollen eine Funktion y herleiten, für welche die Interpolationsbedingung

$$y(nT) = x[n] = x(nT) \quad (5.2.1)$$

erfüllt ist. Hierzu entwickeln wir das Signal x in seine Fourier-Reihe via

$$x(t) = \sum_{k=-\infty}^{+\infty} c[k] \exp(j2\pi kt F_0), \quad (5.2.2)$$

womit nun „nur“ noch die Fourierkoeffizienten $c[\cdot]$ zu finden sind. Nun tasten wir dieses Signal uniform mit Samplerate $F_s = N/T_p = 1/T$ (also passend zur Periodendauer) ab und erhalten die Folge

$$x[n] = x(nT) = \sum_{k=-\infty}^{+\infty} c[k] \exp(j2\pi knTF_0) = \sum_{k=-\infty}^{+\infty} c[k] \exp\left(j2\pi k \frac{n}{N}\right) \quad \text{für } n \in \mathbb{N} \quad (5.2.3)$$

bestehend aus den Samples von x an deren Stellen die Interpolationsbedingung erfüllt sein soll. Mit der Periodizität von $\exp(j2\pi t)$, siehe Abschnitt 4.2.1 und Abschnitt 2.3, und der Abtastung erhalten wir außerdem noch

$$x[n] = \sum_{k=0}^{N-1} \left[\sum_{\ell=-\infty}^{+\infty} c[k - \ell N] \right] \exp\left(j2\pi k \frac{n}{N}\right) = \sum_{k=0}^{N-1} \tilde{c}[k] \exp\left(j2\pi k \frac{n}{N}\right), \quad (5.2.4)$$

wobei wir

$$\tilde{c}[k] = \sum_{\ell=-\infty}^{+\infty} c[k - \ell N] \quad (5.2.5)$$

als Abkürzung benutzt haben. Wir nehmen nun aus guten Grund noch an, dass die Funktion x auch bandbegrenzt ist. Das heißt, ihre Fourier-Transformierte $X : \mathbb{R} \rightarrow \mathbb{C}$ verschwindet außerhalb eines gewissen Bandes, also

$$X(F) = \int_{-\infty}^{+\infty} x(t) \exp(-j2\pi Ft) dt = 0 \quad \text{für } |F| > B. \quad (5.2.6)$$

Außerdem wissen wir, dass die Fourier-Transformation X und die Folge $c[k]$ verknüpft sind via

$$c[k] = \frac{1}{T_p} X(kF_0), \quad (5.2.7)$$

was impliziert, dass die Folge $c[k]$ ab einem gewissen $k > K_0$ verschwindet. Es gilt also

$$c[k] = 0 \quad \text{für } |k| > \left\lceil \frac{B}{F_0} \right\rceil. \quad (5.2.8)$$

Das heißt, dass wir nun $F_s = N/T_p$ so groß wählen müssen, dass sich in (5.2.5) kein Aliasing für $\tilde{c}[k]$ ergeben kann. Es muss also gelten

$$N > \lceil B/F_0 \rceil \quad \text{bzw.} \quad F_s > \lceil B/(F_0 T_p) \rceil. \quad (5.2.9)$$

Es folgt dann, dass $c[k] = X[k]$, wobei $X[k]$ die **DFT** der Folge $x[n]$ darstellt. Das heißt, dass wir die Fourier-Koeffizienten der kontinuierlichen Funktion x durch die **DFT** der Abtastwerte $x[n]$ bestimmen können. Mit (5.2.2) können wir also die Folge $x[n]$ interpolieren, indem wir

$$y(t) = \frac{1}{T_p} \sum_{k=-\frac{B}{F_0}}^{+\frac{B}{F_0}} X[k] \exp(j2\pi kt F_0) \quad (5.2.10)$$

schreiben. Dieses y erfüllt die Interpolationsbedingung (5.2.1), weil wegen der Bandbegrenzung von x und der Periodizität sogar $y(t) = x(t)$ für alle t gilt.

Man beachte hier, dass nun aus der Folge von diskreten Werten $x[n]$ eine analytische Formel in Form einer *endlichen* Summation entstanden ist. Unter der Annahme der Bandlimitierung von x ist diese Interpolation *exakt* und kann effizient implementiert werden, durch die Vorberechnung der Folge $X[k]$ durch die **FFT** [3] der Folge $x[n]$. Eine Anwendung der hier vorgestellten Methode zur Interpolation wird in Abschnitt 5.2.2 aufgezeigt.

5.2.2 Interpolation von Antennen-Richtcharakteristiken

Motivation Mit jeder Erschließung von neuen Frequenzbereichen für die Kommunikation ist es von Interesse das Ausbreitungsverhalten der elektro-magnetischen Wellen für verschiedene Umgebungen zu charakterisieren, beispielsweise innerstädtisch, auf der Autobahn, etc. Zwar können solche Umgebungen auch computerbasiert simuliert werden, doch für eine empirisch abgeleitete Statistik solcher sogenannter Kanalmodelle [4] sind repräsentative Messungen unerlässlich. Diese Charakteristiken werden genutzt, um in realistischen Szenarien Kanalkapazitäten, Datenraten und dergleichen zu bestimmen. Schlussendlich fließen solche Statistiken dann in neue Mobilfunkstandards ein, indem jene Statistiken zur Quantifizierung der Performance von beispielsweise Endgeräten herangezogen werden.

Das FG EMS hat sich deshalb unter anderem auf solche Messungen und deren Auswertung, das sog. Channel Sounding [6], spezialisiert. Hierbei kommen meist breitbandige **Multiple Input Multiple Output (MIMO)** Messsysteme zum Einsatz, die den Funkkanal in Frequenz, Raum und Zeit kohärent vermessen

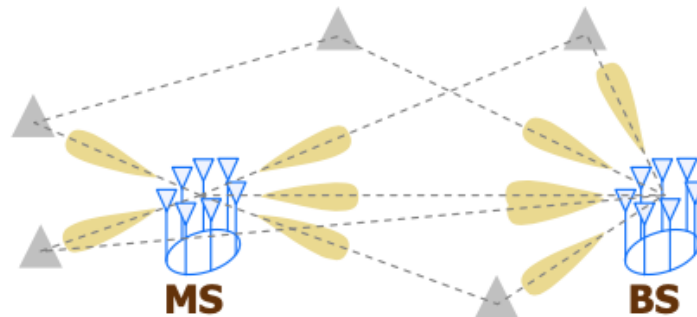


Abbildung 7: Schematische Darstellung einer **MIMO** Kanalmessung. Grafik aus [5].

können, wie in Abbildung 7 dargestellt. Anschließend nutzt man spezielle Signalverarbeitungstechniken [7], die einerseits unter gewissen physikalischen Annahmen das Ausbreitungsverhalten aus den gemessenen Daten ableiten können, und andererseits gleichzeitig den Einfluss des Messsystems so weit wie möglich aus den geschätzten Kanalstatistiken entfernen. Schließlich ist man an der Realität außerhalb des Messaufbaus interessiert.

Natürlich sind hierzu vor der Messung präzise Kalibriermessungen des Systems notwendig. Wir wollen uns im folgenden auf die Wirkung der benutzten Antennenarrays konzentrieren, da diese – wie wir sehen werden – eine gewisse Sonderbehandlung benötigen. Zunächst stellt man bei der Konzipierung und Benutzung des Messsystems sicher, dass es sich um ein LTI System handelt. Betrachtet man nun das Verhalten des Systems im Frequenzbereich für ein einzelnes Paar von Sende- und Empfangsantenne, dann gilt demnach zunächst

$$Y(f) = G_{\text{rx}}(f) \cdot H(f) \cdot G_{\text{tx}}(f) \cdot X(f). \quad (5.2.11)$$

Hierbei steht X für die Anregung des Systems durch ein eingegebenes Signal, $G_{\text{tx/rx}}$ für die Transferfunktion der Sender- bzw. Empfängerhardware, und H für die Transferfunktion des Funkkanals, der demnach auch als ein LTI System modelliert wird. Es stellt sich aber heraus, dass jede Antenne eine *winkelabhängige* Richtcharakteristik besitzt. Das heißt, dass die Systemantworten $G_{\text{tx/rx}}$ davon abhängig sind, in welche Richtungen sich die Wellen vom Sender tx ausbreiten und aus welchen Richtungen, sie am Empfänger rx eintreffen.

Um dies korrekt zu modellieren, muss man sich also zunächst auf einzelne sog. *Ausbreitungspfade* konzentrieren. Das heißt wir nehmen an, dass eine ebene Welle sich in die normierte Richtung Ω_{tx} ausbreitet und nach ihrem Weg durch den Funkkanal am Empfänger aus normierter Richtung Ω_{rx} eintrifft. Folglich ergibt sich für dieses Verhalten

$$Y(f, \Omega_{\text{tx}}, \Omega_{\text{rx}}) = G_{\text{rx}}(f) \cdot a_{\text{rx}}(f, \Omega_{\text{rx}}) \cdot H(f, \Omega_{\text{tx}}, \Omega_{\text{rx}}) \cdot a_{\text{tx}}(f, \Omega_{\text{tx}}) \cdot G_{\text{tx}}(f) \cdot X(f), \quad (5.2.12)$$

wobei $a_{\text{tx/rx}}$ für die richtungs- und frequenzabhängige Antwort der Sende- und Empfangsantenne stehen. In diesem Fall bezeichnet also $H(f, \Omega_{\text{tx}}, \Omega_{\text{rx}})$ die Transferfunktion eines einzelnen Pfades, der den Sender in Richtung Ω_{tx} verlässt und am Empfänger aus Richtung Ω_{rx} eintrifft. Das heißt, wir haben in diesem Fall das Verhalten der Antennen vom Rest des Systems isoliert.

Die Transferfunktion für die gesamte Messung wird dann als Summe der Transferfunktionen solcher ebenen Wellen modelliert, also via

$$Y(f) = \sum_{s=1}^S Y(f, \Omega_{\text{tx},s}, \Omega_{\text{rx},s}), \quad (5.2.13)$$

was sich dadurch rechtfertigt, dass die Transferfunktion des Kanals sich aus der Lösung einer partiellen Differentialgleichung ergibt, deren Lösungsraum lineare Struktur hat. Es zeigt sich aus (5.2.12), dass wir eine möglichst präzise Formulierung für $a_{\text{tx/rx}}$ benötigen, um die Transferfunktion des Kanals H korrekt bestimmen zu können.

Messvorgang Es ist also unsere Aufgabe für eine gegebene Antenne ein parametrisches Modell $a : [0, \pi] \times [0, 2\pi] \rightarrow \mathbb{C}$ der Form $a(\varphi, \vartheta) \in \mathbb{C}$, also in Betrag und Phase, herzuleiten. Aus Gründen der Einfachheit und der Physik vernachlässigen wir die Frequenzabhängigkeit der Antenne und konzentrieren uns auf ihr Verhalten für die Anregung mit einer einzelnen Frequenz. Auch die Polarisierung von ebenen Wellen und das davon abhängige Verhalten einer Antenne vernachlässigen wir hier. Wir konzentrieren uns also auf die *Winkelabhängigkeit* der Antennenantwort.

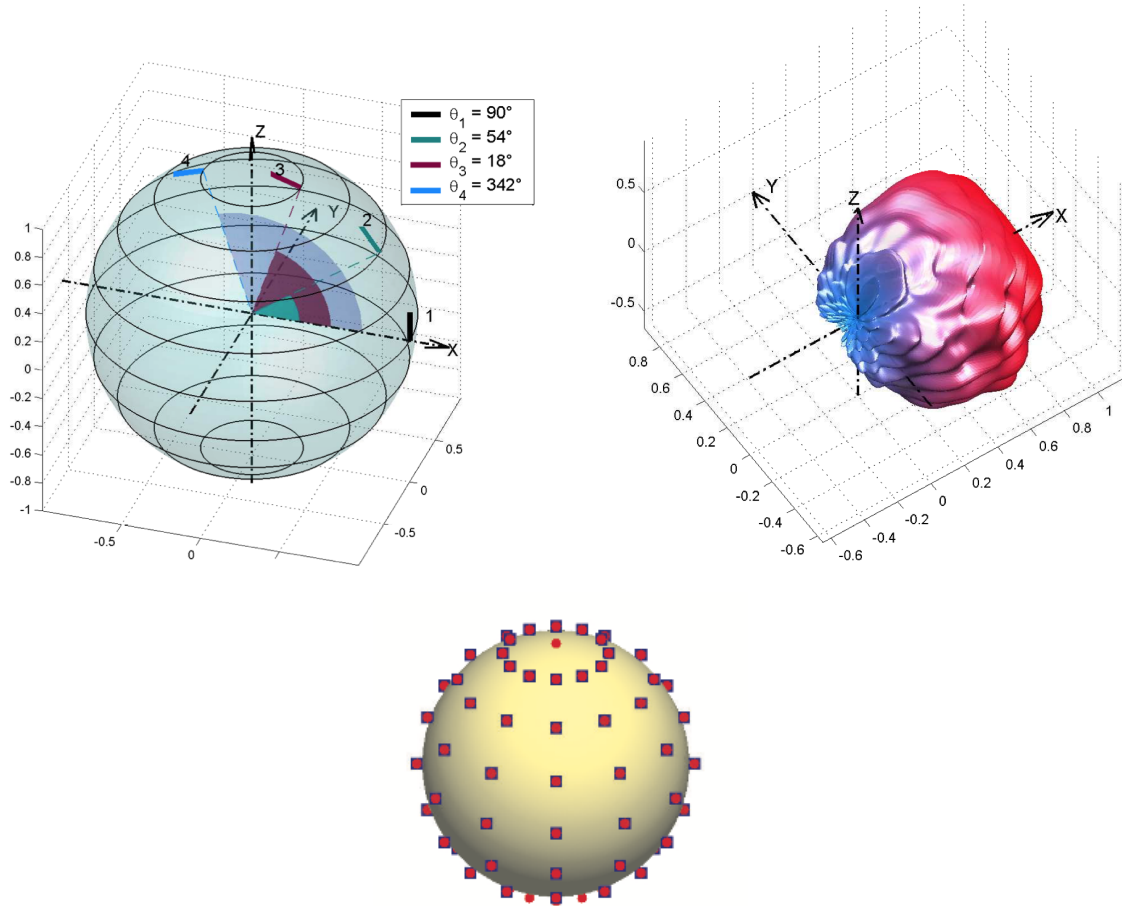


Abbildung 8: Links Oben: Darstellung der Messpositionen für eine Antenne, oder eine Antennenstruktur, welche sich im Ursprung des abgebildeten Koordinatensystems befindet. Rechts Oben: Darstellung der winkelabhängigen Amplitude eines einzelnen Patch-Elements. Unten: Messpunkte für die Abtastung der Funktion a . Grafiken aus [8, 4].

Wie oben motiviert benötigen wir eine kontinuierliche Beschreibung der Antennenantwort. Doch diese ist uns wegen endlichem Speicherplatz auf Festplatten und angepeilter endlicher Messzeit nicht direkt zugänglich. Man weiß jedoch, dass es einen Zusammenhang zwischen der elektrischen Größe einer Antenne und deren winkelabhängigen Verhalten gibt [4, Kapitel 4]. Das heißt, man kann zeigen, dass die Funktion a *bandbegrenzt* ist, beziehungsweise sich sehr gut durch eine bandbegrenzte Funktion *approximieren* lässt. Weiterhin ist durch die Stetigkeit der Physik jede Antennencharakteristik periodisch. *Fourier-Reihen-Sound intensifies*

Das heißt weiterhin, dass es uns möglich ist, die Antenne an diskreten Stellen abzutasten, sodass wir mit der Annahme der Bandbegrenzung und der Aussage des Nyquist-Theorems ein Modell ableiten können, welches die Antenne vollständig charakterisiert. In diesem Sinne geht es darum die kontinuierliche Antennenantwort a zu "digitalisieren". Die "Abtastung" erfolgt demnach im Winkelbereich. Der zugehörige "Frequenzbereich" ist entsprechend der *räumliche Frequenzbereich*. Abbildung 8 zeigt einerseits schematisch den

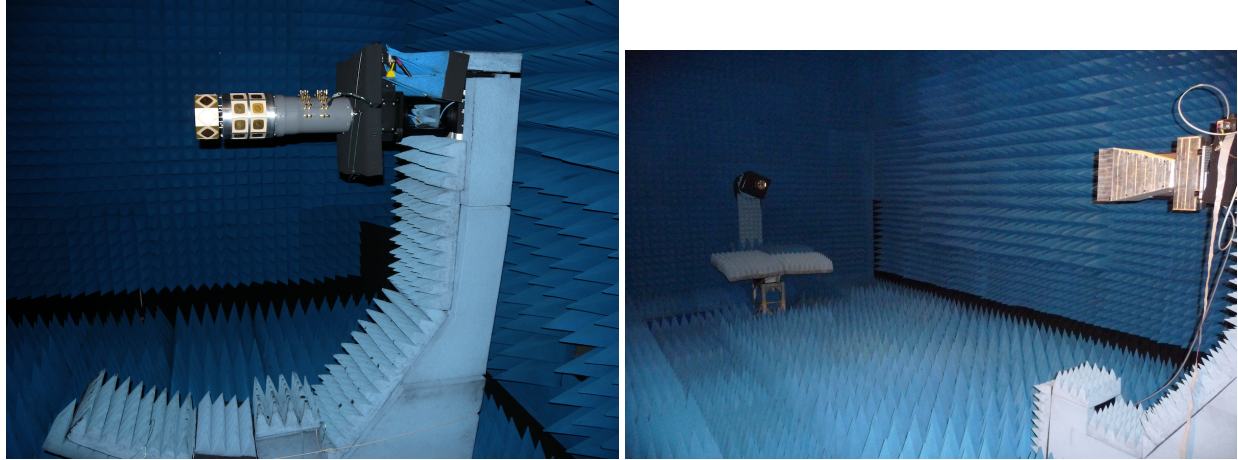


Abbildung 9: Messaufbau zur Kalibrierung eines Antennenarrays. Links: Drehteller für die Positionierung der AUT. Rechts: Weiterer Blickwinkel mit Referenzantenne.

Messaufbau, das genutzte Koordinatensystem und beispielhaft die 3D-Darstellung einer Antennenantwort eines einzelnen Patch-Elements.

Die Messung selbst erfolgt in einer echofreien Messkammer, in welcher es möglich ist, die **Antenna Under Test (AUT)** beliebig relativ zu einer bereits kalibrierten Referenzantenne zu verdrehen, sodass ein Abstraster, wie in Abbildung 8 unten gezeigt, entsteht. Pro Ausrichtung wird die AUT für gewöhnlich für mehrere Frequenzen, zwei orthogonale Polarisierungen und alle ihre Elemente vermessen, bevor die nächste Ausrichtung angefahren wird. Wie erwähnt konzentrieren wir uns auf eine einzelne Frequenz, ein einzelnes Element und eine seiner Polarisierungen. In Abbildung 9 sieht man den Messaufbau, der von unserem FG benutzt wurde, um ein Antennenarray mit 32 Elementen zu vermessen.

Für ein einzelnes Antennenelement beobachten wir also die Funktion a auf einem Gitter, das aus allen Kombinationen der Punkte

$$\varphi_0, \dots, \varphi_{N_\varphi}, \varphi_i = \frac{i\pi}{N_\varphi} \quad \text{und} \quad \vartheta_0, \dots, \vartheta_{N_\vartheta-1}, \vartheta_j = \frac{j2\pi}{N_\vartheta}$$

besteht. Wir erhalten also ein zweidimensionales Array $a[i, j] \in \mathbb{C}^{N_\varphi \times N_\vartheta-1}$, welches wir noch durch einen Trick geeignet periodifizieren müssen, wie in Abbildung 10 links dargestellt. Diese Abtastwerte entspringen also einer zweidimensionalen bandbegrenzten, periodischen Funktion. Aufgabe ist es nun aus diesen Werten eine geeignete Interpolante herzuleiten, die sich diese beiden Eigenschaften zunutze macht.

Ableitung der EADF Wir wollen nun (5.2.10) aus Abschnitt 5.2 benutzen und auf zwei Dimensionen erweitern. Wir folgen damit effektiv [8]. Außerdem ändern wir das Argument der Funktion x und deren Namen zu der oben eingeführten Schreibweise $a : [0, 2\pi] \times [0, 2\pi] \rightarrow \mathbb{C}$ mit Werten $a(\varphi, \vartheta)$. In unserem Fall der Interpolation von Antennenantworten wissen wir, dass a in *beiden* Argumenten 2π -periodisch ist.

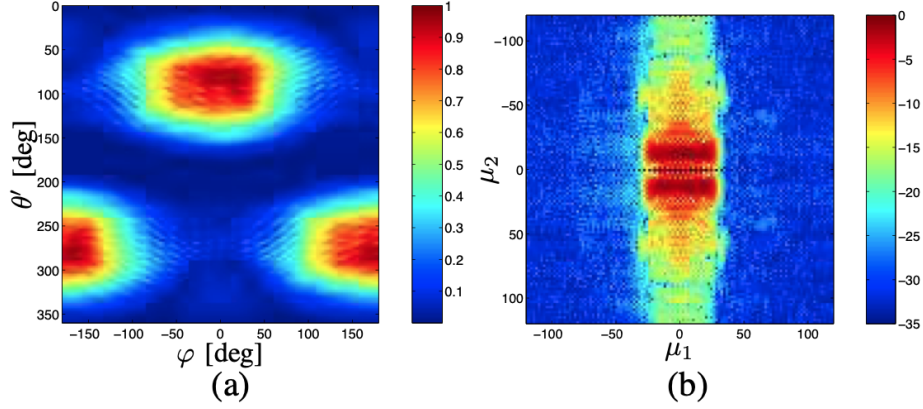


Abbildung 10: Links: Periodifiziertes 2D Array $a[n_\varphi, n_\vartheta]$ der gemessenen Amplituden eines einzelnen Patch-Elements. Rechts: Der Betrag der zugehörigen EADF, wobei hier $\mu_1 = k_\varphi$ und $\mu_2 = k_\vartheta$. Grafik aus [8]

Die Bandlimitierung von a lässt sich formulieren, indem man fordert, dass die Bedingungen

$$A_\varphi(F_\varphi, \vartheta) = \int_{-\infty}^{+\infty} a(\varphi, \vartheta) \exp(-j2\pi F_\varphi \varphi) d\varphi = 0 \quad \text{für } F_\varphi > B_\varphi \quad \text{und alle } \vartheta \in [0, 2\pi] \quad \text{und} \quad (5.2.14)$$

$$A_\vartheta(\varphi, F_\vartheta) = \int_{-\infty}^{+\infty} a(\varphi, \vartheta) \exp(-j2\pi F_\vartheta \vartheta) d\vartheta = 0 \quad \text{für } F_\vartheta > B_\vartheta \quad \text{und alle } \varphi \in [0, 2\pi] \quad (5.2.15)$$

erfüllt sein müssen. Nun lassen sich alle obigen Argumente in Abschnitt 5.2.1 „schnittweise“ auf eine abgetastete Version von a in der Form $a[n_\varphi, n_\vartheta]$ anwenden. Das heißt, wir landen schlussendlich bei einer Interpolations-Formel

$$a(\varphi, \vartheta) = \sum_{k_\varphi = -\frac{B_\varphi}{F_\varphi}}^{+\frac{B_\varphi}{F_\varphi}} \sum_{k_\vartheta = -\frac{B_\vartheta}{F_\vartheta}}^{+\frac{B_\vartheta}{F_\vartheta}} A[k_\varphi, k_\vartheta] \cdot \exp(j2\pi k_\varphi \varphi F_\varphi) \cdot \exp(j2\pi k_\vartheta \vartheta F_\vartheta), \quad (5.2.16)$$

welche eine absolut analoge (nicht als Gegenteil zu digitale) 2D-Version zu (5.2.10) darstellt. Auch in diesem Fall, können wir das 2D-Array $A[k_\varphi, k_\vartheta]$ durch eine 2D-DFT, bzw. der FFT [3], von den uniformen Samples $a[n_\varphi, n_\vartheta]$ der Funktion a effizient vorberechnen.

Um einen möglichst effizienten Algorithmus für die Auswertung der Interpolante zu erhalten, sollte man (5.2.16) geeignet umschreiben. Moderne Rechenarchitekturen und Scientific-Computing-Libraries sind auf schnelle Matrix-Vektor-Produkte optimiert. Nehmen wir an, wir wollen (5.2.16) für mehrere Winkelpaare $(\varphi_1, \vartheta_1), \dots, (\varphi_L, \vartheta_L)$ auswerten. Dann berechnen wir zunächst zwei 2D Arrays

$$D_\varphi = [\exp(j2\pi k_\varphi \varphi F_\varphi)]_{\ell=1, k_\varphi = -\frac{B_\varphi}{F_\varphi}} \in \mathbb{C}^{L \times 2\frac{B_\varphi}{F_\varphi} + 1} \quad \text{und} \quad (5.2.17)$$

$$D_\vartheta = [\exp(j2\pi k_\vartheta \vartheta F_\vartheta)]_{\ell=1, k_\vartheta = -\frac{B_\vartheta}{F_\vartheta}} \in \mathbb{C}^{L \times 2\frac{B_\vartheta}{F_\vartheta} + 1}, \quad (5.2.18)$$

was uns erlaubt (5.2.16) in das folgende Vektor-Matrix-Vektor-Produkt

$$a(\varphi_\ell, \vartheta_\ell) = D_\varphi[\ell, :] \cdot A[:, :] \cdot D_\vartheta[\ell, :]^T \quad (5.2.19)$$

umzuschreiben. Damit besteht der Interpolations-Algorithmus zunächst aus der Vorberechnung des Arrays A , sowie bei Ausführung dann aus der Berechnung von D_φ und D_θ , sowie der Auswertung von (5.2.19).

Man sieht hier der schön, dass die Laufzeitkomplexität von (5.2.19) maßgeblich von der räumlichen Bandbegrenzung der Antennen-Richtcharakteristik beeinflusst wird. Je höher die Bandbreite, desto höher ist nicht nur der Aufwand bei der Messung, sondern auch bei der Berechnung Interpolation. Eine alternative Form der Interpolation, welche diese eventuell nachteilige Eigenschaft nicht hat, ist in Abschnitt 7 dargestellt.

5.3 Schnelle Korrelation für Radar-Signalverarbeitung

Wir wollen noch eine weitere wichtige Anwendung für die in dieser Vorlesung entwickelten Signalverarbeitungstechniken betrachten. Für die Konzipierung eines RADAR-Systems macht man sich zu Nutze, dass sich ausbreitende elektro-magnetische Wellen an Objekten, auf die sie während der Ausbreitung treffen, reflektiert werden. Mit einer Antenne kann man beispielsweise ein Signal $x[\cdot]$ mit einem **transmitter (TX)** aussenden und mit einer zweiten Antenne am **receiver (RX)** auf das eventuelle Echo warten. Wird ein Objekt mit Entfernung d nun von dieser Welle getroffen und reflektiert einen Teil der Energie in Richtung **RX**, so enthält das empfangene Signal $y[\cdot]$ eine zeitlich verzögerte Kopie von $x[\cdot]$ und eine gewisse Menge an Rauschen, das von den Schaltkreisen an **TX** und **RX** eingepreßt wird. Es gilt also

$$y[\cdot] = \gamma x[\cdot - \tau] + w[\cdot],$$

wobei sich τ aus der Samplerate F_s , dem Abstand d und der Lichtgeschwindigkeit c via

$$\tau = F_s \cdot \frac{2d}{c}$$

ergibt. Die Zahl $\gamma \in \mathbb{R}$ repräsentiert einerseits Verluste, die sich durch die Ausbreitung im Raum ergeben und gleichzeitig aus der „Reflektivität“ des Objektes. Für gewöhnlich hat ein RADAR zwei Aufgaben. Einerseits soll detektiert werden, *ob* ein Objekt von unserem RADAR angeleuchtet wurde und im Idealfall, *wie groß* die zurückgelegte Strecke d ist. Wir wollen uns also Signalverarbeitungstechniken ansehen, die die Bestimmung von d erlauben.

5.3.1 Zyklische Korrelationen

Eine Möglichkeit, um d zu erhalten basiert auf Korrelation. Hierzu schränken wir zunächst die Menge der Sendesequenzen ein, indem wir fordern, dass $x[\cdot]$ periodisch mit Periodendauer N ist. Die Korrelation $r_{x,y}$ von zwei periodischen Sequenzen $x[\cdot]$ und $y[\cdot]$ ist gegeben durch

$$r_{x,y}[\ell] = \frac{1}{N} \sum_{n=0}^N x[n]y[n - \ell]. \quad (5.3.1)$$

Stellen wir uns vor, dass unser RADAR-System die Sendesequenz $x[\cdot]$ erzeugt, indem es eine endliche Sequenz $x\mathbb{R}^N$ wiederholt aussendet, so ist $x[\cdot]$ periodisch mit Periodendauer N und auch $y[\cdot]$ – nach den ersten N Samples, wenn wir N so groß gewählt haben, dass alle Echos nach der Aussendung von N Samples am Empfänger angekommen sind. Dies ist gleichbedeutend damit, dass die Sendesequenz länger ist als die Impulsantwort des **LT** Systems, das die Wellenausbreitung in der Umgebung repräsentiert.

Die Frage ist nun, was uns diese Korrelation nützt. Man kann sich überlegen, dass

$$|r_{x,y}[\ell]| \leq \sqrt{r_{x,x}[0] \cdot r_{y,y}[0]} = \sqrt{\mathcal{E}(x)\mathcal{E}(y)},$$

siehe Gleichung (3.1.1). Das heißt, dass die sog. *Kreuzkorrelation* $r_{x,y}[\cdot]$ immer durch die Werte der sog. *Autokorrelation* $r_{x,x}[\cdot]$ wie oben angegeben beschränkt ist. Im Spezialfall $y = x$ ergibt sich

$$|r_{x,x}[\ell]| \leq r_{x,x}[0],$$

was heißt, dass die **Auto-Correlation Function (ACF)** $r_{x,x}[\cdot]$ ihr Maximum immer am Wert $\ell = 0$ annimmt.

Betrachten wir nun wieder unser RADAR-System, so könnte es sich als sinnvoll erweisen, am Empfänger das Signal $y[\cdot]$ mit dem Sendesignal zu korrelieren, also

$$r_{y,x}[\ell] = \gamma r_{x,x}[\ell - \tau] + r_{w,x}[\ell] = \gamma r_{x,x}[\ell - \tau] + r_{w,x}[\ell]$$

zu berechnen. Da $r_{x,x}[\cdot]$ bei 0 ein lokales Maximum hat, so hat $r_{x,x}[\ell - \tau]$ ein lokales Maximum bei τ ! Wir könnten also einfach ein lokales Maximum in $r_{y,x}[\cdot]$ suchen und dessen Stelle in d umrechnen.

Die naive Berechnung von $r_{y,x}[\cdot]$ für N Werte erfordert in der Größenordnung N^2 viele **FLOPs**. Man kann aber ziemlich einfach einsehen, dass man die Kreuzkorrelation als Faltung via

$$r_{x,y} = x[\cdot] \otimes y[-\cdot] \quad (5.3.2)$$

ausdrücken kann. Außerdem gilt für eine Sequenz $x[\cdot]$ und deren **DFT** $X[\cdot]$, dass

$$\text{DFT}(x[-\cdot]) = X[\cdot]^*, \quad (5.3.3)$$

also Umkehrung im Zeitbereich (Reflexion an $n = 0$) entspricht Konjugation im Frequenzbereich. Demzufolge können wir Abschnitt 4.4.2 zusammen mit (5.3.2) verwenden, um $r_{x,y}[\cdot]$ mit $N \log(N)$ vielen **FLOPs** zu berechnen. Das heißt, dass für die Signalverarbeitung im RADAR effiziente Methoden bereitstehen, solange der Ansatz der Berechnung der Kreuzkorrelation des Empfangssignals mit der Sendesequenz als theoretisch fundiert herausstellt.

In Codeschnipsel 28 sind einige verschiedene Sendesignale miteinander verglichen, indem zwei Reflexionen mit verschiedenen Abständen simuliert werden und aus dem resultierenden Empfangssignal $y[\cdot]$ wiederum $r_{x,y}[\cdot]$ bestimmt wird. Wie man sieht, hat das Sendesignal einen signifikanten Einfluss auf die „Form“ der **ACF**, welche wiederum einen Einfluss auf die visuelle Qualität der Funktion $r_{x,y}[\cdot]$ hat. Je stärker ein lokales Maximum ausgeprägt ist, desto besser. Nutzt man nur einen einfachen Sinus als Sendesignal (Codeschnipsel 28, oben), so kann man gar keine Information über den Abstand von eventuellen Reflexionen finden. Der Grund ist, dass sich Zeitversatz nur mit Signalen schätzen lässt, die eine gewisse Bandbreite abdecken, also mehrere Frequenzen gleichzeitig belegen.

Mit einer sinc-Funktion (Codeschnipsel 28 mitte) belegen wir bekanntermaßen im Frequenzbereich einen ausgedehnten Bereich, was dazu führt, dass wir in der Tat sehr ausgeprägte lokale Maxima bei den wahren τ erhalten. Je schmaler die Hauptkeule der sinc-Funktion im Zeitbereich, desto breiter ist der abgedeckte Frequenzbereich, was leicht durch Modifikation von Codeschnipsel 28 überprüft werden kann. Dies ist von Vorteil, wenn man mehrere Reflexionen, die in etwa gleichen Abständen auftreten doch noch von einander unterscheiden möchte. Es ist jedoch zu bemerken, dass ein im Zeitbereich „schmaler“ sinc an die Hardware, die ihn erzeugt einige unangenehme Herausforderungen stellt, weshalb solch ein Sendesignal in der Praxis einige Nachteile aufweist.

Schlussendlich sehen wir (Codeschnipsel 28, unten), dass zufällige Sendesignale auch gute Eigenschaften bezüglich $r_{x,y}[\cdot]$ liefern. Gleichzeitig haben diese den Nachteil, dass nicht klar ist, wie man Signale generiert, die sich wie zufällige Sendesignale verhalten, sich aber dennoch kontrolliert und mit einem deterministischen Prozess erzeugen lassen.

Speziell für unsere Anwendung im RADAR streben wir also an, dass die **ACF** $r_{x,y}[\cdot]$ ein ausgeprägtes lokales Maximum hat, also einerseits sehr schnell abfällt, und andererseits auch wenig bis keine Nebenmaxima erzeugt, weil sich dies beispielsweise in Codeschnipsel 24 als Problem herausgestellt hatte. Im Idealfall finden wir eine Sendesequenz, die sich einerseits gut für RADAR eignet und gleichzeitig sich beispielsweise effizient in Hardware erzeugen lässt.

```
def gen_receive(x, tau, gamma):
    return np.sum(
        [
            np.roll(x, tt) * gg
            for tt, gg in zip(tau, gamma)
        ],
        axis=0,
    )
```

```
N = 64
```

```
tau = [14, 34]
```

```
gamma = [1, 0.75]
```

```
x = np.sin(np.linspace(0, 10 * np.pi, N))
```

```
x = np.sinc(np.linspace(-25, +25, N))
```

```
# x = np.sign(np.random.randn(N))
```

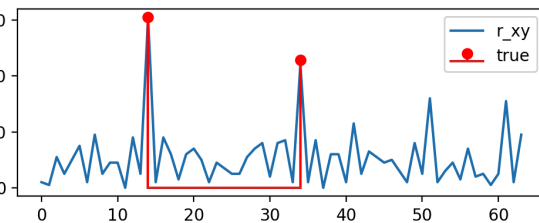
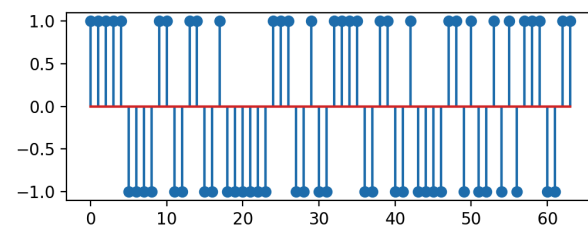
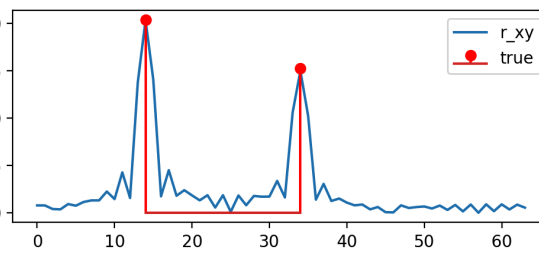
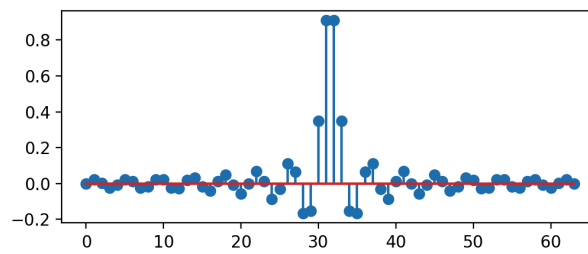
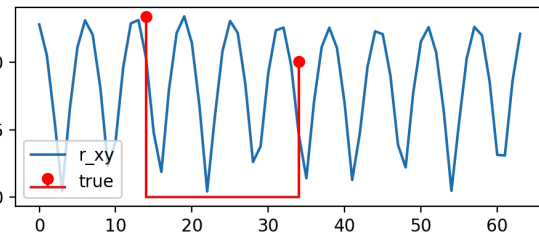
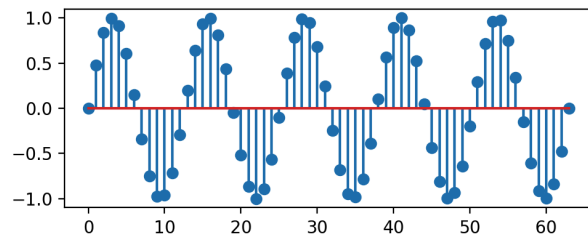
```
X = np.fft.fft(x)
```

```
y = gen_receive(x, tau, gamma)
```

```
Y = np.fft.fft(y)
```

```
# Gleichung (5.3.2)
```

```
r_xy = np.fft.ifft(Y * X.conj())
```



Codeschnipsel 28: Vergleich verschiedener Sendesignale $x[\cdot]$ bezüglich der resultierenden Korrelation $r_{x,y}[\cdot]$, siehe [dsv/code/radar1.py](#)

5.3.2 Erzeugung der Sendesequenz

Wir wollen im Folgenden eine mögliche Art der Sendesequenzherzeugung genauer untersuchen, da sie beide obigen Anforderungen erfüllt. Hierzu betrachten wir sogenannte **Linear Feedback Shift Registers (LFSRs)**, wie in Abbildung 11 dargestellt. Diese bestehen aus L Registern/Taps, welche jeweils ein Bit beinhalten. Die Idee ist nun dieses **LFSR** als diskretes System zu betrachten, dessen Werte in der Menge $\mathbb{Z}_2^L = \{0, 1\}^L$ liegen können und diese Menge $\mathbb{Z}_2$ mit der binären Addition und Multiplikation ausgestattet wurde. In jedem Takt werden die Werte von Links nach Rechts verschoben, wobei der Zustand des letzten Registers als Ausgabe fungiert. Weiterhin werden einige der Zustände durch binäre Addition miteinander verknüpft und an den Eingang, also das erste Register, als Feedback zurückgeführt. Es ist demnach auch leicht zu sehen, dass solch ein System sehr leicht und effizient in diskreter Hardware implementiert werden kann.

Zum Zeitpunkt n ist damit die Ausgabe

$$x[n] = a_1x[n-1] + a_2x[n-2] + \dots + a_{L-1}x[n-(L-1)] + a_Lx[n-L],$$

wobei die Koeffizienten $a_1, \dots, a_L \in \mathbb{Z}_2$, diese als Bits also definieren, welche Werte der Register für die Berechnung des Feedbacks genutzt werden. Es ist anzumerken, dass man nun für einen Start der Simulation die Zustände $x[-L], \dots, x[-1]$ kennen muss. Man kann das System zum Zeitpunkt n durch die Werte $x[n], \dots, x[n-L]$ also eine Binärzahl mit L Stellen/Bits beschreiben. Kennt man all diese Zustände, ist es auch möglich den Nächsten zu berechnen, indem auf die Zahl ein Bitshift und eine „maskierte“ Addition über die Bits des aktuellen Zustands angewandt wird.

Man sieht nun, dass sich mit L binären Registern 2^L Zustände abbilden lassen, und dass das System maximal $2^L - 1$ von diesen durchlaufen kann, wenn man nicht im Zustand $0 \dots 0$ beginnt. Die Werte $y[n]$ müssen demnach periodisch sein, denn sobald sich ein einzelner der $2^L - 1$ möglichen Zustände ein zweites Mal einstellt, folgen ihm wieder diesselben wie beim ersten Auftreten. Demnach ist auch die maximale Periodenlänge mit $2^L - 1$ beschränkt. Die Frage ist nun, ob es Koeffizienten-Konfigurationen gibt, für welche die maximale Periodenlänge erreicht wird.

Hierzu betrachten wir die z-Transformation des Systems, dann finden wir

$$H(z) = a_1z^1 + a_2z^2 + \dots a_{L-1}z^{L-1} + a_Lz^L,$$

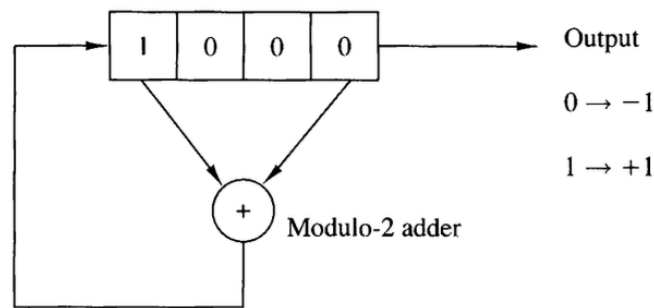


Abbildung 11: Lineares Feedback Shift Register, Quelle [1]

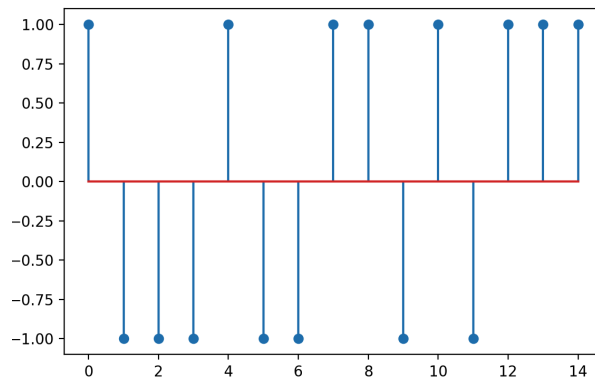
woran wir wiederum sehen, dass das System nur durch die Wahl der Koeffizienten $a_1, \dots, a_L \in \mathbb{Z}_2$ bestimmt wird. Man kann zeigen, dass falls die Transferfunktion H ein sogenanntes irreduzibles und primitives Polynom¹⁷ ist, so hat die von diesem LFSR erzeugte Sequenz maximale Länge, also $2^L - 1$. Das heißt, dass die algebraischen Eigenschaften des z-Transformierten Systems, der Transferfunktion H , wieder Aufschluss darüber geben, welche Eigenschaften das LFSR. In Codeschnipsel 29 wird ausgehend von einem Startwert/-zustand des Registers (0×1) und einer sogenannten Tapkonfiguration ($0 \times D$), welche den Feedback kontrolliert, eine **Maximum Length Binary Sequence (MLBS)** durch einmaliges Simulieren des LFSR erzeugt. Die entstehende Folge hat ähnliche Eigenschaften wie eine „wirklich“ zufällige Zahlenfolge, doch entsteht aus einem vollständig deterministischem Prozess. Man nennt solche Sequenzen „pseudo zufällig“.

```
def gen_MLBS(taps: int, start: int):
    numTaps = int(
        np.ceil(np.log2(taps)).astype(int)
    )
    length = 2**numTaps - 1
    state = start

    sequence = []
    for ll in range(length):
        # move last bit to output
        sequence.append(state & 1)
        feedback = state
        for tt in range(1, numTaps - 1):
            if (taps >> (numTaps - tt - 1)) & 1:
                feedback ^= state >> tt
        # keep only last bit of combined XOR
        feedback &= 1
        # shift registers
        state = state >> 1
        # feedback from back to front
        state |= feedback << (numTaps - 1)

    sequence = 2 * (np.array(sequence) - 0.5)
    return sequence

if __name__ == "__main__":
    sequence = gen_MLBS(0xC + 1, 1)
```



Codeschnipsel 29: Simulation eines LFSR mittels binären Operationen, siehe [dsv/code/mlbs.py](#)

5.3.3 MLBS als RADAR-Sendesequenz

Zum Abschluss wollen wir nun noch untersuchen, wie gut sich die so gewonnenen Sequenzen für RADAR verwenden lassen. Hierzu betrachten wir die Ausgabe von Codeschnipsel 30, wo wir das Wissen aus Code-

¹⁷Siehe, https://de.wikipedia.org/wiki/Irreduzibles_Polynom. Irreduzible Polynome treten im Zusammenhang mit endlichen Körpern auf, beispielsweise $\mathbb{Z}_2$, wo eine andere Art Polynomdivision angewandt werden kann/muss. Irreduzible Polynome nehmen im Grunde hier die Rolle von Primzahlen ein, da sich jedes Polynom in ein Produkt aus irreduziblen Polynomen zerlegen lässt. Mehr Infos gibt es in [9], wo auch eine explizite Formel für die entstehende Sequenz entwickelt wird.

schnipsel 28 und Codeschnipsel 29 kombiniert haben, um einerseits „hardware-effizient“ eine Sendesequenz zu erzeugen und andererseits, um die RADAR-Signalverarbeitung „rechen-effizient“ zu gestalten.

```
from mlbs import gen_MLBS
from radar1 import gen_receive

x = gen_MLBS(0x44, 1)

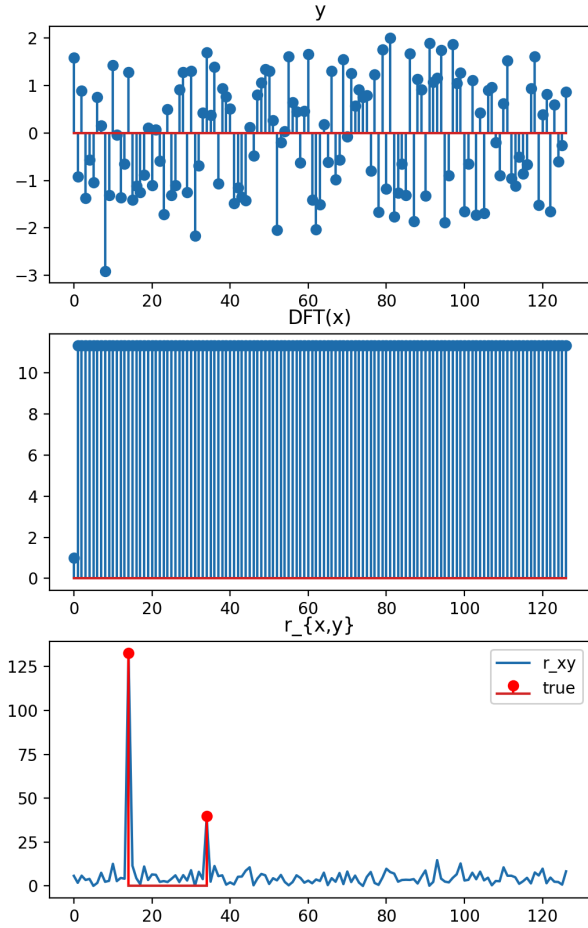
tau = [14, 34]
gamma = [1, 0.3]

N = x.size

x = x
X = np.fft.fft(x)

y = gen_receive(x, tau, gamma) + (
    0.5 * np.random.randn(x.size)
)
Y = np.fft.fft(y)

r_xy = np.fft.ifft(Y * X.conj())
```



Codeschnipsel 30: Simulation eines RADAR-Systems basierend auf MLBS, die von LFSR erzeugt wurden. Oben verrauschtes Empfangssignal $y[\cdot]$, mitte DFT $Y[\cdot]$, unten $r_{x,y}$, siehe [dsv/code/radar2.py](#)

Wie wir sehen, erzeugt das **LFSR** eine Sequenz, welche sehr gut zu unseren Anforderungen passt. Einerseits ist die Aussteuerung des Signals sehr begrenzt und andererseits hat die **ACF** ideale Eigenschaften, um Abstände von Reflexionen zu bestimmen. Man kann sogar beweisen, dass für eine Sequenz maximaler Länge $x[\cdot]$ für die **ACF** gilt, dass

$$r_{x,x}[\ell] = \begin{cases} 1 & \text{für } \ell = 0, \\ -\frac{1}{2^L - 1} & \text{sonst.} \end{cases} \quad (5.3.4)$$

Damit ist das lokale Maximum immer um den Faktor $N = 2^L - 1$ höher als die Werte im Rest der Korrelationsfunktion $r_{x,x}$. Die bedeutet einerseits, dass dieses Verfahren auch mit eventuellem Messrauschen sehr gut umgehen kann. Andererseits kann man auch durch den Einsatz von sehr langen Sequenzen, also mehr Registern im **LFSR**, mit sehr wenig Sendeleistung durch die Transformation von $y[\cdot]$ nach $r_{x,y}[\cdot]$ immernoch Reflexionen detektieren.

6 Stochastische Signalverarbeitung

Bisher waren wir ausschließlich mit deterministischen Signalen befasst. Also solchen, welche sich bequem durch eine Funktionsvorschrift $t \mapsto x(t)$ beschreiben lassen. Doch eigentlich ist dies ein absolut idealisierter Spezialfall, da für gewöhnlich alle Signale, die wir verarbeiten, zu mindestens einem Zeitpunkt von einem Messgerät beobachtet wurden. Somit sind diese notwendigerweise mit Messrauschen behaftet, dessen unangenehme Eigenschaft es ist, zufällig zu sein. Aus diesem Grund ist es uns nicht möglich, ein gemessenes und digitalisiertes Signal $x[\cdot]$ mittels eines algebraischen Ausdrucks zu beschreiben. Notwendigerweise wird $x[\cdot]$ zu einer sog. Zufallsgröße, welche wir „nur“ noch durch deren statistische Eigenschaften beschreiben können. Nicht nur das, auch jede folgende Verarbeitung von $x[\cdot]$, beispielsweise durch ein LTI-System, wird zu einer Zufallsgröße, deren Eigenschaften u.A. von denen des Messrauschens abhängen.

Weiterhin kann es für gewisse Anwendungen auch absolut ausreichend sein, eigentlich deterministische Signale als zufällig zu modellieren. Wenn es beispielsweise darum geht, zu bestimmen, ob ein Kommunikationssystem in der Lage ist, Informationen fehlerfrei zu übertragen, dann kann es ausreichend sein, „nur“ eine Ausfallrate/-wahrscheinlichkeit zu bestimmen, oder diese Rate zu begrenzen, anstatt dies für jedes mögliche zu übertragende Signal zu bestimmen/gewährleisten. Außerdem haben wir auch schon in Abschnitt 5.3 gesehen, dass man die Korrelation von deterministischen Signalen mit Messrauschen in den Griff bekommen muss, wenn man sich mit Radar-Signalverarbeitung beschäftigt.

Um diese „neue“ Sichtweise auf Signale gut verinnerlichen zu können, benötigen wir jedoch einige Werkzeuge aus der Wahrscheinlichkeitstheorie, die wir uns nun ansehen werden, wobei wir versuchen, uns nicht mit vielen mathematischen Details aufzuhalten, sondern wollen stattdessen unsere Intuition schärfen.

6.1 Grundlagen Wahrscheinlichkeitstheorie

Nehmen wir zunächst einmal an, dass wir einen zufälligen Vorgang betrachten, dessen Ausgang sich in Form einer einzelnen reellen Zahl manifestiert. Diesen zufälligen Vorgang assoziieren wir mit der Zufallsgröße $X : \Omega \rightarrow \mathbb{R}$, also $\omega \rightarrow X(\omega)$, wobei wir uns vorstellen, dass Ω , beziehungsweise ω , dafür sorgen, dass Zufall ins Spiel kommt. *MITHÄNDENWEDEL* Wir sind nun daran interessiert, wie die Verteilung der Werte $X(\omega)$ über dem Wertebereich $\mathbb{R}$ charakterisiert werden kann.

Hierzu definieren wir beigeordnet zu X eine Funktion $F_X : \mathbb{R} \rightarrow [0, 1]$, welche uns für $x \in \mathbb{R}$ mitteilt, was die Wahrscheinlichkeit für das zufällig eintretende Ereignis $X \leq x$ ist. Man kann auch sagen/schreiben:

$$F_X(x) = P(X \leq x). \quad (6.1.1)$$

Hier nennt man P auch ein *Wahrscheinlichkeitsmaß*, da P im Grunde die Wahrscheinlichkeitsmasse auf den Wertebereich $\mathbb{R}$ verteilt. Die Funktion F ist monoton wachsend, da natürlich $P(X \leq x_1) \leq P(X \leq x_2)$, falls $x_1 < x_2$, also auch $F_X(x_1) \leq F_X(x_2)$. Jetzt kann man mit der Verteilungsfunktion F_X schon eine Menge anfangen. Beispielsweise können wir damit auch bestimmen, mit welcher Wahrscheinlichkeit das Ereignis $x_1 < X \leq x_2$ eintritt, indem wir einfach

$$P(x_1 < X \leq x_2) = P(\{x_1 < X\} \cup \{X < x_2\}) = F_X(x_2) - F_X(x_1)$$

berechnen.

Wir können sogar noch einen Schritt weiter gehen, und die Größe

$$f_X^h(x) = \frac{F_X(x+h) - F_X(x)}{h} \quad (6.1.2)$$

betrachten. Man kann es sich so vorstellen, dass wir das Intervall $[x, x + h]$ hernehmen, berechnen, wie „viel“ von X in $[x, x + h]$ landen kann, und normieren das mit der Intervallbreite h . Im Grunde berechnen wir, wie „dicht“ die Zufallsgröße zwischen x und $x + h$, oder „um“ x , ist. Aus diesem Grund nennt man den Grenzwert

$$f_X(x) = \lim_{h \rightarrow 0} f_X^h(x) = \lim_{h \rightarrow 0} \frac{F_X(x + h) - F_X(x)}{h} \quad (6.1.3)$$

die *Dichte* von X . Natürlich sehen wir, dass der berechnete Grenzwert der sogenannte Differenzenquotient der Verteilungsfunktion F_X ist. Es gilt also $f_X = F'_X = (d)/dx F_X(x)$. Deshalb kann man die Verteilungsfunktion auch aus der Dichte gewinnen, indem wir

$$F_X(x) = \int_{-\infty}^x f_X(s) ds$$

auswerten. Die obigen Betrachtungen gelten freilich nur, wenn die formulierten Grenzwerte auch immer existieren, doch erstens existieren diese meistens und für ein intuitives Verständnis sollte es zunächst genügen.

Weiterhin gilt für eine reell Zufallsgröße X , dass diese sich „irgendwo“ zwischen $-\infty$ und $+\infty$ mit Wahrscheinlichkeit 1 realisieren muss. Also gilt

$$\lim_{x \rightarrow \infty} F_X(x) = 1,$$

was für die Dichte ergibt, dass

$$\int_{-\infty}^{+\infty} f_X(s) ds = 1.$$

Da F_X monoton wachsend ist, muss auch immer gelten, dass $f_X(x) \geq 0$.

6.1.1 Unabhängigkeit

Gegeben zwei Zufallsgrößen X und Y ist es oft interessant deren Verhältnis zueinander zu untersuchen. Beispielsweise wollen wir $P(X \leq x, Y \leq y)$, also die Wahrscheinlichkeit, dass $X \leq x$ und $Y \leq y$ gemeinsam (gleichzeitig) eintreten, berechnen/untersuchen/kennen. Im allgemeinen Fall, benötigt man hierfür die *gemeinsame* Verteilung von X und Y , also $F_{X,Y}$ oder $f_{X,Y}$, direkt einen zwei-dimensionalen Wertebereich impliziert.

Gilt aber für alle x, y , dass

$$P(X \leq x, Y \leq y) = P(X \leq x) \cdot P(Y \leq y), \quad (6.1.4)$$

so nennt man X und Y (stochastisch) *unabhängig*. Die gemeinsame Verteilung von X und Y ergibt sich als das Produkt der Einzelverteilungen, was *sehr* viele Berechnungen vereinfacht. Jedoch ist es schwer Unabhängigkeit nachzuweisen, weshalb es oft eine Annahme ist, die man im Vorhinein während der Modellierung eines Zufallsprozesses trifft.

6.1.2 Diskrete Verteilungen

Bisher haben wir immer den Fall betrachtet, dass die Verteilungsfunktion F_X (nicht die Dichte f_X , siehe uniforme Verteilung) stetig war. In diesem Fall spricht man auch von stetigen Verteilungen. Deren Eigenschaft

ist (intuitiv), dass wenn $f_X(x) = f_0$ für ein x , dann unterscheiden sich die Werte von f_X in einer kleinen Umgebung von x nicht stark von y_0 . Bei diskreten Verteilungen kann es sein, dass „Wahrscheinlichkeitsmasse“ auf isolierte (diskrete) *Punkte* konzentriert ist und somit die obige Stetigkeitsbedingung nicht erfüllt ist.

Ein einfaches Beispiel (siehe Abschnitt 6.4) wäre die quantisierte Version $Q(X)$ einer stetigen Zufallsgröße X . Dann erhalten wir mit $Q(X)$ wieder eine Zufallsgröße, deren Werte auf die Quantisierungsstufen $\mathcal{Q}$ beschränkt sind. Da wir später diese Art von Verteilung irgendwie in den Griff bekommen müssen, wollen wir uns kurz damit befassen.

Eigentlich müssen diskrete Verteilungen kein Spezialfall von kontinuierlichen Verteilungen sein, wenn man die richtige Theorie¹⁸ verwendet. Um das zu vermeiden, müssen wir auf eine etwas unsaubere Schreib- und Denkweise zurückgreifen und im Hinterkopf behalten, dass hier einige Spitzfindigkeiten versteckt sind, die wir nicht weiter diskutieren.

Hierzu nutzen wir den stetigen Dirac-Stoß. Gegeben eine Menge von diskreten Punkten der Form $D = \{d_k | k \in \mathbb{Z}\}$ und zugehörige Wahrscheinlichkeiten $p_k \geq 0$ für $k \in \mathbb{Z}$ mit der Eigenschaft, dass

$$\sum_{k \in \mathbb{Z}} p_k = 1,$$

dann definieren wir eine diskrete Zufallsgröße X mit Dichte f_X durch

$$f_X(x) = \sum_{k \in \mathbb{Z}} p_k \delta(x - d_k). \quad (6.1.5)$$

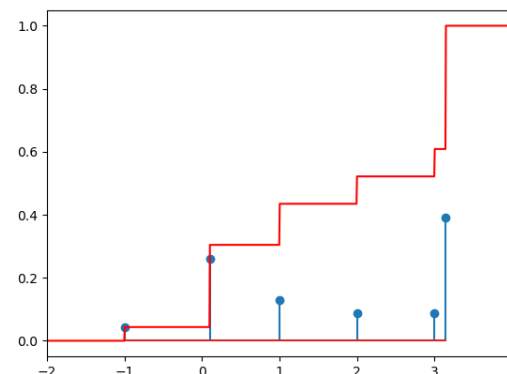
Wir heften also an die Punkte in D jeweils einen Dirac-Stoß, der mit der Wahrscheinlichkeit p_k für diesen Punkt gewichtet wird. Dann ergibt sich also direkt aus den Eigenschaften von δ , dass $P(X = d_k) = p_k$ und $P(X = x) = 0$ für $x \notin D$, siehe Codeschnipsel 31.

```
D = np.array([-1, 2, 3, 1, 0.1, np.pi])
p = np.array([1, 2, 2, 3, 6, 9])
p = p / np.sum(p) # so that sum(p) = 1

def pdf(x):
    where = D <= x
    return np.sum(p[where])

x = np.linspace(-2, +4, 1000)

plt.stem(D, p)
plt.plot(x, [pdf(xx) for xx in x], color="r")
plt.xlim(x.min(), x.max())
```



Codeschnipsel 31: Visualisierung der Dichte und der Verteilungsfunktion einer diskreten Zufallsgröße, siehe [dsv/code/random/discrete.py](#)

Mithilfe von dem Dirac-Stoß δ sind wir also in der Lage, weiterhin die aus der Signalverarbeitung bekannten Integrale und deren Eigenschaften in Bezug auf δ zu nutzen.

¹⁸[https://en.wikipedia.org/wiki/Measure_\(mathematics\)](https://en.wikipedia.org/wiki/Measure_(mathematics))

6.1.3 Beispiele

Es ist zunächst noch eine Schreibweise einzuführen. Wir werden die Notation $X \sim V$, wobei gemeint ist, dass V für eine Verteilung steht, die die Dichte f_X und damit auch die Verteilungsfunktion F_X und damit auch das oben definierte P festlegt. Wenn wir nun schreiben $X, Y \sim V$ (oder kurz: $X \sim Y$), dann meint das *nicht*, dass $X(\omega) = Y(\omega)$! Es meint nur, dass X und Y derselben Verteilung folgen, aber nicht, dass deren Realisierungen $X(\omega)$ und $Y(\omega)$ immer diesselben Werte annehmen. Das Ereignis $X = Y$ tritt also nur mit einer Wahrscheinlichkeit $P(X = Y)$ ein, wobei wir hierfür wiederum Kenntnis über die *gemeinsame Dichte* $f_{X,Y}(x, y)$ benötigen würden. Es ist im Gegenteil bei vielen unserer Beispiele sehr oft der Fall, dass $P(X = Y) = 0$, wenn $X \sim Y$.

Uniforme Verteilung/Gleichverteilung Gegeben zwei reelle Zahlen $a < b$ nennt man eine Zufallsgröße X verteilt nach $\text{Unif}(a, b)$, man schreibt $X \sim \text{Unif}(a, b)$, wenn die Dichte f_X gegeben ist durch

$$f_X(x) = \begin{cases} 0, & \text{falls } x < a, \\ 1/(b-a), & \text{falls } a \leq x < b \\ 0, & \text{falls } b \leq x. \end{cases} = \frac{1}{b-a} \text{rect}\left(\frac{x - (b+a)/2}{b-a}\right)$$

Der Name dieser Verteilung ergibt sich daraus, dass intuitiv alle Intervalle, die im Intervall $[a, b]$ liegen mit einer Wahrscheinlichkeit proportional zu ihrer „Länge“ von X „getroffen“ werden, also $P(X \in [c, d]) = (d - c)/(b - a)$.

Normalverteilung Gegeben zwei Parameter $\mu \in \mathbb{R}$ und $\sigma^2 \in \mathbb{R}^+$, so nennt man eine Zufallsgröße X normalverteilt mit Erwartungswert μ und Varianz σ^2 , falls deren Dichte f_X

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

erfüllt. Die Normalverteilung ist eine sehr wichtige Verteilung, weil – aus Gründen – die meisten Messwerte einer Normalverteilung mit Erwartungswert μ und einer gewissen Varianz σ^2 folgen. Je größer σ^2 , desto stärker ist die Streuung der Werte von X um den Erwartungswert μ . Man schreibt auch $X \sim \mathcal{N}(\mu, \sigma^2)$. Es sind zwei Beispiele in Codeschnipsel 32 zusammen mit der Berechnung der Dichte via `scipy` dargestellt. Auch hier ist es wieder ratsam, keine eigenen Implementierungen solcher Standardfunktionen zu nutzen und stattdessen auf Funktionen aus getesteten Paketen zurück zu greifen.

Exponentialverteilung Gegeben eine Intensität $\lambda > 0$, so folgt eine Zufallsgröße X einer Exponentialverteilung $X \sim \text{Exp}(\lambda)$, falls die Dichte f_X die Form

$$f_X(x) = \begin{cases} \lambda \exp(-\lambda x), & \text{für } x \geq 0, \\ 0 & \text{sonst} \end{cases}$$

besitzt. Auch eine wichtige Verteilung, die beispielsweise die Lebensdauer von irgendwelchen Bauteilen modelliert, oder auch die Wartezeit zwischen zwei Anrufen bei einer Servicehotline. Je kleiner λ , desto höher die Intensität, also desto kürzer die Wartezeit.

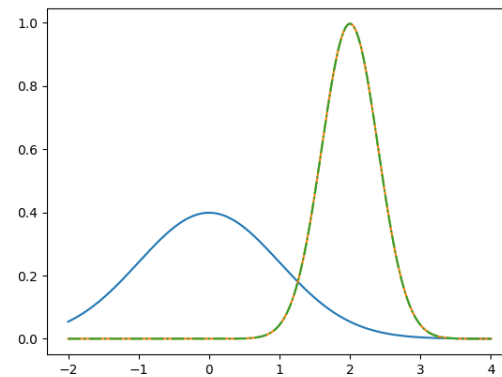
```

from scipy.stats import norm

x = np.linspace(-2, +4, 500)

plt.figure()
plt.plot(x, norm.pdf(x))
plt.plot(x, norm.pdf(x, loc=2, scale=0.4))
plt.plot(x, norm.pdf((x-2)/(0.4))/(0.4), linestyle='--')

```



Codeschnipsel 32: Visualisierung der Dichte der Normalverteilung für verschiedene Parameter, siehe [dsv/code/random/normal1.py](#)

Laplace- und Binomialverteilung Wir stellen uns vor, dass wir ein Zufallsexperiment X durchführen, bei dem Erfolg $X = 1$ mit Wahrscheinlichkeit p auftritt und Misserfolg $X = 0$ mit Wahrscheinlichkeit $(1 - p)$. Demzufolge gibt es nur diese beiden Möglichkeiten, und wir erhalten als Dichte entsprechend

$$f_X(x) = (1 - p)\delta(x) + p\delta(x - 1).$$

Solch eine diskrete Verteilung nennt man Laplaceverteilung mit Parameter p .

Allein ist diese Verteilung so einfach, wie sie nur sein kann. Betrachten wir aber nun n viele solcher *unabhängiger* Laplaceexperimente $X_1, \dots, X_n$, man nennt diese dann **independent and identically distributed (iid)**, und zählen mit X die Anzahl der Erfolge in diesen n Experimenten, also

$$X = \sum_{i=1}^n X_i,$$

so erhalten wir als Dichte

$$f_X(x) = \sum_{k=0}^n \binom{n}{k} p^k (1 - p)^{n-k} \delta(x - k).$$

Diese Verteilung nennt man Binomialverteilung mit Parametern n und p . Die Unabhängigkeit ist hier sehr wichtig, da diese für das Herleiten von f_X aus den einzelnen f_{X_i} hergenommen wird.

6.1.4 Kenngrößen von Zufallsgrößen

Will man eine Zufallsgröße auf einige wenige Kennzahlen herunterbrechen, die in der Praxis auch relevant sein könnten, so bieten sich sogenannte Momente deren Verteilung an.

Erwartungswert Interessiert man sich für den Wert, der sich „im Mittel“ bei einer Verteilung X ergibt, so berechnet man den Erwartungswert $E(X)$, das erste Moment, definiert durch

$$E(X) = \int_{-\infty}^{+\infty} x f_X(x) dx. \quad (6.1.6)$$

Wenn f_X einer physikalischen Dichteverteilung eines Objektes im Raum entspräche, so würde $\mathbf{E}(X)$ genau den Masseschwerpunkt dieser Verteilung darstellen. Ist beispielsweise $X \sim \text{Unif}(a, b)$, so gilt $\mathbf{E}(X) = (a + b)/2$ und falls $X \sim \mathcal{N}(\mu, \sigma^2)$, so gilt $\mathbf{E}(X) = \mu$.

Mit dem Erwartungswert kann man Zufallsgrößen *zentrieren*, indem man für X die Zufallsgröße $\tilde{X} = X - \mathbf{E}(X)$ betrachtet. Dann gilt

$$\mathbf{E}(\tilde{X}) = \mathbf{E}(X) - \mathbf{E}(\mathbf{E}(X)) = \mathbf{E}(X) - \mathbf{E}(X) = 0$$

Man nennt Zufallsgrößen X mit $\mathbf{E}(X)$ *mittelwert-* oder *biasfrei*.

Varianz Basierend auf dem Erwartungswert, können wir auch die mittlere Streuung einer Zufallsgröße um $\mathbf{E}(X)$ herum betrachten, also das zweite zentrierte Moment, die *Varianz*, definiert durch

$$\text{Var}(X) = \mathbf{E}((X - \mathbf{E}(X))^2) = \mathbf{E}(\tilde{X}^2) = \int_{-\infty}^{+\infty} (x - \mathbf{E}(X))^2 f_X(x) dx = \mathbf{E}(X^2) - \mathbf{E}(X)^2. \quad (6.1.7)$$

Die Interpretation ist, dass $\text{Var}(X)$ die mittlere quadratische Abweichung einer Zufallsgröße von ihrem Erwartungswert beschreibt. Es ist anzumerken, dass es Verteilungen gibt, für welche $\mathbf{E}$ und Var nicht existieren, weil die jeweiligen Integrale in (6.1.6) und (6.1.7) nicht endlich sind.

Beispielsweise gilt im Falle von $X \sim \text{Unif}(a, b)$, dass $\text{Var}(X) = (b - a)^2/12$ und im Falle von $X \sim \mathcal{N}(\mu, \sigma^2)$, dass $\text{Var}(X) = \sigma^2$.

Korrelation/Kovarianz Es kommt vor, dass wir auf einfache Weise zwei Zufallsgrößen X und Y mit einander in Zusammenhang bringen wollen. Hierzu kann man deren *Korrelation* $\mathbf{E}(XY)$ betrachten, also den Erwartungswert von deren Produkt $Z = X \cdot Y$. Betrachtet man stattdessen die Korrelation der zentrierten Versionen $\mathbf{E}(\tilde{X}\tilde{Y})$, so spricht man von der *Kovarianz*.

Es ist wieder darauf hinzuweisen, dass man Korrelation und Kovarianz nicht „einfach“ berechnen kann, weil man hierfür wieder die gemeinsame Verteilung der beiden Zufallsgrößen benötigt. Sind aber X und Y unabhängig, dann gilt folgender einfacher Zusammenhang:

$$\begin{aligned} \mathbf{E}(XY) &= \int \int xy f_{X,Y}(x, y) dy dx = \int \int xy f_X(x) f_Y(y) dy dx = \int x f_X(x) \int y f_Y(y) dy dx \\ &= \mathbf{E}(Y) \int x f_X(x) dx = \mathbf{E}(X) \mathbf{E}(Y). \end{aligned}$$

Für unabhängige Zufallsgrößen ist also der Erwartungswert des Produktes das Produkt der einzelnen Erwartungswerte. Eine sehr nützliche Eigenschaft, wie wir bald sehen werden!

Charakteristische Funktion Gegeben eine Zufallsgröße X mit Dichte f_X , ist die zugehörige *charakteristische Funktion* $\phi_X : \mathbb{R} \rightarrow \mathbb{C}$ definiert durch

$$\phi_X(f) = \int_{-\infty}^{+\infty} f_X(x) \exp(-j2\pi f x) dx. \quad (6.1.8)$$

Damit ist ϕ_X die Fourier-Transformierte von f_X . Wir werden bald den Nutzen von dieser Funktion sehen, aber es sei beispielhaft erwähnt, dass

$$\left(\frac{d}{df}\right)^k \phi_X(0) = j^k \mathbf{E}(X^k),$$

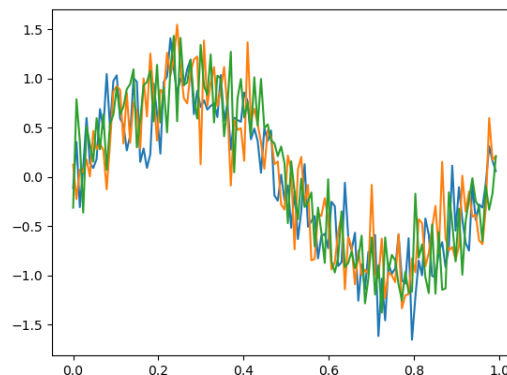
also die Ableitungen von ϕ_X mit den Momenten von X in Zusammenhang stehen.

6.2 Zufällige Signale

Die bisherigen Konzepte reichen jedoch nicht aus, um die zeitliche Komponente von Zufallsprozessen richtig modellieren zu können, da wir bis jetzt nur einzelne Zufallsgrößen betrachtet haben, die nicht in einem gemeinsamen (beispielsweise zeitlichen) Kontext gebracht wurden. Wenn wir von zufallsbehafteten Signalen, beispielsweise im Zeitbereich, sprechen, so müssen wir uns vorstellen, dass an jedem Zeitpunkt t eine Zufallsgröße X_t „angeheftet“ ist, die sich dann in ihrer Gesamtheit zu einer *zufälligen* Funktion $f: \mathbb{R} \rightarrow \mathbb{C}$ realisieren, indem wir $f(t) = X_t(\omega)$ setzen. Die Funktion f ist also die punktweise Realisierung der verschiedenen Zufallsprozesse X_t , siehe Codeschnipsel 33. Man nennt dann die Folge von Zufallsgrößen $(X_t)_{t \in \mathbb{R}}$ einen *stochastischen Prozess*, oder einfach *zufälliges/stochastisches Signal*. Eine spezielle Realisierung zu einer zufälligen Funktion f nennen wir auch *Pfad*.

Daraus ergeben sich nun zwei Sichtweisen auf stochastische Signale. Wir können von einer Folge von Zufallsgrößen im Zeitbereich sprechen, und deren gemeinsame Verteilung betrachten. Alternativ können wir auch direkt auf den Pfaden f eine andere, aber darauf bezogene, Verteilung im Raum der Funktionen betrachten. Im ersten Fall betrachten wir die Verteilung zu einzelnen Zeitpunkten, aber gemeinsam für alle Punkte. Im zweiten betrachten wir das Verhalten als *Ganzes*. Beide Sichtweisen sind in Codeschnipsel 34 zusammengefasst.

```
def X(t):
    return np.sin(
        2 * np.pi * t
    ) + 0.3 * np.random.randn(t.size)
```



Codeschnipsel 33: Drei Realisierungen von $\sin(2\pi t) + 3/10 X_t$, wobei alle $X_t \sim \mathcal{N}(0, 1)$ unabhängig identisch verteilt sind (*iid*), siehe [dsv/code/random/signal1.py](#)

Wie bereits in den vorherigen Abschnitten, wollen wir darauf hinweisen, dass man solche stochastischen Signale auch ohne viel Aufwand im Raum definieren kann. Man kann also auch Bild-Signale der Form $(X_{x,y})_{x,y \in \mathbb{R}}$ als räumlichen Prozess auffassen/modellieren.

```

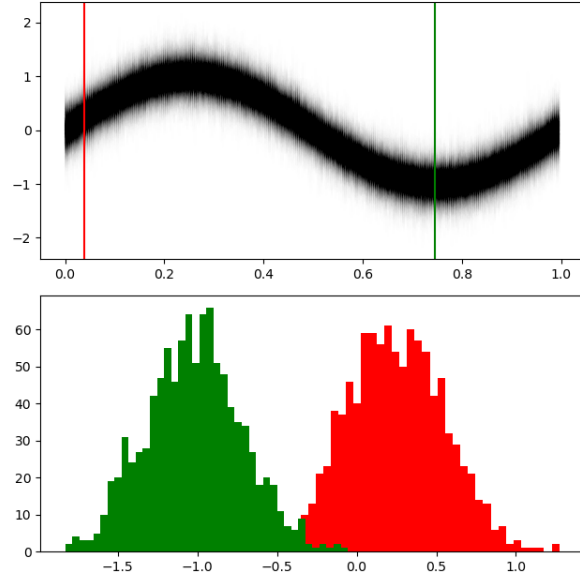
def X(t):
    return np.sin(
        2 * np.pi * t
    ) + 0.3 * np.random.randn(t.size)

N = 1000
T = np.linspace(0, 1, 255, endpoint=False)

X_t = [X(T) for nn in range(N)]

n1, n2 = 10, 190
t1, t2 = T[n1], T[n2]
X_t1 = [X_t[nn][n1] for nn in range(N)]
X_t2 = [X_t[nn][n2] for nn in range(N)]

```



Codeschnipsel 34: Visualisierung der beiden Sichtweisen auf stochastische Signale., siehe [dsv/code/random/signal2.py](#)

Es gibt zu stochastischen Signalen eine Menge zu sagen und man kann viel Theorie dazu konsumieren und betreiben. Wir wollen uns jedoch direkt auf eine kleine Klasse von stochastischen Signalen beschränken, die wir im Digitalen antreffen.

6.3 Diskrete Stationäre Signale

Nehmen wir an, dass wir eine diskrete Folge von Zufallsgrößen $(X_n)_{n \in \mathbb{N}}$, also ein diskretes stochastisches Signal gegeben haben. Dann gibt es für diese Werte eine gemeinsame Dichte $f_{X_1, X_2, \dots}(x_1, x_2, \dots)$. Wenn wir nun diese Samples um ein $\tau \in \mathbb{N}$ verschieben, also $(X_{n+\tau})_{n \in \mathbb{N}}$, und

$$f_{X_1, X_2, \dots}(x_1, x_2, \dots) = f_{X_{1+\tau}, X_{2+\tau}, \dots}(x_1, x_2, \dots)$$

für alle Stellen $n \in \mathbb{N}$ und $\tau \in \mathbb{N}$ gilt, dann nennt man den Prozess X_n *stark stationär*. Dies führt dazu, dass sowohl Erwartungswert als auch Varianz *konstant* sind, also $\mathbb{E}(X_n) = \mathbb{E}(X_0) = c$. Umgekehrt muss der Prozess nicht stark stationär sein, wenn die Momente zeitlich konstant sind. Hat ein Prozess konstante Momente, so nennt man ihn *schwach stationär*.

Wir definieren die **ACF** $\gamma_{X,X} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{R}$ als

$$\gamma_{X,X}[n_1, n_2] = \mathbb{E}(X_{n_1} X_{n_2}^*). \quad (6.3.1)$$

Somit stellt γ die Korrelation des Prozesses X_n zwischen verschiedenen Zeitpunkten dar. Ist nun X_n stark stationär, dann unterscheidet sich die Verteilung von X_n nicht von der von $X_{n+\tau}$. Also gilt wegen

$$\gamma_{X,X}[n_1, n_2] = \gamma_{X,X}[n_1 + \tau, n_2 + \tau] = \gamma_{X,X}[n_1 - n_2, 0] = \gamma_{X,X}[n_1 - n_2], \quad (6.3.2)$$

dass die **ACF** nur von $n_1 - n_2$ abhängig, also dem Abstand der Abtastzeitpunkte, ist. Man nennt solche Funktionen *translationsinvariant*. Da wir in (6.3.2) auch in die andere Richtung hätten argumentieren können, gilt $\gamma_{X,X}[\tau] = \gamma_{X,X}[-\tau]$, was γ zu einer geraden Funktion macht.

Es ist nun essentiell zu sehen, dass wir mit γ aus einem stochastischen Objekt, also dem stationären Prozess X_n die *deterministische*, also nicht-zufällige, Funktion γ gewonnen haben, die uns gleichzeitig etwas über das zeitliche Verhalten des Prozesses, also des Signals, mitteilen kann. Für stationäre stochastische Signale bildet γ also eine Brücke zwischen der zufälligen und stochastischen Welt.

Wir sehen das noch direkter an folgendem

Theorem 6.1 (Rainbows'n'Sunshine Wiener-Chintschin). *Für einen stationären Prozess X_n mit ACF $\gamma_{X,X}$ existiert eine Funktion $S : [-1/2, +1/2] \rightarrow \mathbb{C}$ sodass,*

$$\gamma_{X,X}(n) = \int_{-1/2}^{+1/2} S(f) \exp(j2\pi fn) df \quad (6.3.3)$$

gilt. Man nennt dann S die spektrale Leistungsdichte von X_n .

Wir zeigen in Codeschnipsel 35, wie man aus mehreren Realisierungen eines stationären Signals

$$x[n] = X_n = \sin(2\pi 0.75n\Delta t + \phi_1) + \sin(2\pi 3n\Delta t + \phi_2) + z_n,$$

wobei $z_n \sim \mathcal{N}(0, 0.6^2)$ und $\phi_{1,2} \sim \text{Unif}(0, 2\pi)$ iid, die spektralen Komponenten extrahieren kann. Von diesem zufälligen Signal haben wir in dem oberen Plot von Codeschnipsel 35 verschiedene Realisierungen dargestellt. Im mittleren Plot haben wir Histogramme von X_{10} und von X_{50} geplottet, um eine Möglichkeit zu zeigen, wie man zumindest *visuel* prüfen kann, ob das Signal $x[n]$ stationär ist. Wir sehen, dass die Histogramme in etwa übereinstimmen, also können wir von Stationarität ausgehen, da ähnliche Histogramme an verschiedenen Zeitpunkten auf eine gleiche Dichte an diesen Zeitpunkten schließen lassen. Beispielsweise wäre das Signal aus Codeschnipsel 34 *nicht* stationär.

Im unteren Plot (vorletzter Plot) zeigen wir eine Schätzung von $\gamma_{X,X}$ basierend auf mehreren Realisierungen des Signals $x[\cdot]$, indem wir einfach für jedes einzelne $x[\cdot]$ die Faltung $x * x[-\cdot]$ berechnet haben und dann über *diese* Realisierungen der ACF gemittelt haben, um den Erwartungswert in (6.3.1) zu approximieren. Man sieht sehr gut, dass die (**zufällige**) Approximation von $\gamma_{X,X}$ sich sehr stabil verhält, auch wenn wir N in Codeschnipsel 35 viel kleiner werden lassen. Weiterhin sehen wir im Plot ganz unten den Effekt von Theorem 6.1, weil die DFT von $\gamma_{X,X}$ an den Stellen $F = \pm 3$ und $F = \pm 0.75$ lokale Maxima besitzt, was genau den spektralen Komponenten des *zufälligen* Signals X_n entspricht.

Mit der Berechnung der ACF kann man also aus einigen wenigen Realisierungen eines stochastischen Signals dessen spektrale Komponenten extrahieren. Diese Herangehensweise ist durch das Wiener-Chintschin-Theorem 6.1 theoretisch untermauert.

```

def X(t):
    return (
        0.2
        * np.sin(
            2 * np.pi * 0.75 * t
            + np.random.uniform(0, 2 * np.pi, 1)
        )
        + np.sin(
            2 * np.pi * 3 * t
            + np.random.uniform(0, 2 * np.pi, 1)
        )
        + 0.6 * np.random.randn(t.size)
    )

N = 1000
F_s = 16 # sampling freq
N_s = 64 # sample duration
T = np.linspace(
    0, N_s / F_s, N_s, endpoint=False
)

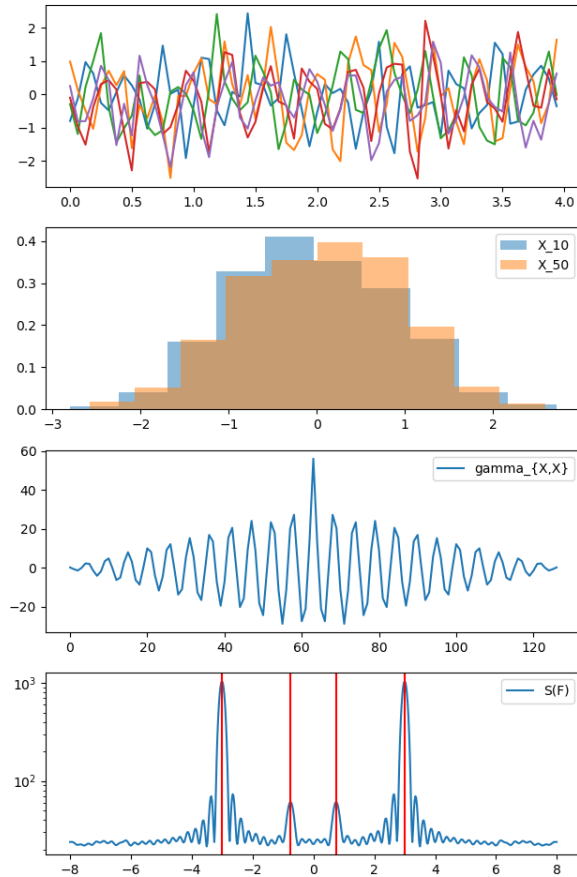
X15 = [X(T) for nn in range(5)]

XN = np.array([X(T) for nn in range(N)])

acf = np.mean(
    [
        fftconvolve(XN[nn, :], XN[nn, ::-1])
        for nn in range(N)
    ],
    axis=0,
)

psd = np.fft.fft(acf, n=5 * acf.size)

```



Codeschnipsel 35: Extraktion der spektralen Komponenten eines stationären Signals, siehe [dsv/code/random/signal3.py](#)

6.4 Quantisierung

Wir wollen die entwickelte Theorie zu stochastischen Signalen nun auf ein Problem anwenden, das wir bis jetzt gutmütig ingoriert haben: Quantisierung. Bisher sind wir von einem zeitdiskreten, aber nicht wert-diskreten Signal ausgegangen. Dies wollen wir nun noch einmal kurz umreißen, indem wir den Effekt von Quantisierung auf zufällige stationäre Signale betrachten. Hierzu folgen wir im Grunde den jeweiligen Abschnitten in [10].

Wir gehen nun davon aus, dass wir ein stationäres Signal X_n quantisieren, und dadurch $Q_n = q(X_n)$ erhalten, wobei q die Quantisierungsoperation darstellt. Zunächst wollen wir ein Gefühl für die Wirkung von Quantisierung bekommen und betrachten hierzu den Effekt vom sogenannten „Area Sampling“, siehe Abbildung 12.

Dies ist in Codeschnipsel 36 durchgeführt, wobei wir von 3 Quantisierungsstufen ausgegangen sind,

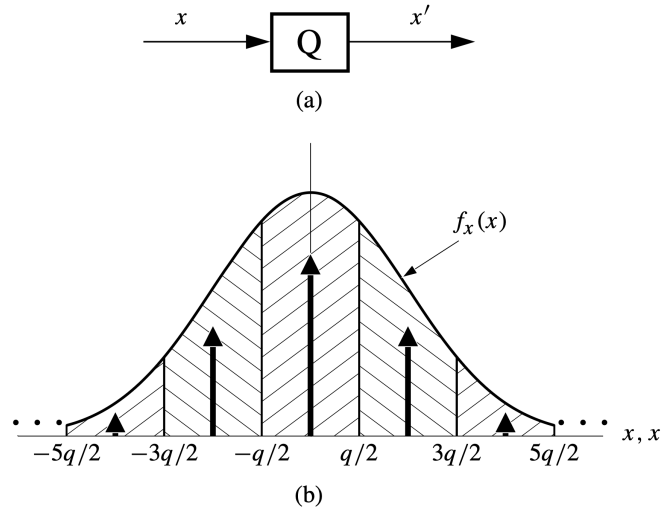


Abbildung 12: Visualisierung von Area Sampling, Quelle: [10]

und deren Werte so gelegt haben, dass sie zum Signal passen, das wie in Codeschnipsel 35 generiert wird. Außerdem sind zusätzlich die Bereiche (edges) markiert, die gleiche Quantisierungswerte ergeben, sowie die einzelnen Quantisierungsstufen (centers). Man sieht, dass diese Art der Quantisierung sehr leicht durch geschicktes Runden implementiert werden kann.

Ein Grund, weshalb wir uns auf die Verteilung, oder Dichte f_X , eines stationären Signals konzentrieren ist der Fakt, dass zwar einerseits Quantisierung eines Signals nicht-linear auf das eine einzelne Signalrealisierung wirkt, doch wir sehen gleich, dass Area Sampling eine lineare Operation auf der Dichte f_X darstellt.

Geometrisch wird aus Abbildung 12 klar, dass Area Sampling im Grunde alle Werte in einem Bereich, einer Fläche (daher der Name) unter der Dichte des nicht-quantisierten Signals, auf einen Punkt zusammenzieht. Die Verteilung der nicht-quantisierten Werte ist also auch eine stetige Verteilung (wenn wir von einem ursprünglich analogen Signal ausgehen), während die Verteilung des quantisierten Signals diskret sein *muss* – schließlich ist ja genau das unser Ziel bei der Quantisierung.

Betrachten wir also nun einmal die Dichte $f_{q(X)}$ eines mittels Area Samplings mit $\Delta Q > 0$ als Abstand zwischen den Quantisierungsstufen quantisierten Signals X analytisch via

$$f_{q(X)}(x) = \sum_{k \in \mathbb{Z}} \delta(x - k\Delta Q) \int_{k\Delta Q - \Delta Q/2}^{k\Delta Q + \Delta Q/2} f_X(x) dx.$$

Definieren wir nun die Funktion $\text{rect}_{\Delta Q} : \mathbb{R} \rightarrow \mathbb{R}$ via

$$\text{rect}_{\Delta Q}(x) = \begin{cases} 1 & \text{für } x \in [-\Delta Q/2, +\Delta Q/2], \\ 0 & \text{sonst} \end{cases},$$

so ergibt die Faltung von $\text{rect}_{\Delta Q}$ mit einer Funktion f , dass

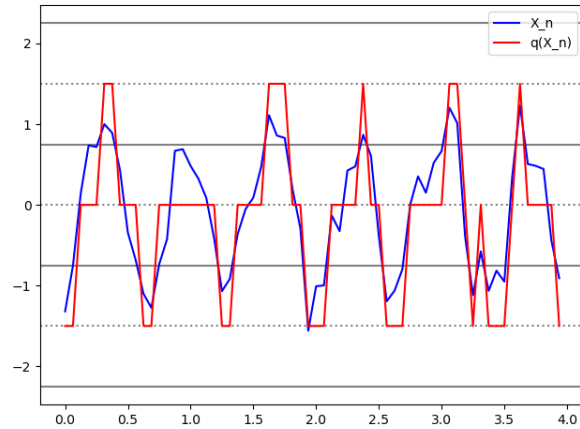
$$[\text{rect}_{\Delta Q} * f](x) = \int_{x - \Delta Q/2}^{x + \Delta Q/2} f(t) dt = F_X(x + \Delta Q/2) - F_X(x - \Delta Q/2). \quad (6.4.1)$$

```
def quantize(x, deltaQ, levels):
    q = np.round(x / deltaQ) * deltaQ
    q[q < (-levels - 0.5) * deltaQ] = (
        -levels * deltaQ
    )
    q[q > (levels + 0.5) * deltaQ] = (
        levels * deltaQ
    )

    centers = np.linspace(
        -levels * deltaQ,
        +levels * deltaQ,
        2 * levels + 1,
        endpoint=True,
    )

    edges = np.concatenate(
        [
            centers - deltaQ / 2,
            [centers[-1] + deltaQ / 2],
        ]
    )

    return q, centers, edges
```



Codeschnipsel 36: Quantisierung via Area Sampling, siehe [dsv/code/random/quantization1.py](#)

Somit müssen wir nur noch die Faltung von $\text{rect}_{\Delta Q}$ und f_X an der richtigen Stelle auswerten, um das Area Sampling analytisch zu reproduzieren. Diese Auswertung kann wegen Gleichung (6.1.3) durch Verwenden der Verteilungsfunktion F_X erfolgen.

Definieren wir für die Auswertung einen entsprechenden Dirac-Kamm in Abhängigkeit von ΔQ via

$$c(x) = \sum_{k \in \mathbb{Z}} \delta(x - k\Delta Q),$$

dann ergibt sich

$$f_{q(X)} = (\text{rect}_{\Delta Q} * f) \cdot c. \quad (6.4.2)$$

Damit haben wir gezeigt, dass sich die diskrete Dichte des quantisierten Signals bei Area Sampling als Folge von Punktauswertungen der Faltung der Dichte des nicht-quantisierten Signals mit $\text{rect}_{\Delta Q}$ ergibt. Dies ist offensichtlich eine lineare Transformation der Dichte, da alle involvierten Operationen, also Faltung mit $\text{rect}_{\Delta Q}$ und Multiplikation mit c , linear in f_X sind. Dieser Zusammenhang wird in Codeschnipsel 37 beispielhaft für die Signale aus Codeschnipsel 35 durchgeführt.

Wir sehen in Codeschnipsel 37 in `empirical_pdf`, wie man aus N Realisierungen eine Schätzung für die Dichte f_X bekommen kann. Dies nennt man entsprechend *empirische Dichte*, da sie nicht analytisch, sondern empirisch durch Mittelung, entstanden ist. Wir berechnen erst die *empirische Verteilung* als Schätzung von F_X aus Gleichung (6.1.1) und bilden dann numerische Ableitungen, wie in Gleichung (6.1.2) gefordert und erhalten somit xN_f . Dann vergleichen wir die beiden Möglichkeiten für die Berechnung der Dichte der quantisierten Werte $f_{q(X)}$, indem wir erst quantisieren und dann das Histogramm an den Quantisierungsstufen bilden,

um $x_{N_f,q}$ zu erhalten, oder eben die entsprechende Faltung aus Gleichung (6.4.2) auswerten. Schließlich erhalten wir $x_{N_f,q,as}$ durch direktes Implementieren von Gleichung (6.4.2) mittels `density_area_sampling`. Wie man im unteren Plot von Codeschnipsel 37 erkennen kann, stimmen die Ergebnisse der beiden Implementierungen sehr gut überein. Kleinste Abweichungen kommen von der Tatsache, dass die empirische Dichte verwendet wurde und dass wir auch für die alternative Implementierung des Area Samplings eine empirische Version der Verteilung F_X verwendet haben.

```
def empirical_pdf(x, edges):
    density_eval = np.linspace(
        edges[0], edges[-1]
    )
    # Gleichung (6.1.1)
    x_ecdf = ecdf(XN.flatten()).cdf

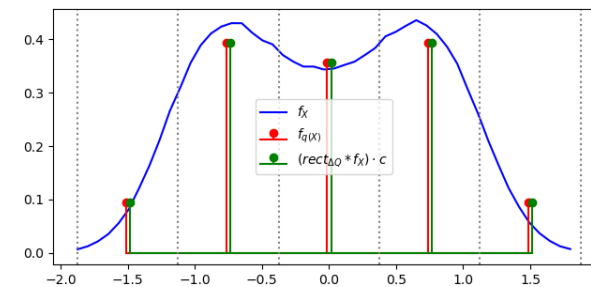
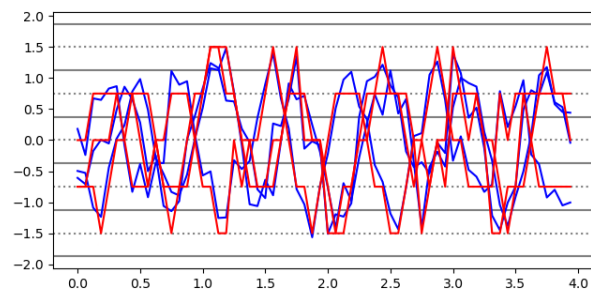
    # Gleichung (6.1.2)
    x_epdf = (
        np.diff(x_ecdf.evaluate(density_eval))
        / np.diff(density_eval)[0]
    )

    return x_epdf, density_eval
```

```
# Gleichung (6.4.2)
def density_area_sampling(t, x, deltaQ):
    x_ecdf = ecdf(x.flatten()).cdf

    # Gleichung (6.4.1)
    return (
        x_ecdf.evaluate(t + deltaQ / 2)
        - x_ecdf.evaluate(t - deltaQ / 2)
    ) / deltaQ
```

```
X15_q, centers, edges = quantize(
    X15, deltaQ, numLevels
)
XN_f, density_eval = empirical_pdf(
    XN.flatten(), edges
)
XN_q, _, _ = quantize(XN, deltaQ, numLevels)
XN_f_q, _ = np.histogram(
    XN_q, edges, density=True
)
XN_f_q_as = [
    density_area_sampling(
        cc, XN.flatten(), deltaQ
    )
    for cc in centers
]
```



Codeschnipsel 37: Quantisierung via Area Sampling, siehe [dsv/code/random/quantization2.py](#)

Die Frage, die wir uns nun stellen wollen ist: Unter welchen Bedingungen kann man aus der diskreten Dichte $f_{q(X)}$ der quantisierten Werte die ursprüngliche Dichte f_X des Nicht-quantisierten Signals rekonstruieren? Somit erhalten wir zwar nicht die ursprünglichen Werte, aber immerhin deren Statistik.

Hierzu betrachten wir den Zusammenhang aus Gleichung (6.4.2) im Frequenzbereich. In (6.1.8) haben wir die charakteristische Funktion ϕ_X als Fourier-Transformierte der Dichte f_X eingeführt. Definieren wir $F_Q = 1/\Delta Q$, dass ergibt sich aus den Eigenschaften der FT direkt, dass

$$\phi_{q(X)}(F) = \sum_{k \in \mathbb{Z}} (\phi_X \cdot \text{sinc}_{\Delta Q})(F - kF_Q) = \sum_{k \in \mathbb{Z}} (\phi_X(F - kF_Q) \cdot \text{sinc}_{\Delta Q}(F - kF_Q)), \quad (6.4.3)$$

wobei wir

$$\text{sinc}_{\Delta Q}(F) = \text{sinc}\left(\frac{\Delta Q F}{2}\right)$$

definiert haben.

Wir sehen hier schön die Eigenschaften, die wir aus Abschnitt 2 bereits kennen, am Werk. Da die Dichte der quantisierten Werte aus Faltung und Abtastung mit Rate $F_Q = 1/\Delta Q$ hervorgeht, ist deren Spektrum periodisch mit Periodenlänge ΔQ . Weiterhin muss vor Periodifizierung die charakteristische Funktion der nicht-quantisierten Werte ϕ_X mit der Fourier-Transformierten von $\text{rect}_{\Delta Q}$ multipliziert werden. Dies ist in Abbildung 13 sehr schön aufbereitet.

Wir kommen nun zum Abschluss, da wir basierend auf (6.4.3) ein Kriterium angeben können, sodass wir f_X aus $f_{q(X)}$ rekonstruieren können. Dies ist in der Praxis natürlich von großem Interesse, da wir für gewöhnlich nur $f_{q(X)}$ ermitteln können, aber eigentlich Informationen über f_X aus unseren Beobachtungen ziehen wollen.

Wie wir gesehen haben, hängt die Periodifizierung im Frequenzbereich in (6.4.3) von dem Abstand der Quantisierungsstufen ab. Wählen wir also ΔQ so klein, dass $\phi_X(F) = 0$ für alle $|F| > F_Q$, dann tritt in Gleichung (6.4.3) kein Aliasing auf. Das heißt, dass sich die Kriterien für hinreichend feines Quantisieren *direkt* aus der Anwendung des Samplingtheorems 2.1 auf f_X beziehungsweise $f_{q(X)}$ ergeben! Man kann sich in diesem Sinne Quantisierung als „Abtastung“ der Dichte f_X (gefaltet mit rect) vorstellen. Deshalb ergibt sich auch, dass die Quantisierungsstufen so nah beieinander liegen müssen, dass diese Dichte kritisch abgetastet wurde.

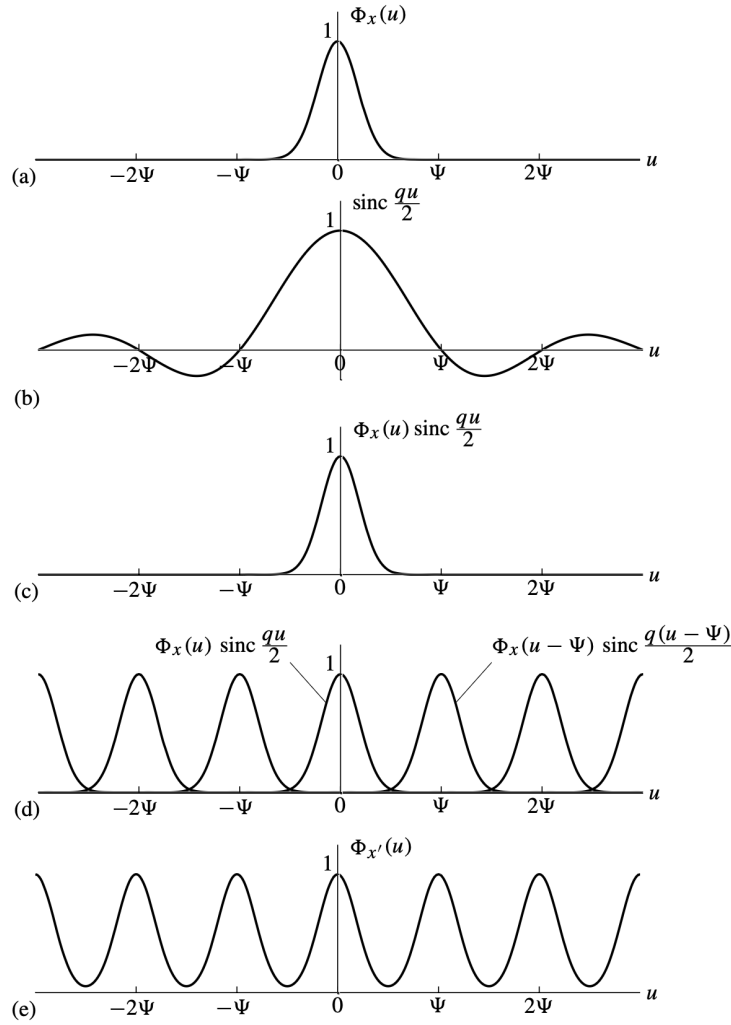


Abbildung 13: Visualisierung des Effekts von Area Sampling im Frequenzbereich. Quelle: [10]

Theorem 6.2 (Widrow-Quantisierungs-Theorem). *Ist die Dichte der nicht-quantisierten Werte f_x bandbegrenzt, also gilt*

$$\phi_X(F) = 0 \quad \text{für} \quad |F| > \frac{1}{\Delta Q},$$

dann lässt sich die Dichte f_X schreiben als

$$f_X(x) = \sum_{k \in \mathbb{Z}} \sum_{\ell=-\infty}^k f_{q(X)}(\ell \Delta Q) \frac{d}{dx} \text{sinc} \left(\frac{x - (k + 1/2) \Delta Q}{\Delta Q} \right)$$

Beweis. Wir führen den Beweis der Vollständigkeit halber, da in [10] auf <http://www.mit.be.hu/book/s/quantization> verwiesen wird. Anyways.

Die Funktion $\text{rect}_{\Delta Q} * f_X$ ist genau dann bandbegrenzt, wenn f_X bandbegrenzt ist, da $\text{rect}_{\Delta Q}$ nicht bandbegrenzt ist und die beiden Spektren sich wegen der Faltung im Frequenzbereich multiplizieren. Die Formel für die Interpolation ergibt sich aus der Anwendung von Theorem 2.1 auf die Funktion $\text{rect}_{\Delta Q} * f_X$,

von der wir aus Gleichung (6.4.2) wissen, dass

$$[\text{rect}_{\Delta Q} * f_X](x) = \sum_{k \in \mathbb{Z}} f_{q(X)}(k\Delta Q) \text{sinc}\left(\frac{x - k\Delta Q}{\Delta Q}\right).$$

Dann wissen wir aus Gleichung (6.4.1), dass

$$[\text{rect}_{\Delta Q} * f_X](x) = F_X(x + \Delta Q/2) - F_X(x - \Delta Q/2),$$

was nach Differentiation beider Seiten

$$f_X(x + \Delta Q/2) - f_X(x - \Delta Q/2) = \sum_{k \in \mathbb{Z}} f_{q(X)}(k\Delta Q) \frac{d}{dx} \text{sinc}\left(\frac{x - k\Delta Q}{\Delta Q}\right) = g(x)$$

impliziert. Betrachten wir $g(x) + g(x - \Delta Q)$ so ergibt sich

$$\begin{aligned} g(x) + g(x - \Delta Q) &= f_X(x + \Delta Q/2) - f_X(x - \Delta Q/2) + f_X(x + \Delta Q/2 - \Delta Q) - f_X(x - \Delta Q/2 - \Delta Q) \\ &= f_X(x + \Delta Q/2) - f_X(x - \Delta Q/2 - \Delta Q) \end{aligned}$$

Also folgern wir induktiv, dass

$$\sum_{\ell=0}^{\infty} g(x - \ell\Delta Q) = f_X(x + \Delta Q/2) + \lim_{\ell \rightarrow \infty} f_X(x - \ell\Delta Q),$$

Da Dichten endliches Integral besitzen und die Dichten, die wir betrachten gutartig genug sind, gilt

$$\lim_{\ell \rightarrow \infty} f_X(x - \ell\Delta Q) = 0,$$

sodass wir mit ein bisschen Umindizieren von $p = k - \ell$ und Vertauschung von Summen nachrechnen, dass

$$\begin{aligned} f_X(x + \Delta Q/2) &= \sum_{\ell=-\infty}^0 \sum_{k \in \mathbb{Z}} f_{q(X)}(k\Delta Q) \frac{d}{dx} \text{sinc}\left(\frac{x - (k - \ell)\Delta Q}{\Delta Q}\right) \\ &= \sum_{p \in \mathbb{Z}} \left(\sum_{\ell=-\infty}^p f_{q(X)}(\ell\Delta Q) \right) \frac{d}{dx} \text{sinc}\left(\frac{x - p\Delta Q}{\Delta Q}\right) \end{aligned}$$

□

Wir können nun mit Theorem 6.2 aus der Dichte der quantisierten Werte $f_{q(X)}$ die Dichte der nicht-quantisierten Werte f_X bestimmen. Dies ist einmal basierend auf dem vorherigen Beispiel aus Codeschnipsel 37 in Codeschnipsel 38 durchgeführt.

Zusammenfassend ist also zu sagen, dass Quantisierung von zufällig modellierten stationären Signalen mit Dichte f_X einer Abtastung dieser in Abhängigkeit von der Granularität jener gesehen werden kann. Demzufolge konnten wir schlussfolgern, dass bei Quantisierung darauf geachtet werden muss, dass keine Information der Dichte verloren geht, sodass sich die Dichte des analogen Signals aus der Dichte der quantisierten Werte rekonstruieren lässt. Somit haben wir eine Anwendung des Samplingtheorems 2.1 gefunden, die nicht direkt mit Abtastung im Raum oder Zeit zu tun hat, sondern mit Abtastung im Wertebereich eines analogen Signals! Damit haben wir eine wunderschöne Brücke zwischen der Theorie der zufälligen Signale und der Samplingtheorie aus Abschnitt 2 gefunden – ein toller Abschluss!

```

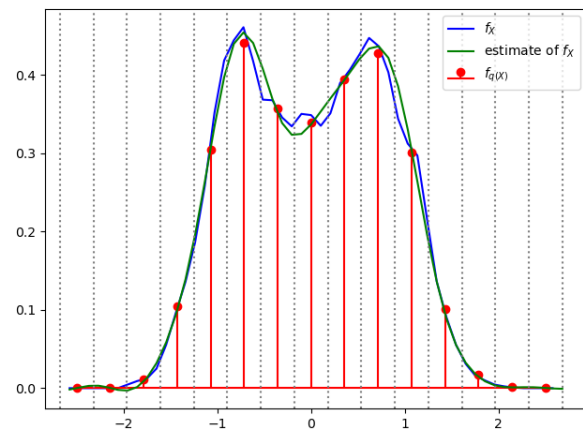
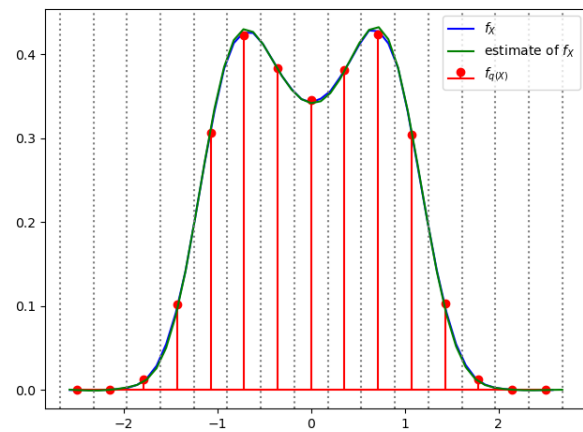
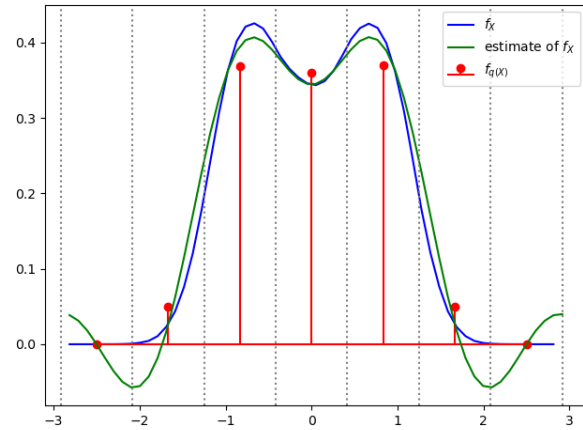
def sinc_deriv(x):
    return np.where(
        np.isclose(x, 0),
        np.zeros_like(x),
        ((np.cos(np.pi * x) - sinc(x)) / x),
    )

def invert_quant(X_f_q, evalGrid, centers):
    sinc_shift = np.diff(centers)[0]
    X_f_q_sum = np.cumsum(X_f_q)

    # periodifizierung vermeided ringing
    padded_centers = lin_extrplt(
        centers, (0, X_f_q_sum.size)
    )
    X_f_q_sum = np.concatenate(
        [X_f_q_sum, X_f_q_sum[::-1]]
    )

    return sinc_deriv(
        np.add.outer(
            evalGrid,
            -padded_centers - sinc_shift / 2,
        )
        / sinc_shift
    ).dot(X_f_q_sum)

```



Codeschnipsel 38: Schätzung der Dichte f_X aus $f_{q(X)}$, die aus Area Sampling gewonnen wurde, siehe [dsv/code/random/quantization3.py](#)

7 Signalverarbeitung mit B-Splines

Eine Grundvoraussetzung für eine praktisch nützliche digitale Signalverarbeitung ist die Möglichkeit zwischen dem analogen und digitalen Bereich wechseln zu können. Hierbei sollte man auch genau quantifizieren können, ob bei diesem Prozess Informationen verloren gehen, oder wie man garantieren kann, dass diese Umwandlung verlustfrei vonstatten geht. Meist nutzt man hierfür das Sampling Theorem 2.1. Die hieraus resultierende sogenannte Nyquist-Sampling-Theorie fußt bekanntermaßen auf der Repräsentation von bandbegrenzten Signalen durch hinreichend dichte äquidistante Abtastwerte.

Es gibt jedoch auch einige Nachteile von Nyquist-Sampling, die aus dessen Annahmen und der daraus folgenden Verarbeitung entstehen. Einerseits kann ein endliches Signal im Allgemeinen *nicht* bandbegrenzt sein. Weiterhin entstehen durch die Bandlimitierung von Signalen Gibbs-Artefakte, siehe Codeschnipsel 19, die besonders bei der Bilderverarbeitung nicht erwünscht sind. Geht es um die Auswertung $x(t)$ eines Signals x zwischen den aufgenommenen Samples $x[n]$, also um Interpolation, hat man das Problem, dass die sinc-Funktion nur sehr langsam mit Rate $1/t$ abfällt. Diese Eigenschaft führt dazu, dass man für die Bestimmung eines Wertes $x(t)$ mit einer Genauigkeit von 1 % etwa 100 um t benachbarte Samples betrachten muss. Das heißt, vor allem bei 2D-Interpolation, siehe Abschnitt 5.2, skaliert der resultierende Rechenaufwand nicht sehr günstig, falls hohe Genauigkeit benötigt wird.

Aus diesem Grund möchten wir uns eine alternative Sampling-Theorie genauer ansehen – die B-Splines [11]. Wir führen zunächst die auf Polynomen basierende Signalverarbeitung ein und vergleichen sie anschließend zur bereits bekannten Nyquist-Theorie.

7.1 B-Splines als Polynome

Allgemein bezeichnet man stückweise definierte und stetig differenzierbare Polynome als Splines. Man nennt die Stellen an denen zwei unterschiedliche Polynome zusammenstoßen als Knoten. Ein Spline der Ordnung $\ell \in \mathbb{N}$ ist ein Polynom vom Grad ℓ , ist also von der Form

$$p(t) = a_\ell t^\ell + a_{\ell-1} t^{\ell-1} + \dots + a_1 t + a_0. \quad (7.1.1)$$

Ein Spline ist nun eine Funktion $s(t)$, welche für Knoten $n = 1, 2, \dots$ definiert ist durch

$$s(t) = \begin{cases} p_1(t) & \text{für } x \in [1, 2], \\ p_2(t) & \text{für } x \in [2, 3], \\ \vdots & \end{cases} \quad (7.1.2)$$

wobei sich die Glattheit durch die Forderung ergibt, dass die Funktion und ihre Ableitungen an den Knoten stetig sei, also

$$\lim_{t \rightarrow n^-} s^{(m)}(t) = \lim_{t \rightarrow n^+} s^{(m)}(t) \quad (7.1.3)$$

erfüllt ist, wobei $s^{(m)}$ für $m \geq 0$ die m -te Ableitung des Splines s repräsentiert. In einer Arbeit [12], die sogar dem berühmten Paper von Shannon vorausgeht, beschreibt Schoenberg, dass sich diese Splines der Ordnung ℓ via

$$s(t) = \sum_{k \in \mathbb{Z}} c[k] \beta^\ell(t - k) \quad (7.1.4)$$

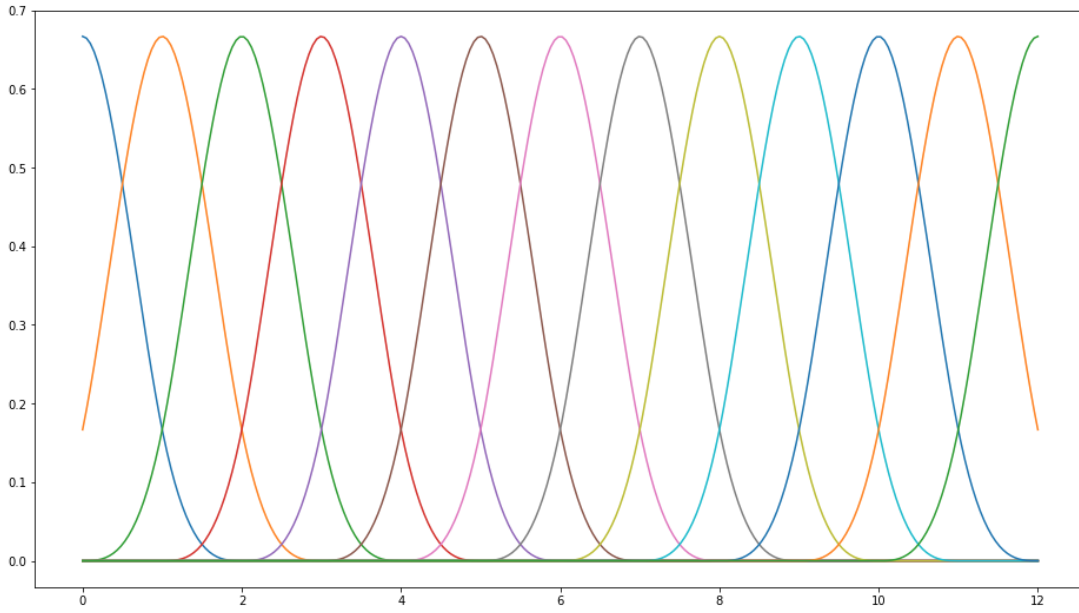


Abbildung 14: Kubische B-Splines für Abtastung an den Werten $n = 0, \dots, 12$.

darstellen lassen. Hierbei ist die Funktion $\beta^\ell : \mathbb{R} \rightarrow \mathbb{R}$ definiert als eine iterierte Faltung einer Rechteckfunktion via

$$\beta^\ell = \underbrace{\beta^0 * \dots * \beta^0}_{(\ell+1) \text{ mal}}, \quad \text{wobei} \quad \beta^0(t) = \begin{cases} 1, & |t| < \frac{1}{2} \\ \frac{1}{2}, & |t| = \frac{1}{2} \\ 0, & \text{sonst.} \end{cases} \quad (7.1.5)$$

In Abbildung 14 sind die Funktionen β^ℓ für $\ell = 0, \dots, 3$ dargestellt. Man erkennt sehr gut, dass der Grad der Glattheit von der Ordnung des Splines abhängt und dass die Funktionswerte $\beta^\ell(t)$ für $|t| > \ell + 1/2$ verschwinden. Man spricht von Funktionen mit *kompaktem Träger*. Das heißt die Summation in (7.1.4) ist für fixes $t \in \mathbb{R}$ endlich und ist auf $\ell + 1$ Summanden beschränkt! Wir wollen nun eine explizite Formel für β^ℓ entwickeln. Hierzu betrachten wir die Fourier-Transformation

$$B^\ell(\omega) = \left(\frac{\sin(\omega/2)}{(\omega/2)} \right)^{\ell+1} = \frac{(\exp(j\omega/2) - \exp(-j\omega/2))^{\ell+1}}{(j\omega)^{\ell+1}} \quad (7.1.6)$$

mit einigen Rechentricks (siehe [11, Box 1.]) kann man dies so lange umformen, bis man

$$\beta^\ell(t) = \frac{1}{\ell!} \sum_{p=0}^{\ell+1} \binom{\ell+1}{p} (-1)^p \left(t - p + \frac{\ell+1}{2} \right)_+^\ell \quad \text{mit} \quad (x)_+ = \begin{cases} x, & \text{für } x \geq 0, \\ 0, & \text{sonst,} \end{cases} \quad (7.1.7)$$

erhält. Damit ist β^ℓ wirklich ein Polynom ℓ -ten Grades. Die Stetig- und Differenzierbarkeit muss man sich aber noch separat überlegen.

Weiterhin kann man zeigen, dass folgende Formeln für Differentiation und Integration von B-Splines gelten:

$$\left(\beta^\ell \right)'(t) = \beta^{\ell-1}(t + 1/2) - \beta^{\ell-1}(t - 1/2), \quad \int_{-\infty}^t \beta^\ell(s) ds = \sum_{p=0}^{+\infty} \beta^{\ell+1}(t - 1/2 - p). \quad (7.1.8)$$

Das heißt, dass man auch einen kompletten Spline s differenzieren und integrieren kann, indem man nutzt, dass sowohl Differentiation, als auch Integration lineare Operationen sind. Es gilt also mit (7.1.4) und (7.1.8) beispielsweise für die Differentiation, dass

$$s'(t) = \sum_{k \in \mathbb{Z}} c[k] \left(\beta^\ell \right)'(t - k) = \sum_{k \in \mathbb{Z}} c[k] \left(\beta^{\ell-1}(t + 1/2 - k) - \beta^{\ell-1}(t - 1/2 - k) \right). \quad (7.1.9)$$

Diese Eigenschaft macht man sich auch für kompliziertere Operationen, wie Rotationen und Verzerrungen auf einem Spline s zunutze.

7.2 Kubische B-Spline Interpolation

Wir möchten uns eine spezielle Version der B-Splines genauer Ansehen, da diese in der Anwendung den Spagat zwischen Komplexität und Approximationsgüte sehr gut hinbekommen. Wir setzen hierzu $\ell = 3$ und erhalten somit ein Polynom dritten Grades der Form

$$\beta^3(t) = \begin{cases} \frac{2}{3} - |t|^2 + \frac{|t|^3}{2} & \text{für } |t| < 1 \\ \frac{(2-|t|)^3}{6}, & \text{für } |t| \in [1, 2) \\ 0, & \text{für } |t| > 2, \end{cases} \quad (7.2.1)$$

welche in Codeschnipsel 39 auch einmal implementiert wurde.

In Analogie zum bekannten Nyquist-Sampling wollen wir untersuchen, wie wir aus endlich vielen gegebenen Abtastwerten $x[n]$ mit $n \leq N$ eine Darstellung wie in (7.1.4) herleiten können, welche die abgetasteten Werte exakt interpoliert. Aufgabe ist es also aus $x[n]$ die Folge $c[k]$ zu bestimmen.

Hierzu benötigen wir die sogenannte Interpolationsbedingung, siehe (5.2.1), welche für eine zu interpolierende Funktion x und ihre Abtastwerte $x[n] = x(n)$ fordert, dass

$$x(n) = x[n] = s(n) = \sum_{k \in \mathbb{Z}} c[k] \beta^3(n - k) \quad (7.2.2)$$

gilt. Wir fordern also *exakte* Interpolation. Nun könnte man für die Bestimmung der Folge $c[k]$ ein lineares Gleichungssystem aufstellen, welches die Form

$$\mathbf{B} \cdot \mathbf{c} = \mathbf{x} \quad (7.2.3)$$

```
def cubic_spline(t: np.ndarray) -> np.ndarray:
    s1: np.ndarray = (
        2.0 / 3.0
        - np.abs(t) ** 2
        + 0.5 * np.abs(t) ** 3
    )
    s2: np.ndarray = (2 - np.abs(t)) ** 3 / 6
    s3: np.ndarray = np.zeros_like(t)
    return (
        s3
        + s1 * (np.abs(t) < 1)
        + s2 * (np.abs(np.abs(t) - 1.5) <= 0.5)
    )
```

Codeschnipsel 39: Implementierung von (7.2.1), siehe [dsv/code/bsplines_eval.py](#)

hat, wobei die Systemmatrix B durch die Auswertung der B-Splines an den Stellen n bestimmt ist und der Vektor x den Werten $x[n]$ entspricht. Dem endlichen Träger der Funktionen β^ℓ ist es zu verdanken, dass die Matrix B mit nur wenigen von 0 verschiedenen Werten besetzt ist (genauer: band-diagonal) und demzufolge effizient invertierbar ist. Es ist also nicht „schwer“ $c = B^{-1}y$ zu berechnen. Doch die Anwendung von B^{-1} ist numerisch instabil, weshalb wir einen alternativen Weg einschlagen, der auf inverser Filterung beruht.

Sehen wir uns (7.2.2) genauer an. Wir finden, dass sich diese Gleichung nach Definition von $\beta[k] = \beta^3(k)$ als Faltung via

$$x[n] = (c * b)[n] \quad (7.2.4)$$

schreiben lässt. Nach Transformation in den z -Bereich erhalten wir

$$X(z) = C(z) \cdot B(z) \Rightarrow C(z) = \frac{X(z)}{B(z)}, \quad (7.2.5)$$

was uns motiviert eine Darstellung von $1/B(z)$ im Zeitbereich herzuleiten. Für die kubischen B-Splines folgt, dass

$$B(z) = \frac{z + 4 + z^{-1}}{6} \Rightarrow \frac{1}{B(z)} = 6 \left(\frac{1}{1 - z_1 z^{-1}} \right) \left(\frac{-z_1}{1 - z_1 z} \right) \quad (7.2.6)$$

gilt. Wobei man zeigen kann, dass $z_1 = \sqrt{3} - 2 < 1$ gilt. Wir betrachten nun $1/B(z)$ als einen Filter, der auf die Abtastwerte $x[n]$ angewandt werden soll. Aus (7.2.6) erkennen wir, dass $1/B(z)$ ein Filter ohne Nullstellen ist und als Hintereinanderausführung von zwei rekursiven Filtern betrachtet werden kann. Wir erhalten mit $c^-[k] = c[k]/6$, dass sich $1/B(z)$ durch

$$c^+[k] = x[n] + z_1 c^+[k-1] \quad \text{für } k = 1, \dots, N-1 \quad (7.2.7)$$

$$-c[k]/6 = c^-[k] = z_1 (c^-[k+1] - c^+[k]) \quad \text{für } k = N-2, \dots, 0 \quad (7.2.8)$$

ausdrücken lässt. Diese Methode der kausalen und anti-kausalen Filterung ist deutlich effizienter und stabiler, als (7.2.3), da beispielsweise keine Divisionen notwendig sind. Nun ist es noch notwendig Anfangswerte für $c^+[k]$ und $c^-[k]$ zu finden. Dies ist aufgrund der Endlichkeit von x nicht ohne Weiteres möglich. Man sieht, dass die Impulsantwort von c^+ eine abklingende Exponentialfunktion ist, also gilt

$$c^+[0] = \sum_{k=0}^{\infty} x[-k] z_1^k \approx \sum_{k=0}^K x[-k] z_1^k, \quad (7.2.9)$$

wobei man $K \in \mathbb{N}$ so wählen kann, dass $z_1^K \leq \varepsilon$ erfüllt ist, siehe Codeschnipsel 14. Nach Ausführung von c^+ kann man $c^-[N-1]$ durch

$$c^-[N-1] = \frac{z_1}{1 - z_1^2} (c^+[N-1] + z_1 c^+[N-2]) \quad (7.2.10)$$

effizient und exakt initialisieren. Beides wurde in Codeschnipsel 40 beispielhaft implementiert und eine mögliche Ausgabe nach Auswertung von (7.1.4) ist in Abbildung 15 dargestellt.

```

def filter_coeffs(x_n: np.ndarray) -> np.ndarray:
    z_1: float = np.sqrt(3) - 2.0
    c_k_p: np.ndarray = np.zeros_like(x_n)
    c_k_m: np.ndarray = np.zeros_like(x_n)

    c_k_p[0] = np.sum(
        x_n * np.power(z_1, np.arange(x_n.size))
    )

    for kk in range(1, x_n.size):
        c_k_p[kk] = x_n[kk] + z_1 * c_k_p[kk - 1]

    c_k_m[-1] = (
        z_1
        * (c_k_p[-1] + z_1 * c_k_p[-2])
        / (1.0 - z_1**2)
    )

    for kk in range(2, x_n.size + 1):
        c_k_m[-kk] = z_1 * (
            c_k_m[-kk + 1] + c_k_p[-kk]
        )

    return -6 * c_k_m

```

Codeschnipsel 40: Berechnung der B-Spline Koeffizienten, siehe [dsv/code/bsplines_coeffs.py](#)

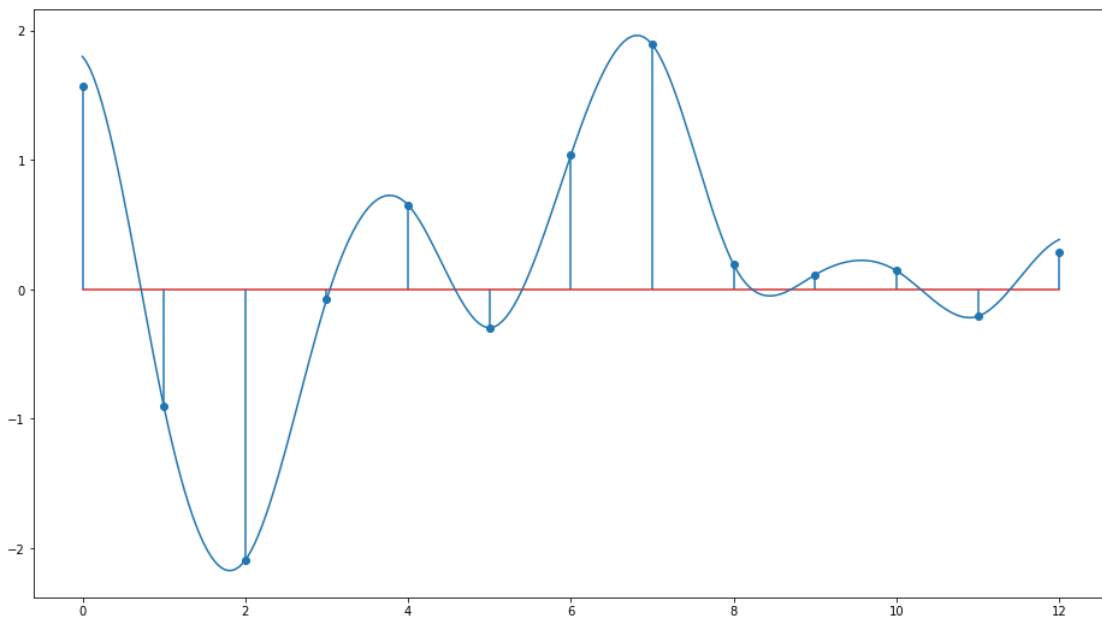


Abbildung 15: Kubische B-Spline-Interpolation für Abtastung an den Werten $n = 0, \dots, 12$.

7.3 Verbindung zur Nyquist-Sampling-Theorie

Wir wollen als Abschluss eine Verbindung zum Sampling und der Interpolation [1, Kapitel 6.1] von bandbegrenzten Funktionen mit endlicher Energie ziehen. Nehmen wir als Wiederholung zunächst an, dass das bandbegrenzte Signal x_a mit endlicher Energie und Fourier-Transformation X_a mindestens kritisch mit Rate F_s zu den Werten $x[n]$ abgetastet wurde. Wir bezeichnen mit X die DTFT von $x[n]$. Dann gilt als Zusammenhang zwischen den beiden Spektren, wie in Abschnitt 2.5, dass

$$X(f) = F_s \sum_{k=-\infty}^{+\infty} X_a((f-k)F_s), \quad (7.3.1)$$

was die Periodifizierung des Frequenzbereiches durch Abtastung ausdrückt. Nach Annahme der kritischen Abtastung findet hier kein Aliasing statt, sodass für $f \in [-F_s/2, +F_s/2]$ gilt, dass $F_s \cdot X_a(f) = X(f)$. Außerdem können wir das Spektrum der abgetasteten Werte X durch

$$X(f) = \sum_{n=-\infty}^{+\infty} x[n] \exp(-j2\pi f n / F_s) \quad (7.3.2)$$

ausdrücken. Nun können wir das analoge Signal x_a in Abhängigkeit von den Abtastwerten darstellen. Es gilt mit $T = 1/F_s$ und nach Theorem 2.1, dass

$$x_a(t) = \sum_{n=-\infty}^{+\infty} x[n] \frac{\sin(\pi(t-nT)/T)}{\pi(t-nT)/T}.$$

Das heißt, dass die Funktion $g : \mathbb{R} \rightarrow \mathbb{R}$ mit

$$g(t) = \frac{\sin(\pi t/T)}{(\pi t/T)} \quad (7.3.3)$$

als *Interpolations-Kernel* von bandbegrenzten und abgetasteten Funktionen betrachtet werden kann. Siehe Abschnitt 5.2.2 für eine Anwendung dieser Art der Interpolation.

Nun können wir eine analoge Rechnung für die B-Splines durchführen, indem wir einen Filter $b^{-1}[k]$ als inverse Z-Transformation von $1/B(z)$ aus (7.2.5) definieren. Dann gilt

$$c[k] = (b^{-1} * x)[k], \quad (7.3.4)$$

was wir in (7.2.2) einsetzen und dann

$$s(t) = \sum_{n \in \mathbb{Z}} (b^{-1} * x)[n] \beta^\ell(t-n) = \sum_{n \in \mathbb{Z}} x[n] \sum_{p \in \mathbb{Z}} b^{-1,\ell}[p] \beta^\ell(t-n-p) = \sum_{n \in \mathbb{Z}} x[n] h^\ell(t-n) \quad (7.3.5)$$

erhalten, wobei wir analog zu (7.3.3) den Interpolationskernel $h : \mathbb{R} \rightarrow \mathbb{R}$ durch

$$h^\ell(t) = \sum_{p \in \mathbb{Z}} b^{-1,\ell}[p] \beta^\ell(t-p) \quad (7.3.6)$$

definiert haben. Man kann zeigen, dass $\lim_{\ell \rightarrow \infty} h^\ell = g$. Das heißt, dass die sinc-Interpolation als Grenzwert der B-Spline Interpolation aufgefasst werden kann – oder andersherum – die B-Spline-Interpolation als Approximation der sinc-Interpolation. Das heißt, dass auch im Frequenzbereich Konvergenz in der Form

$$H^\ell(\omega) = \left(\frac{\sin(\omega/2)}{\omega/2} \right)^{\ell+1} \frac{1}{B^\ell(\exp(j\omega))} \xrightarrow{\ell \rightarrow \infty} \text{rect}(\omega) \quad (7.3.7)$$

gegeben sein muss.

Zusammenfassend kann man sagen, dass B-Splines einen alternativen Zugang zu digitaler Signalverarbeitung bieten, welcher eng mit dem des Nyquist-Samplings verknüpft ist und als Approximation von diesem gesehen werden kann. B-Splines sind wegen ihrer effizienten und stabilen Implementierung sowohl bei der Analyse (7.2.6), als auch der Synthese (7.1.4) vor allem für hochdimensionale Interpolationen sehr interessant¹⁹.

¹⁹<https://developer.nvidia.com/gpugems/gpugems2/part-iii-high-quality-rendering/chapter-20-fast-third-order-texture-filtering>

Akronyme

- ACF** Auto-Correlation Function. 83, 84, 88, 96, 97
- ADC** Analog-to-Digital Converter. 7
- AUT** Antenna Under Test. iv, 80
- BIBO** Bounded-Input-Bounded-Output. 29, 35, 61
- CMOS** Complementary Metal Oxide Semiconductor. 7
- CPU** Central Processing Unit. 57
- DFT** Discrete Fourier Transform. ii, v, 57, 61, 62, 64–68, 70, 72, 73, 75–77, 81, 83, 88, 97
- DTFT** Discrete Time Fourier Transform. 18, 19, 58, 59, 61, 62, 64, 112
- EADF** Effective Aperture Distribution Function. iv, 81
- FFT** Fast Fourier Transform. 66, 77, 81
- FIR** Finite Impulse Response. ii, iv, 36–39, 66
- FLOP** Floating Point Operation. 66, 83
- FT** Fourier Transform. 16–18, 47, 51, 53, 54, 58, 72, 103
- GPU** Graphical Processing Unit. 57
- iDFT** Inverse Discrete Fourier Transform. 64, 65
- iDTFT** Inverse Discrete Time Fourier Transform. 59
- iid** independent and identically distributed. v, 93, 95, 97
- IIR** Infinite Impulse Response. ii, iv, 36–39
- LFSR** Linear Feedback Shift Register. v, 86–88
- LTi** Linear Time-Invariant. i, ii, 5, 31–33, 35, 40, 43, 44, 61, 65, 78, 82, 89
- LTV** Linear Time-Variant. 36
- MIMO** Multiple Input Multiple Output. iv, 77
- MLBS** Maximum Length Binary Sequence. ii, v, 87, 88
- ROC** Region of Convergence. iv, 40–42, 45, 46, 61
- RX** receiver. 82
- STFT** Short Time Fourier Transformation. ii, v, 69, 70, 72–75
- TX** transmitter. 82

Literatur

- [1] J. Proakis und D. Manolakis. *Digital Signal Processing*. Pearson Deutschland, 2013. URL: <https://elibrary.pearson.de/book/99.150005/9781292038162> (siehe S. 8, 14, 20, 22, 31, 37, 41, 43, 45, 65, 86, 112).
- [2] T. E. Oliphant. *A guide to NumPy*. Trelgol Publishing USA, 2006 (siehe S. 22).
- [3] M. Frigo und S. Johnson. „The Design and Implementation of FFTW3“. In: *Proc. IEEE* 2 (2005). Special issue on “Program Generation, Optimization, and Platform Adaptation”. DOI: [10.1109/jproc.2004.840301](https://doi.org/10.1109/jproc.2004.840301). URL: <https://doi.org/10.1109/jproc.2004.840301> (siehe S. 77, 81).
- [4] G. Del Galdo Prof. Dr.-Ing. „Geometry-Based Channel Modeling for Multi-User MIMO Systems and Applications“. en. Diss. 2007. URL: https://www.db-thueringen.de/receive/dbt_mods_00011002 (siehe S. 77, 79).
- [5] A. Richter. „Estimation of Radio Channel Parameters“. In: (2005). URL: https://www.db-thueringen.de/receive/dbt_mods_00004815 (siehe S. 77).
- [6] M. Haardt, R. Thomä und A. Richter. „Multidimensional High-Resolution Parameter Estimation with Applications to Channel Sounding“. English. In: *High-Resolution and Robust Signal Processing*. Hrsg. von Y. Hua. 2003 (siehe S. 77).
- [7] S. Semper, M. Döbereiner, C. Steinmetz, M. Landmann und R. S. Thomä. „High-Resolution Parameter Estimation for Wideband Radio Channel Sounding“. In: *IEEE Transactions on Antennas and Propagation* 8 (2023). DOI: [10.1109/tap.2023.3286024](https://doi.org/10.1109/tap.2023.3286024) (siehe S. 78).
- [8] M. Landmann und G. D. Galdo. „Efficient antenna description for MIMO channel modelling and estimation“. In: *7th European Conference on Wireless Technology, 2004*. 2004 (siehe S. 79–81).
- [9] R. J. McEliece. *Finite Fields for Computer Scientists and Engineers*. Springer US, 1987. DOI: [10.1007/978-1-4613-1983-2](https://doi.org/10.1007/978-1-4613-1983-2). URL: <http://dx.doi.org/10.1007/978-1-4613-1983-2> (siehe S. 87).
- [10] B. Widrow und I. Kollár. *Quantization Noise: Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge University Press, 2008. URL: <https://books.google.de/books?id=8q-xcGeEJDwC> (siehe S. 98, 99, 104).
- [11] M. Unser. „Splines: a perfect fit for signal and image processing“. In: *IEEE Signal Processing Magazine* 6 (1999). DOI: [10.1109/79.799930](https://doi.org/10.1109/79.799930) (siehe S. 107, 108).
- [12] I. J. Schoenberg. „Contributions to the Problem of Approximation of Equidistant Data by Analytic Functions“. In: *I. J. Schoenberg Selected Papers*. Hrsg. von C. de Boor. Birkhäuser Boston, 1988. DOI: [10.1007/978-1-4899-0433-1_1](https://doi.org/10.1007/978-1-4899-0433-1_1). URL: https://doi.org/10.1007/978-1-4899-0433-1_1 (siehe S. 107).